Aajiveka — System Documentation

Codebase analysis of commit 8de48cf (branch main), October 2026.

1. Aajiveka — Project Overview
2. Project Structure
3. Role & Permission Matrix
4. Complete End-to-End Workflow
5. Module Dependencies
6. Database Schema
7. API Documentation
8. Frontend ↔ Backend Mapping
9. Testing Status & Issues
10. Master System Flow

Aajiveka — Project Overview

> Start here. This documentation set was produced by tracing the code at commit `8de48cf` (branch `main`, October 2026). It describes what the code does today, not what it is intended to do; where the two differ, the gap is recorded in TESTING_AND_ISSUES.md.

1. What the application is

Aajiveka is a **recruitment / applicant-tracking platform** for the Indian job market. It rebuilds the legacy **AajivikaPortal** (ASP.NET Web Forms + SQL Server) as a TypeScript monorepo. Its schema was derived from a restored copy of the legacy database (73 tables, 97 stored procedures).

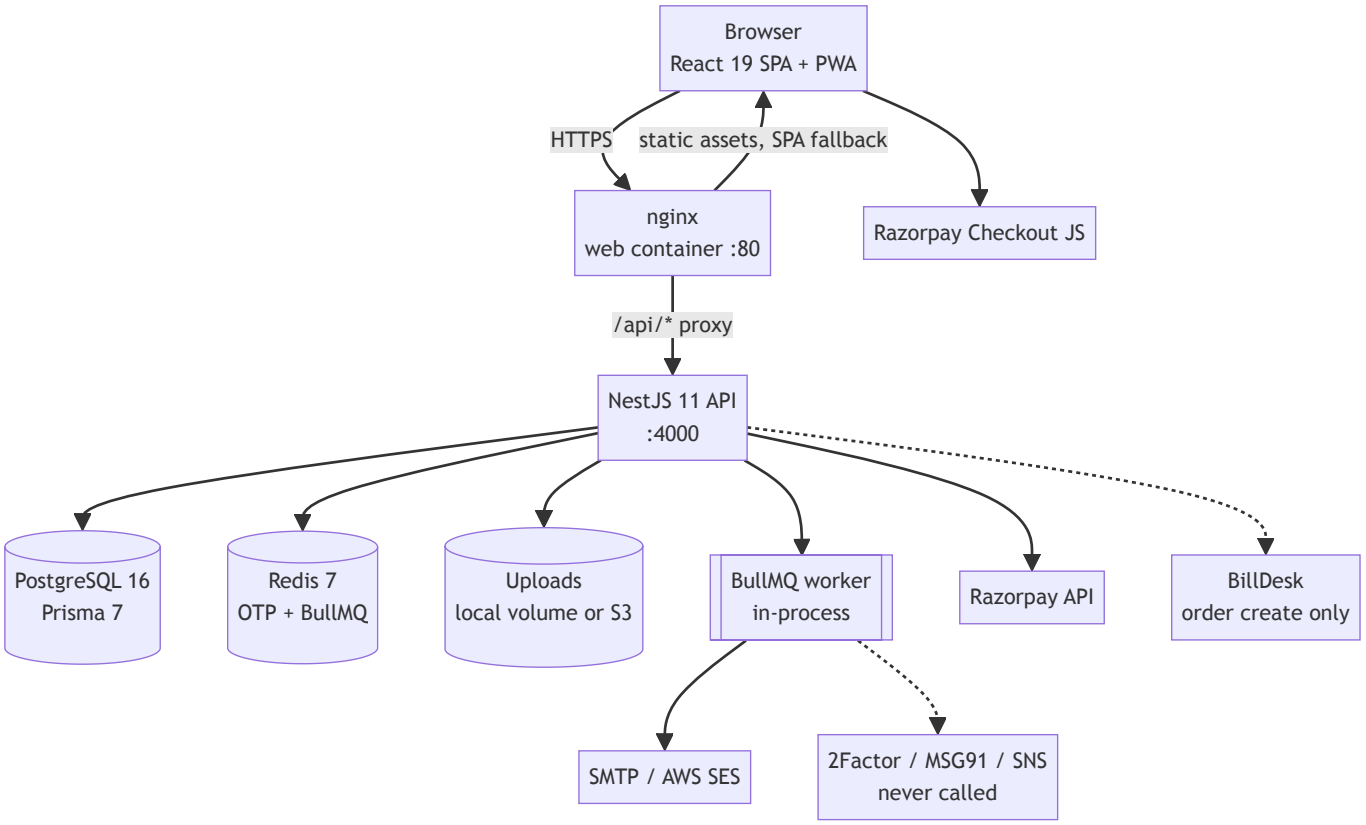
The business model:

- **Candidates** register, build a profile and CV, buy a time-limited subscription, and apply to jobs.
- **Employers** register, are approved by an admin, post jobs and run their own selection.
- An internal staff pipeline sits between the two:
  - **Q1** screens candidate profiles.
  - **Q2** scores each application against the job and refers the best CVs.
  - **Q3** forwards CVs to the company, coordinates interview rounds, collects documents and issues the offer through to joining.
- **Admins** approve employers, manage users and moderate jobs.

2. Documentation map

Document	Answers
PROJECT_OVERVIEW.md	What it is, architecture, technology, environments (this file)
PROJECT_STRUCTURE.md	Which directory or file does what
ROLE_PERMISSION_MATRIX.md	Who can do what; implemented vs not-implemented roles
COMPLETE_WORKFLOW.md	Every business workflow end to end, with sequence diagrams and the status state machine
MODULE_DEPENDENCIES.md	Backend/frontend modules, how they depend on each other, implementation status
DATABASE_SCHEMA.md	Models, relationships, ERD, status vocabularies, data lifecycle, migrations
API_DOCUMENTATION.md	Every endpoint: method, path, auth, roles, payloads, errors, integrations
FRONTEND_BACKEND_MAPPING.md	Page → component → API → service mapping; dead UI
TESTING_AND_ISSUES.md	What was tested, coverage gaps, classified findings, runtime checks still needed
MASTER_SYSTEM_FLOW.md	One master diagram plus focused diagrams for complex flows

3. Architecture



- **Single API, single SPA.** The browser reaches the API only through nginx at `/api`, and the API container is never published (`docker-compose.yml`, `deploy/nginx.conf`). In development, Vite proxies `/api` to `localhost:4000`.
- **Authentication.** Short-lived JWT access tokens (15 min, held in memory) are paired with rotating refresh tokens (7 days, localStorage, allow-listed in `tblSecActiveSessions`). Separate secrets sign the two token types.
- **Authorization.** Global guards make every route authenticated by default (`@Public()` opts out) and restrict it by role id (`@Roles(...)`). Ownership checks live in services.
- **Background work.** There are two BullMQ queues on Redis: `notifications` (email/SMS delivery with retries) and `cv-expiry` (hourly job that expires CVs a company has not reviewed in 14 days).
- **Legacy compatibility.** Table and column names keep their SQL Server spelling (`tblXxx`, PascalCase) via Prisma `@@map` / `@map`, and role ids and status ids match the legacy seed data.

4. Technology stack

Layer	Technology
Frontend	React 19, Vite 7, TypeScript (strict), Tailwind CSS v4, Radix UI (shadcn structure), TanStack Query, Zustand, React Hook Form + Zod, react-router (data router, lazy routes), react-i18next, vite-plugin-pwa, MSW (dev/CI mocks), Storybook
Backend	NestJS 11 (Express 5), Prisma 7 with `@prisma/adapter-pg`, PostgreSQL 16, Redis 7 + ioredis, BullMQ, Argon2id, class-validator, Zod (env), Swagger, helmet, @nestjs/throttler, exceljs, pdfkit, nodemailer, AWS SDK v3 (S3, SES, SNS)
Payments	Razorpay (active), BillDesk ve1_2 JWS (legacy, incomplete)
Infra	Docker Compose (postgres, redis, api, web/nginx), AWS RDS + S3 + SNS in the prod overlay, EC2 deploy over SSH
CI	GitHub Actions: typecheck, lint, unit tests, Playwright (mocked), real-DB e2e, Docker build, deploy
Tests	node:test + tsx (API), Vitest + Testing Library (web), Playwright (mocked browser tests and real-API suites)

5. Users at a glance

Role (id)	Lands on	Main job
Candidate (1)	`/candidate/profile`	Build profile, subscribe, apply, attend interviews, upload documents, accept offer
QC1 (2)	`/q1/dashboard`	Screen candidate profiles

Role (id)	Lands on	Main job
QC2 (3)	`/q2/dashboard`	Score and match applications; refer to Q3
Employer (4)	`/company`	Post jobs, review applicants and forwarded CVs, run interview rounds
Admin (5)	`/admin`	Approve employers, manage users, moderate jobs, reports, audit
Subscription (6)	`/pricing`	**Defined but not implemented**
Q3 (7)	`/q3/dashboard`	Forward CVs, coordinate interviews, collect documents, issue offers, mark joined

Details: ROLE_PERMISSION_MATRIX.md.

6. End-to-end business flow (summary)

1. **Candidate onboarding**: email-OTP registration → onboarding wizard → profile and CV.
2. **Employer onboarding**: registration → admin approval (creates the company, login and temporary password) → job posting (Active immediately).
3. **Subscription**: the candidate buys a plan via Razorpay. This unlocks applying.
4. **Apply**: creates an application (status 1 MAPPED).
5. **Q1** screens the profile (advisory). **Q2** scores the application and forwards it to Q3 (14), sends it back to Q1 (11) or rejects it (5).
6. **Q3** validates and forwards the CV to the company (15). The company has 14 days to respond, or the CV expires (23) and returns to Q2.
7. **Company shortlist**: interview rounds R1 → R2 → R3 → Final (16–19), scheduled and confirmed by Q3.
8. **Selected (4)** → documents requested, verified, forwarded and approved → **Offer sent (20)** → **Accepted (21)** → **Joined (22)**.

Full detail with diagrams: COMPLETE_WORKFLOW.md and MASTER_SYSTEM_FLOW.md.

7. Environments and configuration

Environment	How it runs
Local dev	`docker compose -f docker-compose.yml -f docker-compose.dev.yml up -d postgres redis`, then `npm run dev` (API :4000, web :5173 proxied, BillDesk stub :4999)
Single-host Docker	`cp .env.example .env` → `docker compose up -d --build` → app on :8080. Migrations apply on API start. Seed masters with `npx tsx prisma/seed.ts`. First admin via `scripts/create-admin.ts`
Production overlay	`docker compose -f docker-compose.yml -f docker-compose.prod.yml up -d --build`: RDS (`RDS_HOST`), S3 storage, SNS SMS, SMTP/SES email, web on :80
CI deploy	Push to `main` → all CI jobs → SSH to EC2 → `git pull` → `docker compose up -d --build`

Environment variables (names only; values are secrets and are not reproduced here). The authoritative schema is `apps/api/src/config/env.ts`.

Group	Variables
Core	`NODE_ENV`, `PORT`, `CORS_ORIGIN`, `APP_URL`, `THROTTLE_LIMIT_PER_MIN`
Data	`DATABASE_URL`, `SHADOW_DATABASE_URL` (dev), `REDIS_URL`, `POSTGRES_USER`, `POSTGRES_DB`, `POSTGRES_PASSWORD`, `RDS_HOST`
Auth	`JWT_ACCESS_SECRET`, `JWT_REFRESH_SECRET`, `JWT_ACCESS_TTL`, `JWT_REFRESH_TTL`
Storage	`STORAGE_DRIVER`, `STORAGE_LOCAL_ROOT`, `S3_BUCKET`, `AWS_REGION`
Email	`EMAIL_DRIVER`, `SMTP_HOST`, `SMTP_PORT`, `SMTP_USER`, `SMTP_PASSWORD`, `SMTP_SECURE`, `SMTP_FROM`, `SMTP_FROM_NAME`, `SES_FROM_EMAIL`
SMS	`SMS_DRIVER`, `TWOFACTOR_API_KEY`, `MSG91_AUTH_KEY`, `MSG91_TEMPLATE_ID`, `SNS_SENDER_ID`, `SNS_DLT_ENTITY_ID`, `SNS_DLT_TEMPLATE_ID`
Payments	`RAZORPAY_KEY_ID`, `RAZORPAY_KEY_SECRET`, `RAZORPAY_WEBHOOK_SECRET`, `BILLDESK_BASE_URL`, `BILLDESK_CLIENT_ID`, `BILLDESK_MERCHANT_ID`, `BILLDESK_SECRET_KEY`
Web build	`VITE_API_BASE_URL`, `VITE_USE_MOCKS`, `VITE_OAUTH_ENABLED`, `VITE_ANALYTICS_ENDPOINT`
Scripts	`SEED_DEMO_USERS`, `MSSQL_SA_PASSWORD`

Configuration gaps found:

- No compose file forwards the `RAZORPAY_*` variables.
- The base compose file does not forward `EMAIL_DRIVER` / `SMS_DRIVER`, so it only logs email.

See TESTING_AND_ISSUES P-01 and I-01.

8. Implementation status

Area	Status
Auth (email OTP, JWT rotation, password reset)	Implemented
Candidate portal (profile, CV, applications, interviews, documents, offers)	Implemented; some sections have no editor UI
Job search and apply	Implemented (subscription-gated)
Employer portal (jobs, applicants, rounds, documents, analytics, billing view)	Implemented; team, messaging and settings are placeholders
Q1 / Q2 / Q3 staff workspaces	Implemented; hand-off and decision logic inconsistent across paths
Admin panel	Implemented (read-only over pipeline by design); settings not enforced
Payments	Razorpay implemented; BillDesk incomplete; no receipts
Email	Implemented (log driver by default)
SMS, in-app notifications	Infrastructure only, never triggered
Exports	Implemented; two filter bugs
Legacy `/recruitment/*` screens	Still routed alongside the new workspaces; Q3's legacy menu is broken

9. Key risks (top 8)

Details and evidence are in TESTING_AND_ISSUES.md.

1. Any signed-in user can download any stored file by key (S-01). An employer can expose arbitrary files through the public logo route (S-02).
2. Employer document access is scoped to the candidate rather than to the application (S-03).
3. Razorpay is not configured in any Docker deploy, and BillDesk orders can never settle (P-01, P-02).
4. Two interview-decision engines (employer/recruitment vs Q3) write different fields. After an employer hire, the Q3 documents/offer flow never starts (W-01).
5. Status transitions have no from-state guard, and an unaudited "set any status" endpoint exists (W-02).
6. Employer registration fails when HR email is blank (F-01). Staff cannot log in by email (A-01).
7. Q1/Q2 CV download returns 401 (F-03). Q3's legacy menu screens return 403 (F-02).
8. There are no automated tests for payments, the employer lifecycle or the Q2/Q3 transition services.

Project Structure

> What each directory and key file is responsible for. Paths are repo-relative, commit `8de48cf`. Generated/vendored directories (`node_modules`, `dist`, `playwright-report`, `test-results`, `apps/api/storage`) are not described.

1. Top level

```
Aajiveka-Aplication/
├── apps/
│   ├── api/                NestJS 11 API (Prisma 7, PostgreSQL 16, Redis/BullMQ)
│   └── web/                React 19 SPA (Vite 7, Tailwind v4, TanStack Query, Zustand)
├── db/                     Restored legacy SQL Server database artefacts (reference only)
├── deploy/                 nginx config + deployment guide
├── tests/                  Real-API end-to-end suites + BillDesk payment-gateway stub
├── reference/              Extracted legacy ASP.NET sources (git-ignored)
├── scripts/                (empty)
├── .github/workflows/ci.yml CI: static checks, unit tests, Playwright, real-DB e2e, docker build, deploy
├── docker-compose.yml      Base stack: postgres, redis, api, web (nginx on :8080)
├── docker-compose.dev.yml  Publishes postgres :15432 and redis :16379 for host-run development
├── docker-compose.prod.yml Production overlay: RDS instead of postgres, S3 storage, SNS SMS, SMTP/SES email
├── .env.example            Names of the env vars compose needs (copy to .env; .env is git-ignored)
├── package.json            npm workspaces (apps/web, apps/api); scripts dev / build / lint / typecheck
├── README.md, CONTRIBUTING.md
└── docs/                   This documentation set
```

Untracked files present in the working tree: `Aajivika Book.pdf` (company brochure, untracked), `dump.rdb` (local Redis snapshot, git-ignored), `.env` (git-ignored, not read for this analysis).

2. `apps/api` — backend

```
apps/api/
├── Dockerfile, docker-entrypoint.sh   Build + run; entrypoint runs `prisma migrate deploy` then `node dist/main.js`
├── prisma.config.ts                   Prisma 7 config (DATABASE_URL, optional SHADOW_DATABASE_URL)
├── prisma/
│   ├── schema.prisma                  94 models + PaymentStatus enum, legacy table names via @@map
│   ├── migrations/                    34 migrations (Jul–Oct 2026)
│   ├── seed.ts                        Masters, plans, demo users (SEED_DEMO_USERS=1)
│   ├── migrate-data.ts                One-off legacy SQL Server → Postgres copy (destructive)
│   ├── data/                          India education / institute master data
│   └── tools/build-education-sql.ts   Generator for education master SQL
├── scripts/
│   ├── create-admin.ts                Bootstrap the first admin (prints a set-password link)
│   ├── publish-site-document.ts       Publish a downloadable site document (tblSiteDocument)
│   ├── seed-demo-applicants.ts        Demo data helpers
│   └── seed-applicant-profile-resume.ts
└── src/
    ├── main.ts                        Bootstrap: /api prefix, helmet, CORS, ValidationPipe, Swagger /api/docs, BigInt JSON
    ├── app.module.ts                  Module wiring + global guards (Throttler, JwtAuth, Roles) + request logger
    ├── config/env.ts                  Zod-validated environment (fails fast)
    ├── prisma/                        PrismaService / client factory (@prisma/adapter-pg), global module
    ├── common/
    │   ├── guards/                    jwt-auth.guard.ts, roles.guard.ts
    │   ├── decorators/                @Public, @Roles, @CurrentUser
    │   ├── filters/                   AllExceptionsFilter (error shape)
    │   ├── middleware/                RequestLoggerMiddleware
    │   └── email/                     EmailService + HTML templates (OTP, welcome, reset, status, interview, Q3 notice, job alert,
receipt)
    ├── shared/
    │   ├── roles.ts                   Role ids 1–7
    │   ├── status.ts                  JobMapStatus 1–23, job status 1–4, SubscriberStatus
    │   ├── hiring.ts                  Q3 vocabulary, IST date maths, stage derivation (stageOf)
    │   ├── matching.ts                Q2 criteria, weights, weightedScore, relevance threshold
    │   ├── screening.ts               Q1 statuses, priority, checklist
    │   └── pdf.ts                     PDF page counter
    └── modules/
        ├── auth/                      Login, refresh, OTP registration, employer registration, password reset, CSRF token, Redis
provider
        ├── candidates/                Candidate profile, sections, documents, saved jobs, alerts, prefs, referrals, privacy
        ├── files/                     Upload/download endpoints (resume, avatar, generic)
        ├── storage/                   StorageService + local / S3 drivers
        ├── jobs/                      Public search, job detail, apply; JobApplicationsService (apply + transitionStatus)
        ├── employers/                 Employer portal API (`/clients`): company, jobs, applicants, rounds, docs, analytics, billing
        ├── clients/                   Deprecated re-exports of employers (not imported)
        ├── companies/                 Public company search/page
        ├── recruitment/               Legacy QC pipeline + shared hiring services: referrals, interview rounds, offers, scoring,
mapping access, CV-expiry processor
        ├── q1/                        Q1 screening workspace + analytics
        ├── q2/                        Q2 matching workspace + analytics
        ├── q3/                        Q3 hiring-ops workspace: read models, referrals, interviews, documents, mail
        ├── admin/                     Admin panel API + staff account creation
        ├── payments/                  Razorpay (active), BillDesk (legacy), plans, subscriptions
        ├── exports/                   CSV / XLSX / PDF exports
        ├── notifications/             BullMQ queue + worker, SMS/email providers, SSE in-app notifications
        ├── audit/                     AuditService (tblAuditLog, tblSecUserLogin)
        ├── site-documents/            Public downloadable documents
        └── health/                    /api/health
```

Convention: each module has `*.module.ts`, `*.controller.ts`, `*.service.ts`, `dto/*.dto.ts` (class-validator), optional `*.test.ts` (node:test).

3. `apps/web` — frontend

```
apps/web/
├── Dockerfile                Build with VITE_USE_MOCKS=0, VITE_API_BASE_URL=/api; serve with nginx (deploy/nginx.conf)
├── vite.config.ts            Vite + PWA plugin (manifest, shortcuts) + dev proxy /api → :4000
├── vitest.config.ts          Unit test config (jsdom, src/test-setup.ts)
├── playwright.config.ts      Mocked-API browser tests (e2e/)
├── vercel.json               SPA rewrite config (alternative hosting)
├── public/                   favicon, manifest.json, offline.html, robots/sitemap, locales, images, templates, mockServiceWorker.js
├── e2e/                      43 Playwright specs + support/ (auth helpers, fixtures, mocks)
└── src/
    ├── main.tsx              Entry: optional MSW (VITE_USE_MOCKS=1), providers, router, PWA prompts, offline sync
    ├── app/providers.tsx     QueryClient, Toast, auth bootstrap
    ├── routes/
    │   ├── router.tsx        All routes, lazy-loaded; role groups via ProtectedRoute
    │   └── ProtectedRoute.tsx Auth / role / onboarding guard
    ├── types/roles.ts        Role ids, labels, ROLE_HOME
    ├── lib/                  axios (interceptors), tokenStorage, queryClient (+ query keys), env, offlineQueue, analytics, ga4,
webVitals, errorTracking, sanitize, i18n, theme
    ├── hooks/                useSSE, useRealtimeNotifications, useOfflineSync, form/UX hooks
    ├── components/
    │   ├── ui/               Design-system primitives (Button, Input, Select, Table, Modal, Toast, searchable selects…)
    │   ├── layout/           PublicLayout, DashboardLayout, Navbar, Sidebar, menu.ts (role menus), NotificationBell
    │   └── *.tsx             Cross-cutting: CommandPalette, Seo, ErrorBoundary, PWA prompts, OfflineBanner, etc.
    ├── features/             Feature-sliced UI (pages + components + api hooks per domain)
    │   ├── auth/             Login (role "portals"), register + OTP, employer register, forgot/reset, auth.store (Zustand)
    │   ├── home/             Home page + /brochure (page-for-page brochure rebuild)
    │   ├── public/           About, Contact, Pricing, Blogs, Help, Legal, Career, Student guide, Salary insights, Company reviews…
    │   ├── jobs/             JobsLayout, search, detail, apply, search bar
    │   ├── candidates/       Candidate portal (portal/: layout, profile, wizard, tracker, interviews, documents, refer, account,
prefs, resume builder) + legacy pages (many unrouted)
    │   ├── payments/         Pricing plans, Razorpay checkout, membership, payment return
    │   ├── clients/          Public company page; deprecated client API + orphan components
    │   ├── recruitment/      Legacy QC / Q3 screens (/recruitment/*)
    │   ├── q1/  q2/  q3/     Staff workspaces (own shells, api hooks, components, formatters)
    │   ├── admin/            Admin panel pages + AdminShell + primitives
    │   ├── notifications/, messaging/, dashboard/, misc/, dev/   Mostly unrouted or placeholder (NotFound lives in misc/)
    ├── employer/             Employer portal: routes/, layouts/ (EmployerLayout), pages/ (Dashboard, ManageJobs, Applicants,
CompareCandidates, CompleteProfile, Analytics, Billing, Messages*, Notifications*, Settings*), services/, hooks/, components/
(*placeholders)
    ├── mocks/                MSW handlers + fixture data (dev/CI only)
    ├── styles/               Tailwind entry + animations
    ├── test/, test-setup.ts, test-utils/   Test setup and helpers
```

4. `db/` — legacy database reference (read-only)

Path	Content
`db/schema.sql`, `db/types.sql`, `db/indexes.sql`	Restored SQL Server DDL: 73 tables, 10 table-valued types
`db/procs/`	97 stored procedures (the legacy business logic the rebuild was derived from)
`db/procs_damaged_strings_extraction/`	Procedures recovered by string extraction
`db/seed/*.psv`	Master/lookup data (statuses, roles, cities, skills, documents, …) loaded by `seed.ts`
`db/ER-DIAGRAM.md`, `db/SCHEMA_REPORT.md`	Legacy model and inventory (partly stale; see DATABASE_SCHEMA)
`db/foreign-keys.psv`	Derived relations with confidence levels (used by `migrate-data.ts`)
`db/dead-tables.txt`	13 legacy tables omitted from the new schema
`db/columns.psv`, `db/tools/`	Column inventory and the Prisma-schema generator

5. `deploy/` and `tests/`

Path	Purpose
`deploy/nginx.conf`	Web container: serves the SPA, proxies `/api/` → `api:4000`, caching and security headers
`deploy/DEPLOYMENT.md`	Secrets, storage, payments, legacy migration, operations

Path	Purpose
`tests/e2e.mjs`	Real-API crawl of every public page and each role's pages
`tests/admin-e2e.mjs`	Real-API admin panel suite incl. responsive checks
`tests/billdesk-stub.mjs`	Local BillDesk gateway stand-in (`npm run dev:billdesk`)

6. Where to start reading

Goal	Start at
Understand roles and access	`apps/api/src/shared/roles.ts` , `apps/api/src/common/guards/*` , `apps/web/src/routes/router.tsx`
Understand the hiring pipeline	`apps/api/src/shared/status.ts` , `apps/api/src/modules/jobs/job-application.service.ts` (`transitionStatus`), `modules/q2` , `modules/q3` , `modules/recruitment/cv-referral.service.ts`
Understand data	`apps/api/prisma/schema.prisma` , DATABASE_SCHEMA.md
Run it	`README.md` , `CONTRIBUTING.md` , `deploy/DEPLOYMENT.md`

Role & Permission Matrix

> Derived from `apps/api/src/shared/roles.ts`, every `@Roles(...)` / `@Public()` decorator in `apps/api/src/modules/**`, the web route guards in `apps/web/src/routes/router.tsx` and `apps/web/src/employer/routes/`, and `ROLE_HOME` in `apps/web/src/types/roles.ts`. Commit `8de48cf`.

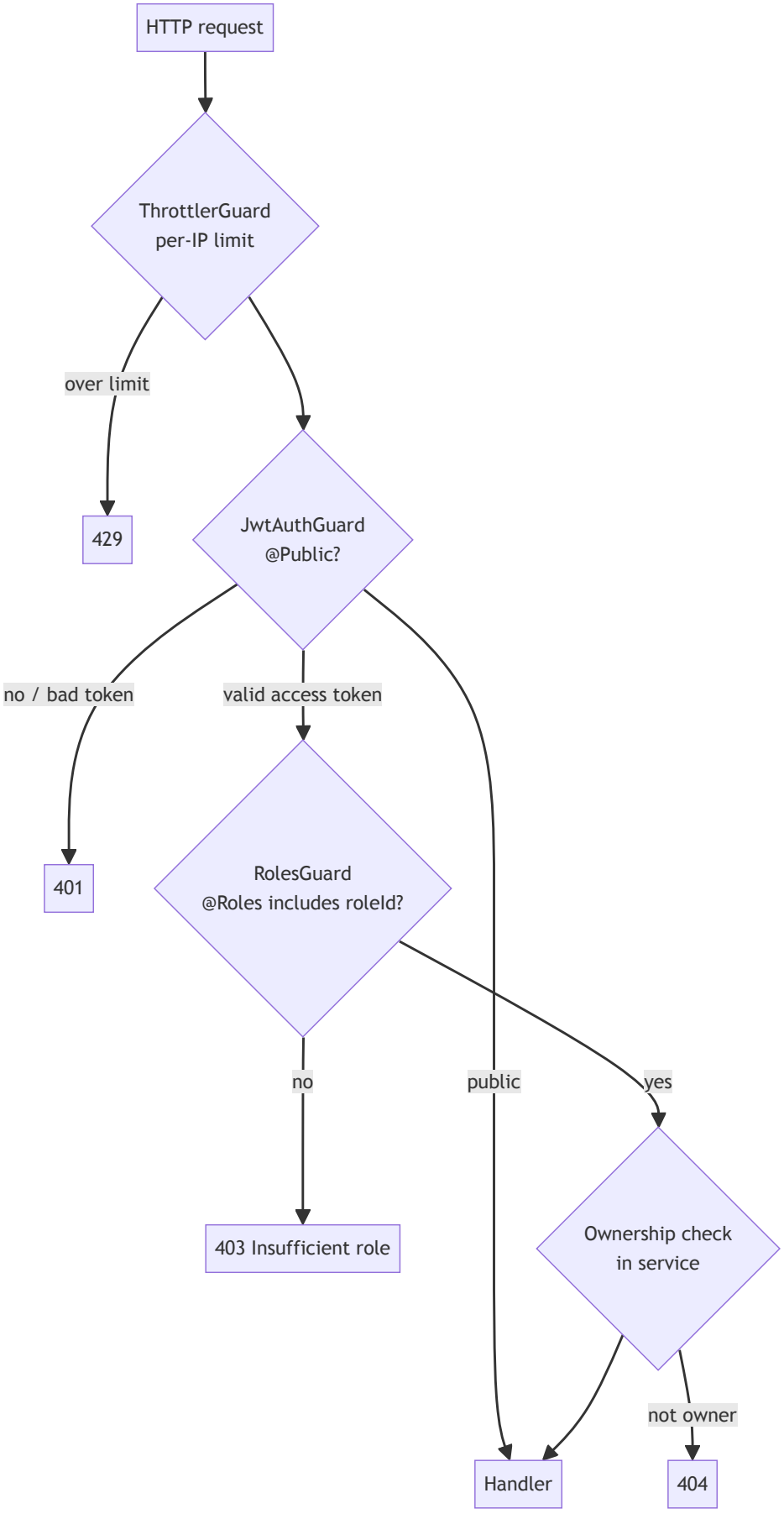
1. Roles that exist in code

Id	Code name	UI label	Implemented?	Home route after login	How the account is created
1	`Subscriber`	Candidate	**Yes**	`/candidate/profile` (→ `/candidate/onboarding` until onboarded)	Self-registration with email OTP (`POST /api/auth/register` → `/verify-otp`)
2	`QC1`	QC1 (Q1 screening)	**Yes**	`/q1/dashboard`	Admin creates staff login (`POST /api/admin/users`) or seed
3	`QC2`	QC2 (Q2 matching)	**Yes**	`/q2/dashboard`	Admin / seed
4	`Client`	Employer	**Yes**	`/company` (employer portal)	Public employer registration → **Admin approval** creates the login and emails a temporary password
5	`Admin`	Admin	**Yes**	`/admin`	Admin, seed, or CLI `apps/api/scripts/create-admin.ts`
6	`Subscription`	Subscription	**Not implemented** — the id exists, `ROLE_HOME` points to `/pricing`, but no API route references it and payment never assigns it	`/pricing`	Never assigned
7	`Q3`	Q3 (interview coordination / hiring ops)	**Yes**	`/q3/dashboard`	Admin / seed

Demo accounts (dev/CI only, created by `apps/api/prisma/seed.ts` with `SEED_DEMO_USERS=1`): `anuj` (candidate), `qc1`, `qc2`, `anuj@aajiveka.com` (employer), `admin`, `q3` (`seed.ts:190-197`). Passwords are not reproduced here. Staff/admin logins are created with `NodeType 100`, which the email-login path excludes (`auth.service.ts:197`), so **staff can log in by username but not by email** (employers use NodeType 200 and are unaffected).

Proposed / implied but not implemented: company sub-users or team members (one login per company only); a "Subscription" membership role; candidate-facing "Recruiter" messaging (privacy flag `allowRecruiterMessages` exists, no messaging module is routed).

2. How access is enforced



- **Server**: global guards (`apps/api/src/app.module.ts`). Ownership is checked inside services: candidates by `tblSecUser.SubscriberID` ; employers by `tblSecUser.NodeID → tblMstrPerson.ClientID` (`clientIdFor`); shared hiring routes

by `MappingAccessService` (checks Candidate and Client only — staff roles pass through).

- **Client**: `ProtectedRoute` (`apps/web/src/routes/ProtectedRoute.tsx`) redirects anonymous users to `/login`, wrong roles to their `ROLE_HOME`, and non-onboarded candidates to `/candidate/onboarding`. Client guards were deliberately aligned to the server's `@Roles` lists (commit `2eb4db7` referenced in router comments).
- **Token lifetime caveat**: the access token's `roleId` is trusted for its full TTL (default 15 min). Deactivation or a role change takes effect for API calls only after the access token expires.

3. Responsibility matrix (who does what)

Legend: **C** create · **R** read · **U** update · **D** delete · **A** approve/decide · — no access. "own" = restricted to the user's own records.

Capability	Candidate (1)	QC1 (2)	QC2 (3)	Employer (4)	Q3 (7)	Admin (5)	Public
Register / log in	C (self)	login	login	C registration (pending) → login after approval	login	login	browse
Own profile & CV	CRUD own	R all candidates	R all	R applicants of own jobs	R referred	R all	—
Upload resume / photo / requested docs	C own	—	—	logo only	—	—	—
Browse / search jobs	R	—	R active jobs	R own jobs	—	R all	R active (and any job by id)
Apply to job	C (needs active subscription)	C on behalf (assign-job)	C on behalf	—	—	C on behalf	—
Post / edit / close / archive / delete jobs	—	—	—	CRUD own	—	Close / Reopen only	—
Q1 screening (verify, follow-up, no-response, not-interested, checklist, contact log)	—	**A**	—	—	—	A	—
Registration decision (Approved/Rejected/OnHold/…)	—	**A**	—	—	—	A	—
Match scoring & Q2 decisions (forward to Q3, send back to Q1, reject)	—	score only	**A**	—	—	A	—
Refer CVs to Q3	—	—	C	—	—	C	—
Forward CVs to company / return to Q2	—	—	—	—	**A**	A	—
Company review of forwarded CV (shortlist / reject)	—	—	—	**A** own	capture on company's behalf (round-setup)	A (only if linked to a company — effectively 404)	—
Employer decision on any own applicant (Shortlisted / Hired / Rejected / undo)	—	—	—	**A** own	—	404 (no company)	—
Create interview rounds, slots	—	—	—	C own	C (any mapping via /recruitment)	C	—
Confirm slot / send invites	pick own slot	—	—	—	**A**	A	—
Record round result / decision	—	—	—	**A** own (submitResult)	**A** (Q3 decide)	A	—
Request / verify / send back documents	upload own	verify/reject any (legacy)	assign doc types; verify (legacy)	request & review own applicants	**A** request, verify, forward, approve	A	—
Create / send offer	—	—	—	C own	**A**	A	—
Accept / reject offer	**A** own	—	—	—	—	A (on behalf)	—
Mark joined	—	—	—	U own	**A**	A	—
Legacy interviews (tblJobInterviewStatus)	—	C/U	C/U	—	—	C/U	—
Employer registration approval	—	—	—	—	—	**A**	—

Capability	Candidate (1)	QC1 (2)	QC2 (3)	Employer (4)	Q3 (7)	Admin (5)	Public
User management (create staff, active flag, role, reset link)	—	—	—	—	—	**CRUD**	—
Platform settings	—	—	—	—	—	R/U (not enforced anywhere)	—
Blog CMS	—	—	—	—	—	CRUD	R (public pages)
Audit log	—	—	—	—	—	R	—
Exports	—	candidates	candidates	applicants (10-row bug)	—	users, jobs, applications, payments, candidates	—
Analytics	own dashboard	Q1 analytics	Q2 analytics	own company analytics	Q3 dashboard & quarterly report	platform stats	—
Payments / subscription	C/R own	—	—	R own invoices (no online payment)	—	export	R plans
Notifications (SSE / in-app)	R (always empty)	R	R	R	R	R	—

4. Per-role detail

4.1 Candidate (Subscriber, 1)

- **Pages**: public site, `/jobs`, `/jobs/:id`, `/jobs/:id/apply`, `/candidate/applications`, `/candidate/saved-jobs`, portal (`/candidate/profile`, `/onboarding`, `/tracker`, `/interviews`, `/documents`, `/referrals`, `/account`, `/notifications`, `/all-notifications`, `/resume-builder`), `/candidate/subscription`.
- **Can**: build profile (personal, professional, education, employment, certificates, skills, projects, languages, etc.), upload resume/photo, save jobs, set alerts and saved searches (stored only), pay for a subscription (Razorpay), apply, pick an interview slot (shared `/recruitment/interview-rounds/:roundId/select-slot`), upload requested documents, accept/reject an offer, refer friends (stored only), change password, request data export / account deletion (recorded only).
- **Cannot**: see other candidates; change application status (except offer response); withdraw an application (no endpoint).
- **Depends on**: QC1/QC2/Q3/Employer to move applications; Admin for nothing directly.
- **Restrictions**: must complete onboarding before any other candidate route; apply needs an active subscription; legacy (migrated) candidates are not linked to a profile and get 404 on all `/candidates/me*` routes.

4.2 QC1 (2) — Q1 screening

- **Pages**: `/q1/*` (dashboard, candidates, candidate profile, follow-ups, no-response, verified, not-interested, analytics). Also legacy `/recruitment/candidates`, `/recruitment/candidates/:id`, `/recruitment/documents`, `/recruitment/interviews`, `/recruitment/qc1`.
- **Can**: work the screening queue, open a profile (New → Screening), tick the 7-item checklist, log contacts (no message is actually sent), set FollowUp/NoResponse/NotInterested, mark Verified, make registration decisions, compute match scores, assign a candidate to a job, schedule/close legacy interviews, list/download/verify any candidate document, export candidates.
- **Status changes**: screening status (string state); registration `flgstatus`; legacy interview statuses 3/6/9; any status via the unused `PATCH /recruitment/pipeline/:id`.
- **Hand-off**: Q1 "Verified" is **not enforced** as a gate — Q2 sees all applications on active jobs regardless.

4.3 QC2 (3) — Q2 matching

- **Pages**: `/q2/*` (dashboard, employer jobs, job applicants, all applicants, applicant detail, forwarded, sent-back, analytics); legacy `/recruitment/*` incl. `/recruitment/referrals`.
- **Can**: view ranked applicants per job (scores are computed lazily), toggle score criteria, **forward to Q3** (→ 14 REFERRED_TO_Q3), **send back to Q1** (→ 11 NEED_MORE_INFO, screening → Incomplete), **reject** (→ 5), assign document types, legacy interview and document actions.
- **Depends on**: Q1 (advisory), Q3 (returns CVs with a reason; auto-expiry returns them after 14 days).

4.4 Employer (Client, 4)

- **Pages**: employer portal under `/company/*` (see FRONTEND_BACKEND_MAPPING). Public: `/register/employer`.

- **Can**: manage company profile/logo, post/edit/close/archive/reactivate/duplicate/delete own jobs (delete only without applications), CSV bulk-upload jobs, view **all** applicants on own jobs with contact details, shortlist/hire/reject (and undo), request first interview round with 3 slots, record round results (Select → next round / Hire, Reject, Hold), request and review documents for selected candidates, create/send offers and mark joined via shared routes, add private notes, view analytics and invoices, export CSVs.
- **Cannot**: add team members, pay invoices online, see other companies' jobs.
- **Restrictions / gaps**: can act on applicants not yet released by QC1→QC2→Q3 (product decision needed); document list/review is scoped to the candidate rather than the application.

4.5 Q3 (7) — interview coordination & hiring ops

- **Pages**: `/q3/*` (dashboard, referred, company-review, scheduling, interviews, decisions, documents, offers, returned). Sidebar also links legacy `/recruitment/q3/*` — the candidate list and interviews screens there call `/recruitment/*` endpoints that **exclude Q3** (403).
- **Can**: forward referred CVs to the company (14-day window starts) or return them to Q2; capture company shortlist & panel; request slots from HR; confirm a slot and send invites (Jitsi link optional); remind, reschedule, mark conducted; chase companies for results; record round decisions (NextRound / FinalRound / Hold / Rejected / Selected); request, verify, send back, forward and approve documents; generate & send the offer; mark joined; quarterly report.
- **Scope gap**: through `/recruitment/*` routes Q3 can act on applications never referred to Q3.

4.6 Admin (5)

- **Pages**: `/admin/*` (dashboard, users, user detail, employer approvals, companies, jobs, job detail, applications, application detail, blog, settings, reports, audit). Also admitted to every Q1/Q2/Q3/recruitment/employer route.
- **Owns**: employer registration approval/rejection; job Active ↔ Closed; user active flag and staff role (never self, never across candidate/employer/staff lines); staff account creation; password-reset links; platform settings (stored, not enforced); blog posts; exports; audit log.
- **By design read-only over the hiring pipeline**: `/admin/applications/:mapId` shows a "next step" link to the owning workspace instead of duplicating transitions.
- **Effectively blocked**: employer `me/*` routes and the company-review route return 404 for Admin (no linked company).

5. Client-route ↔ server-role alignment

Web route group	Client guard (`allow`)	Server roles of the APIs it calls	Aligned?
`/candidate/*` , `/jobs/:id/apply`	Subscriber	Subscriber	Yes
`/q1/*`	QC1, Admin	`/q1/*` QC1, Admin	Yes
`/q2/*`	QC2, Admin	`/q2/*` QC2, Admin	Yes
`/q3/*`	Q3, Admin	`/q3/*` Q3, Admin	Yes
`/admin/*`	Admin	`/admin/*` Admin	Yes
`/recruitment/candidates` , `/documents` , `/interviews`	QC1, QC2, Admin	QC1, QC2, Admin	Yes
`/recruitment/qc1`	QC1, Admin	QC1, Admin	Yes
`/recruitment/referrals`	QC2, Admin	list QC2/Q3/Admin, create QC2/Admin	Yes
`/recruitment/q3/candidates` , `/q3/interviews` , `/q3/documents`	Q3, Admin	`/recruitment/candidates` , `/interviews` , `/documents` = QC1, QC2, Admin	**No — Q3 gets 403**
Employer portal	Client, Admin	`/clients/*` Client, Admin (Admin → 404 no company)	Partially (Admin unusable)

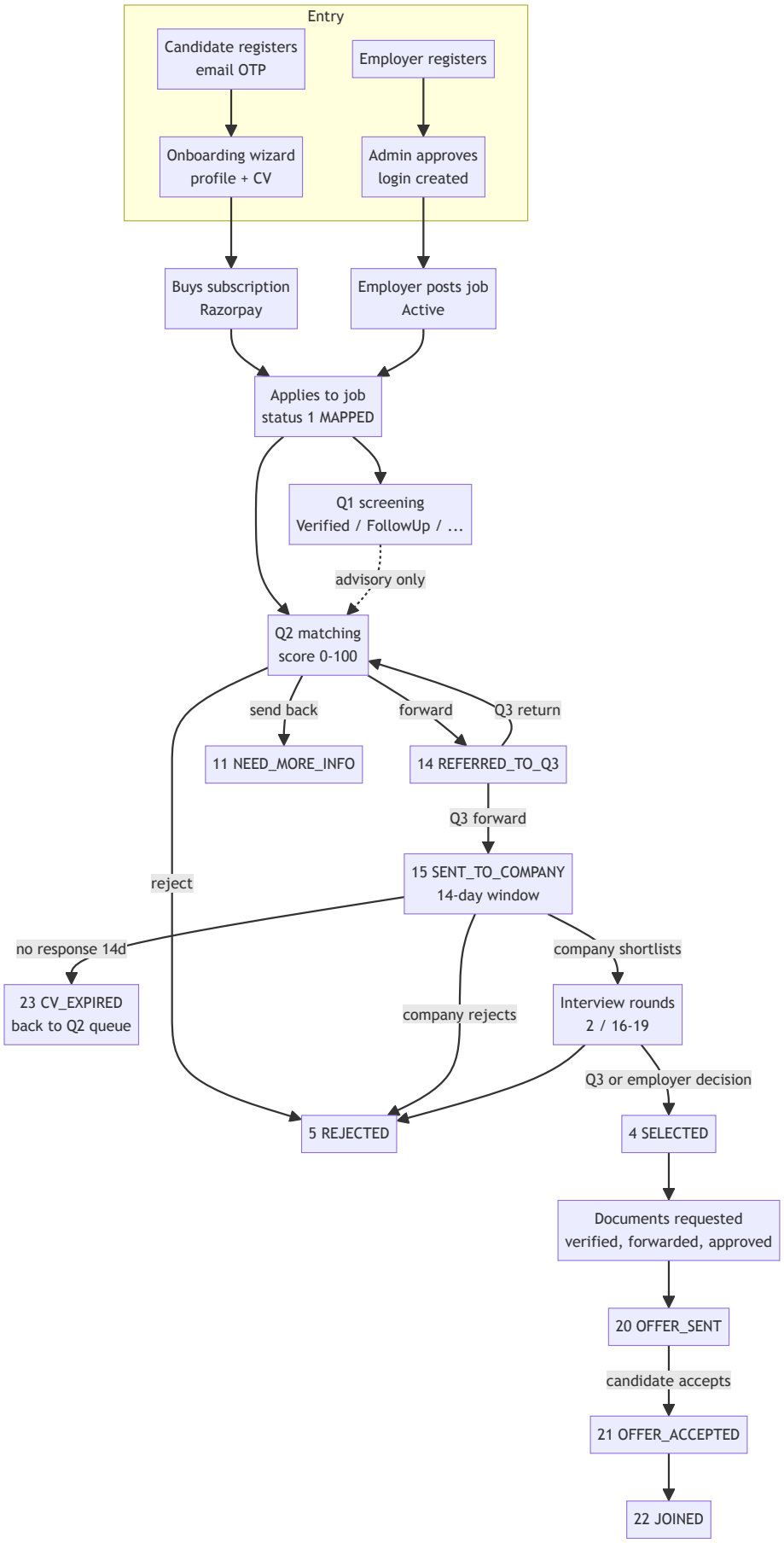
Complete End-to-End Workflow

> Traced through the code at commit `8de48cf`. Each workflow answers: who initiates, which screen, which action, which frontend component, which API, which backend service, which tables, which checks, success path, failure path, who else is affected, and what notifications / status changes follow. "VERIFIED" facts come from reading code; anything inferred is marked.

0. The business in one picture

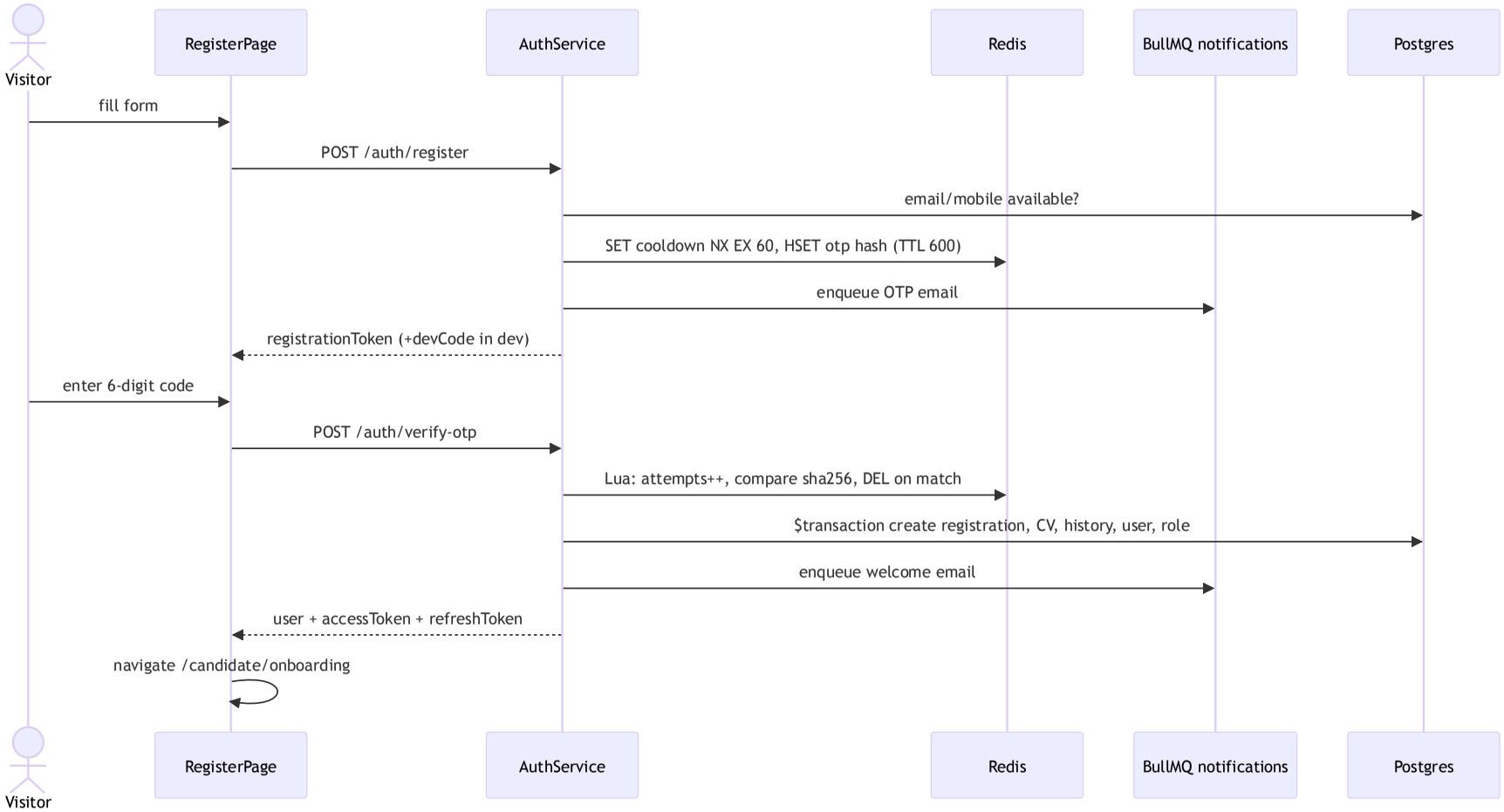
Aajiveka is a recruitment/ATS platform. Candidates pay for a subscription and apply to jobs posted by approved employers. Every application passes through three internal staff desks before and during the employer's own process:

- **Q1 (QC1)**: screens the candidate's profile.
- **Q2 (QC2)**: scores and matches the candidate against the job, then refers the best CVs to Q3.
- **Q3**: forwards CVs to the company, coordinates interview rounds, collects documents, and issues the offer through to joining.
- **Admin**: approves employers, manages users and moderates jobs.


Entry
Candidate registers email OTP
Employer registers
Onboarding wizard profile + CV
Admin approves login created
Buys subscription Razorpay
Employer posts job Active
Applies to job status 1 MAPPED
Q1 screening Verified / FollowUp / ...
advisory only
Q2 matching score 0-100
send back
forward
Q3 return
11 NEED_MORE_INFO
14 REFERRED_TO_Q3
Q3 forward
15 SENT_TO_COMPANY 14-day window
reject
no response 14d
company shortlists
23 CV_EXPIRED back to Q2 queue
company rejects
Interview rounds 2 / 16-19
Q3 or employer decision
5 REJECTED
4 SELECTED
Documents requested verified, forwarded, approved
20 OFFER_SENT
candidate accepts
21 OFFER_ACCEPTED
22 JOINED

1. Candidate registration (email OTP)

#	Question	Answer
1	Initiator	Anonymous visitor
2	Screen	`/register` → `apps/web/src/features/auth/pages/RegisterPage.tsx`
3	Trigger	Register form submit, then OTP form submit
4	Frontend	`RegisterPage` (RHF + Zod), auth API hooks under `apps/web/src/features/auth/`, `auth.store`
5	API	`POST /api/auth/register` → `POST /api/auth/verify-otp` (`POST /api/auth/resend-otp` to resend)
6	Backend	`AuthController` → `AuthService.register` / `verifyEmailOtp` / `resendEmailOtp`; `OtpService` (Redis Lua script)
7	Data	Redis: `otp:register:email:<token>` (600 s), cooldown key (60 s). Postgres on verify (one `$transaction`): `tblSubscriberRegistration`, `tblSubscriberCVDetails` (emailVerified), `tblSubscriberStatusHistory` (1 Account created), `tblSecUser` (userName = mobile), `tblSecMapUserRoles` (role 1); then `tblSecActiveSessions`, `tblAuditLog`
8	Checks	DTO: name 2–100, valid email, mobile pattern (Indian `^[6-9]\d{9}$` for +91), password 8–72. Email must not belong to a verified candidate; mobile must not be an existing username. OTP: 6 digits, max 5 attempts, 10-min expiry, 60 s resend cooldown. 5 req/min/IP
9	Success	Tokens returned; `isOnboarded:false` → web routes the user to `/candidate/onboarding`. Welcome email queued
10	Failure	409 email/mobile in use; 429 cooldown (with `retryAfterSeconds`); 400 wrong code (with `attemptsRemaining`) / expired / too many attempts; 503 if the email could not be queued. Nothing is written to Postgres before verification
11	Affected	Q1 queue gains a candidate (screening row is created lazily when Q1 views it)
12	Notifications	OTP email + welcome email (via BullMQ `notifications` queue). With the default `EMAIL_DRIVER=log` they are only logged. In non-production the API also returns `devCode`


Visitor
RegisterPage
AuthService
Redis
BullMQ notifications
Postgres
fill form
POST /auth/register
email/mobile available?
SET cooldown NX EX 60, HSET otp hash (TTL 600)
enqueue OTP email
registrationToken (+devCode in dev)
enter 6-digit code
POST /auth/verify-otp
Lua: attempts++, compare sha256, DEL on match
$transaction create registration, CV, history, user, role
enqueue welcome email
user + accessToken + refreshToken
navigate /candidate/onboarding
RegisterPage
AuthService
Redis
BullMQ notifications
Postgres
Visitor

Gaps: no SMS/mobile verification (SMS providers exist but are never called); the welcome email links to `/profile`, which is not a web route.

2. Login, session and logout (all roles)

#	Answer
1–3	Any user, `/login` (`apps/web/src/features/auth/pages/LoginPage.tsx`), Sign-in button
4	`LoginPage`, `useAuth` store (`apps/web/src/features/auth/auth.store`), `tokenStorage` (`apps/web/src/lib/tokenStorage.ts`)
5	`POST /api/auth/login`; later `POST /api/auth/refresh` (axios interceptor on 401), `GET /api/auth/me`, `POST /api/auth/logout`
6	`AuthService.login` / `refresh` / `me` / `logout`
7	Reads `tblSecUser` (by username, else email via `tblSubscriberCVDetails` or `tblMstrPerson`), `tblSecMapUserRoles` (role source of truth); writes `tblSecActiveSessions`, `tblSecUserLogin`, `tblAuditLog`
8	Argon2id verify (dummy hash when user missing); `active='1'`; 5 attempts/min per IP; no per-account lockout
9	Web stores tokens, redirects to `ROLE_HOME[roleId]`: candidate `/candidate/profile`, QC1 `/q1/dashboard`, QC2 `/q2/dashboard`, employer `/company`, admin `/admin`, Q3 `/q3/dashboard`
10	401 "Invalid username or password"; on refresh failure the axios interceptor clears tokens and sends the user to `/login`
11	—
12	Audit `auth.login` / `auth.logout`

Notes: staff can sign in by username but not by email (NodeType 100, see ROLE_PERMISSION_MATRIX). Refresh tokens rotate and are single-use; access tokens are not revoked on logout (valid until expiry).

3. Password recovery

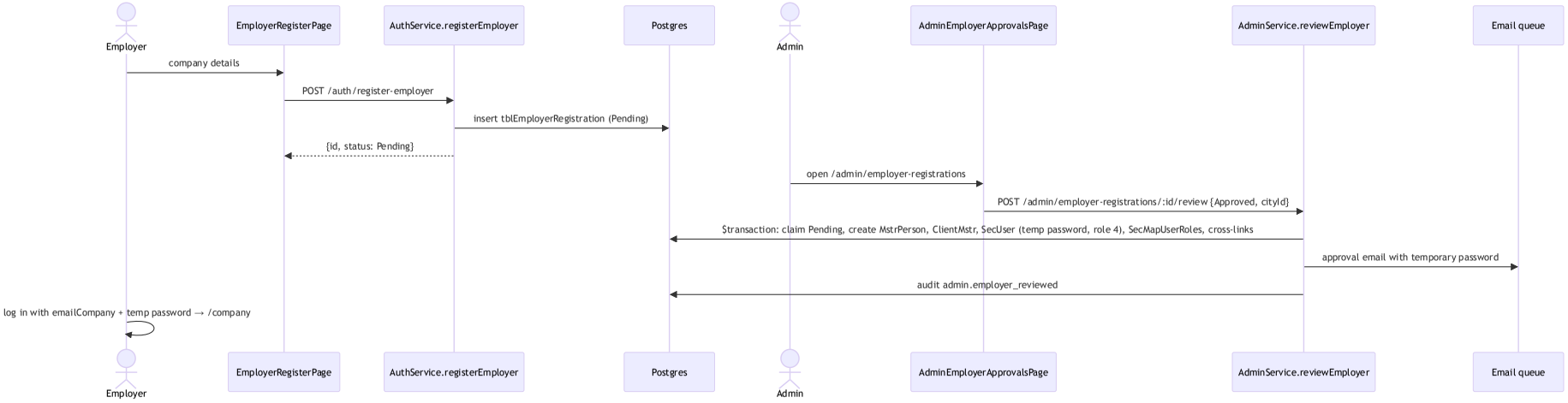
`/forgot-password` → `POST /api/auth/forgot-password {userName}` → `AuthService.forgotPassword` creates `tblAuthPasswordReset` (sha256 token, 1 h) and emails `${APP_URL}/reset-password?token=…`. `/reset-password` → `POST /api/auth/reset-password` → password updated, token marked used, **all sessions deleted** (transaction). The response is always generic. Forgot-password matches **username/mobile only**, not email. Admin can also trigger a reset link (`POST /api/admin/users/:id/password-reset`).

4. Candidate onboarding and profile building

#	Answer
1–3	Candidate; `/candidate/onboarding` (`ProfileWizardPage`) then `/candidate/profile` (`ProfileOverviewPage`), `/candidate/resume-builder`; section Save buttons
4	`apps/web/src/features/candidates/portal/*` (`CandidatePortalLayout`, wizard steps, section editors)
5	`GET /api/candidates/me`, `/me/cv-edit`, `/me/cv-masters`, `/me/institutes`; `PUT /me/personal`, `/me/professional`, `/me/education`, `/me/employment`, `/me/certificates`, `/me/headline`, `/me/summary`, `/me/key-skills`, `/me/career-profile`, `/me/it-skills`, `/me/projects`, `/me/languages`, …; `POST /api/files/resume`, `/api/files/avatar`; `POST /api/candidates/me/complete-onboarding`
6	`CandidatesService` (`apps/api/src/modules/candidates/candidates.service.ts`), `StorageService`
7	`tblSubscriberCVDetails`, `tblSubscriberProfileExtra`, `tblSubscriberEducation`, `tblSubscriberEmployer`, `tblSubscriberCertificate`, `tblSubscriberITSkill`, `tblSubscriberProject`, `tblSubscriberAccomplishment`, `tblSubscriberLanguage`, `tblSubscriberPrefferedLocations`, `tblSubscriberTags`, `tblSubscriberCVUploaded`; first personal save also writes `CV_CREATED` history and `flgCVUploaded`
8	DTO validators per section; education degree/course consistency and year ranges; upload ≤10 MB, pdf/doc/docx/jpeg/png with matching extension; all writes scoped to the caller's `subscriberID`
9	Data saved; completing the wizard sets `onboardedAt`, so `ProtectedRoute` stops redirecting
10	400 validation (incl. "Save your personal details first"); 404 if the login is not linked to a candidate profile (all migrated legacy candidates); FK errors on bad master ids surface as 500 (inferred)
11	Q1 completeness and priority, Q2 scoring and employer applicant views read this data
12	Audit for resume/avatar/document uploads only

Gaps: the advanced section editor (`/candidate/profile/advanced`) is commented out in the router, so IT skills, diversity, personal details and accomplishments have no UI and profile completion cannot reach 100% (router comment). API and web compute profile completion differently.

5. Employer registration and approval


Employer
EmployerRegisterPage
AuthService.registerEmployer
Postgres
Admin
AdminEmployerApprovalsPage
AdminService.reviewEmployer
Email queue
company details
POST /auth/register-employer
insert tblEmployerRegistration (Pending)
{id, status: Pending}
open /admin/employer-registrations
POST /admin/employer-registrations/:id/review {Approved, cityId}
$transaction: claim Pending, create MstrPerson, ClientMstr, SecUser (temp password, role 4), SecMapUserRoles, cross-links
approval email with temporary password
audit admin.employer_reviewed
log in with emailCompany + temp password → /company
Employer
EmployerRegisterPage
AuthService.registerEmployer
Postgres
Admin
AdminEmployerApprovalsPage
AdminService.reviewEmployer
Email queue

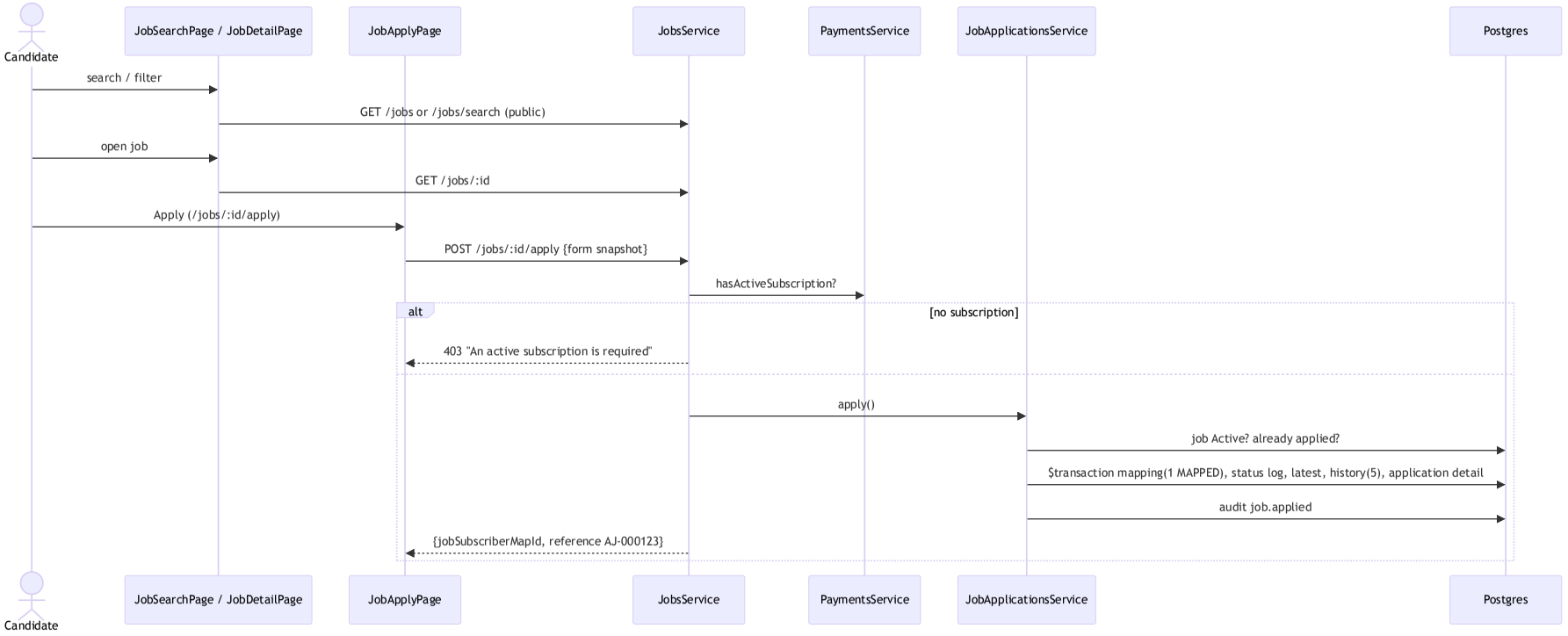
Item	Detail
Checks	Duplicate `emailCompany` unless the previous one was Rejected (case-sensitive compare). Approval: must be Pending (409), no existing login with that username (409), city must resolve (400)
Failure	Reject sends no email. Approval rolls back entirely on error
Affected	Employer can now post jobs; company appears in `/admin/companies`
Gaps	Temporary password is emailed in plain text and nothing forces it to be changed; employers have no change-password endpoint; free-text `companyLogo` from the registration is copied to the company and served publicly (see issues)

6. Job posting lifecycle (Employer, Admin)

#	Answer
1–3	Employer; employer portal "Post job" / "Manage jobs" screens (`apps/web/src/employer/pages/ManageJobs/*`); Save draft, Publish, Close, Archive, Activate, Duplicate, Delete, Bulk upload
5	`POST /api/clients/me/jobs?draft=` , `PATCH /api/clients/me/jobs/:id?draft=` , `POST …/status` , `/deactivate` , `/archive` , `/activate` , `/duplicate` , `DELETE …/:id` , `POST /api/clients/me/jobs/bulk-upload?commit=`
6	`EmployersService.createJob / updateJob / setJobStatus / duplicateJob / deleteJob / bulkUploadJobs`
7	`tblClientJobs` , `tblClientJobSkill` , `tblMstrSkills` (new skills auto-created), delete cascades education/gender maps and saved jobs
8	Ownership (`ownedJob`); required designation, employment type, work mode, city, CTC range ints; **no min ≤ max checks**; delete refused when applications exist
9	Job is Active (1) immediately — **there is no review/approval state** — or Draft (4)
10	404 not owner; 400 delete-with-applicants; bad master ids → FK error
11	Active jobs appear in public search, Q2 job lists, recommendations
12	Audit on status change and delete only (not on create/update/duplicate/bulk)

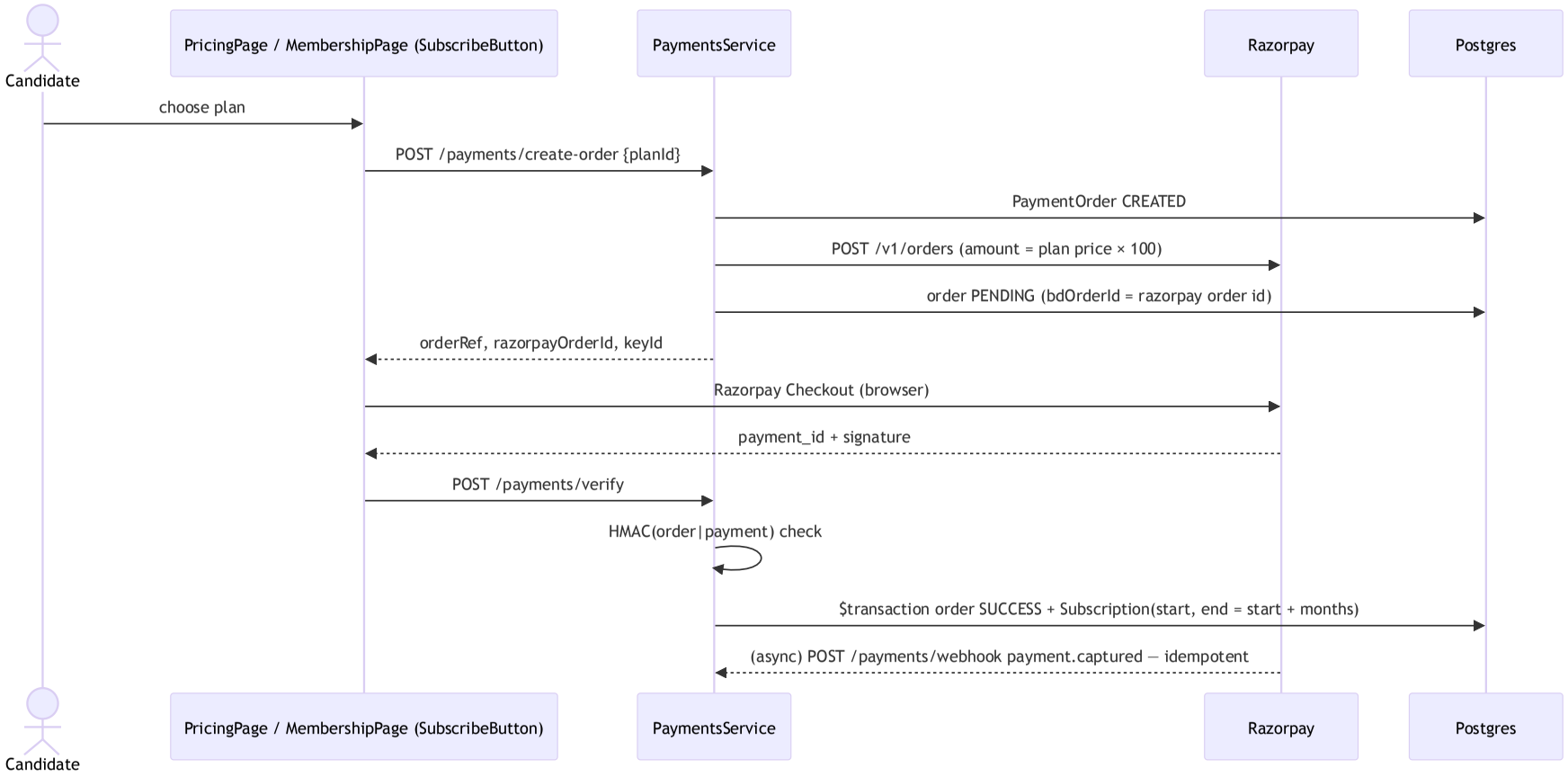
Status states: **1 Active ↔ 2 Closed ↔ 3 Archived, 4 Draft** — any-to-any for the employer. **Admin** can only `close` / `reopen` (Active ↔ Closed) via `POST /api/admin/jobs/:jobId/status` or `bulk-moderate` ; nothing prevents the employer re-activating an admin-closed job. Closing a job does not touch its in-flight applications.

7. Job search and apply (Candidate)


Candidate
JobSearchPage / JobDetailPage
JobApplyPage
JobsService
PaymentsService
JobApplicationsService
Postgres
search / filter
GET /jobs or /jobs/search (public)
open job
GET /jobs/:id
Apply (/jobs/:id/apply)
POST /jobs/:id/apply {form snapshot}
hasActiveSubscription?
alt
[no subscription]
403 "An active subscription is required"
apply()
job Active? already applied?
$transaction mapping(1 MAPPED), status log, latest, history(5), application detail
audit job.applied
{jobSubscriberMapId, reference AJ-000123}
Candidate
JobSearchPage / JobDetailPage
JobApplyPage
JobsService
PaymentsService
JobApplicationsService
Postgres

- Failure: 403 no subscription; 404 job not active; 400 already applied (race-prone: no unique index on job+candidate).
- No email to the candidate or employer. The application immediately becomes visible to Q2 (all applications on active jobs) **and to the employer** (all statuses).
- Staff can create the same application with `POST /api/recruitment/candidates/:id/assign-job` (no subscription gate).

8. Subscription purchase (Candidate)


Candidate
PricingPage / MembershipPage (SubscribeButton)
PaymentsService
Razorpay
Postgres
choose plan
POST /payments/create-order {planId}
PaymentOrder CREATED
POST /v1/orders (amount = plan price × 100)
order PENDING (bdOrderId = razorpay order id)
orderRef, razorpayOrderId, keyId
Razorpay Checkout (browser)
payment_id + signature
POST /payments/verify
HMAC(order|payment) check
$transaction order SUCCESS + Subscription(start, end = start + months)
(async) POST /payments/webhook payment.captured — idempotent
Candidate
PricingPage / MembershipPage (SubscribeButton)
PaymentsService
Razorpay
Postgres

- Plans: `GET /api/payments/plans` (12 seeded; tier not tied to the candidate's experience).
- Effect: a `tblSubscription` row only; it unlocks self-apply. Role is unchanged; no receipt email.
- Failure: 400 Razorpay not configured (the Docker compose files do not pass `RAZORPAY_*`); 400 bad signature; Razorpay HTTP failure → 500 leaving a CREATED order; concurrent verify + webhook can make one request 500 (unique subscription per order).
- **BillDesk** path (`POST /api/payments/orders` + `/payment/return` polling `GET /payments/orders/:orderRef`) can create orders but **cannot settle them**: `PaymentsService.settle()` has no route.

9. Q1 screening (QC1)

#	Answer
1–3	QC1; `/q1/dashboard` , `/q1/candidates` , `/q1/candidates/:id` , `/q1/follow-ups` , `/q1/no-response` , `/q1/verified` , `/q1/not-interested` , `/q1/analytics` ; buttons Verify, Contact, Request update, Request CV, Follow-up, No response, Not interested, checklist ticks
4	`apps/web/src/features/q1/pages/*` , `features/q1/components/*`
5	`/api/q1/*` (see API_DOCUMENTATION §8)
6	`Q1Service` , `Q1AnalyticsService`
7	`tblSubscriberScreening` (status, priority, 7 checklist flags, follow-up fields), `tblScreeningContactLog` , reads profile tables
8	`@Roles(QC1, Admin)` ; DTO enums; **no precondition for Verify**
9	Screening status changes; priority recomputed (completeness ≥ 80 → High)
10	404 unknown candidate; 400 validation
11	Q2 (advisory only — Q2 does not filter on Q1 status); Q3 shows "contact verified"
12	Audit `q1.*` ; "Contact"/"Request CV" only **log** the attempt — no message is sent

Screening states: `New → Screening` (on first open) → `Verified | FollowUp | NoResponse | NotInterested` ; `Incomplete` when completeness < 100 or when Q2 sends back. The separate registration decision (`POST /api/recruitment/candidates/:id/decision` , Approved/Rejected/OnHold/NeedMoreInfo/Duplicate/Withdrawn) updates `tblSubscriberRegistration.flgstatus` and candidate history but **no application status**.

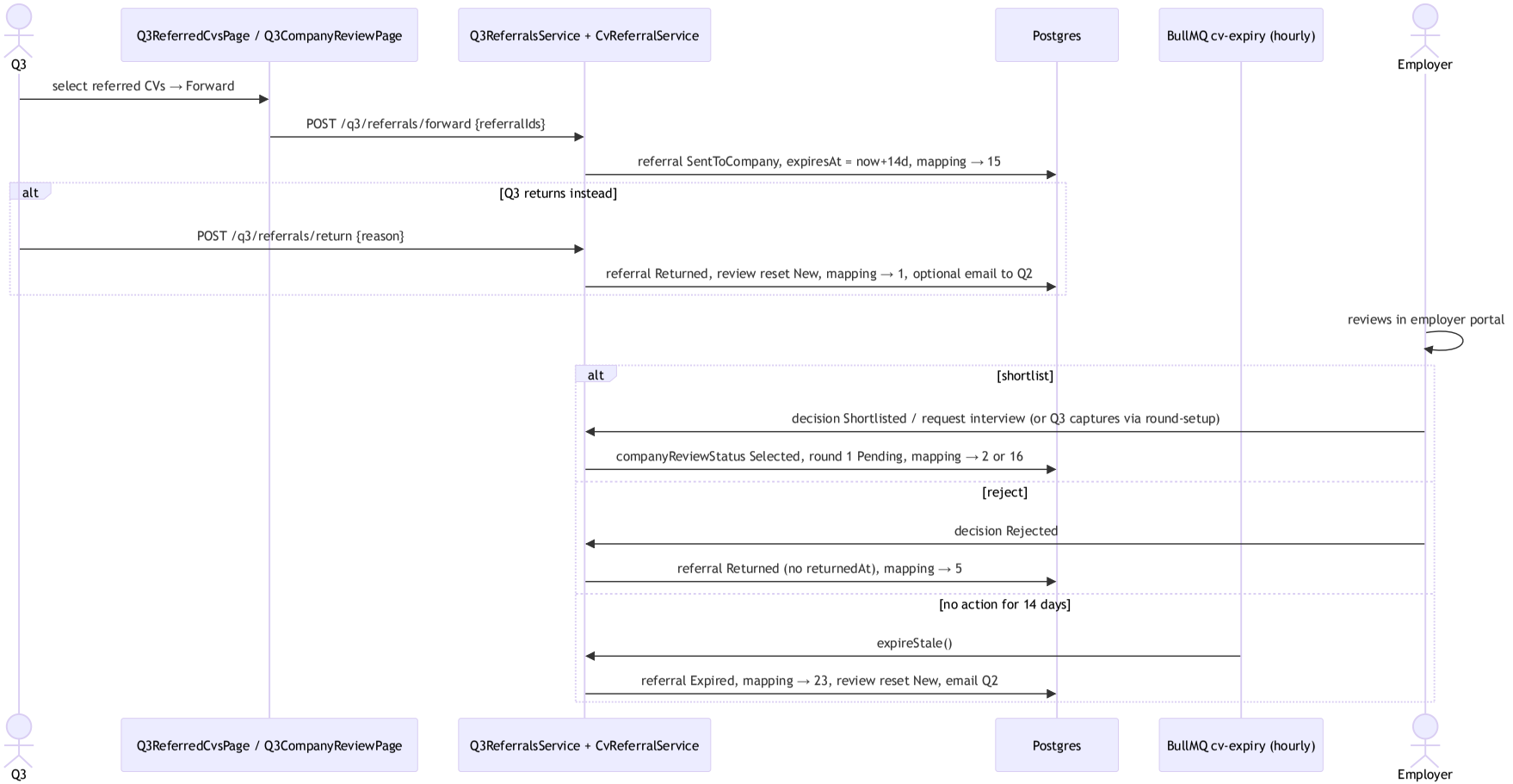
10. Q2 matching (QC2)

#	Answer
1–3	QC2; `/q2/jobs` → `/q2/jobs/:jobId` → `/q2/applications/:mapId` ; `/q2/applicants` , `/q2/forwarded` , `/q2/sent-back` ; buttons Forward to Q3, Send back to Q1, Reject, criterion toggles
4	`apps/web/src/features/q2/pages/*` (e.g. `Q2ApplicantDetailPage.tsx`)
5	`/api/q2/*` ; forward internally calls `CvReferralService.referToQ3`
6	`Q2Service` , `ScoringService` , `CvReferralService` , `JobApplicationsService.transitionStatus`
7	`tblCandidateJobScore` (7 sub-scores + total), `tblApplicationMatchReview` , `tblCvReferral` , mapping + status log, `tblSubscriberScreening` (on send-back)
8	Review must be `New` (409 otherwise). No confirmation dialog on Forward/Send back/Reject in the UI
9	Forward → **14 REFERRED_TO_Q3**, referral `Referred` . Send back → **11 NEED_MORE_INFO**, screening `Incomplete` . Reject → **5 REJECTED**
10	404 / 409
11	Q3 receives forwarded CVs; Q1 sees sent-back candidates as Incomplete
12	Audit `q2.*` , `cv.referred_to_q3` ; no email to candidate

Scoring (`apps/api/src/modules/recruitment/scoring.service.ts`): skills Jaccard 30%, experience 25%, job role 20%, education 10%, location 5%, salary 5%, notice 5%. Relevant ≥ 60. Scores are computed lazily on Q2 GETs (≤25 per request), not at apply time.

Gap: a send-back is a dead end — nothing returns the application to Q2's queue after Q1 re-verifies.

11. Q3: forward to company, company review, expiry


Q3
Q3ReferredCvsPage / Q3CompanyReviewPage
Q3ReferralsService + CvReferralService
Postgres
BullMQ cv-expiry (hourly)
Employer
select referred CVs → Forward
POST /q3/referrals/forward {referralIds}
referral SentToCompany, expiresAt = now+14d, mapping → 15
alt
[Q3 returns instead]
POST /q3/referrals/return {reason}
referral Returned, review reset New, mapping → 1, optional email to Q2
reviews in employer portal
alt
[shortlist]
decision Shortlisted / request interview (or Q3 captures via round-setup)
companyReviewStatus Selected, round 1 Pending, mapping → 2 or 16
[reject]
decision Rejected
referral Returned (no returnedAt), mapping → 5
[no action for 14 days]
expireStale()
referral Expired, mapping → 23, review reset New, email Q2
Q3
Q3ReferredCvsPage / Q3CompanyReviewPage
Q3ReferralsService + CvReferralService
Postgres
BullMQ cv-expiry (hourly)
Employer

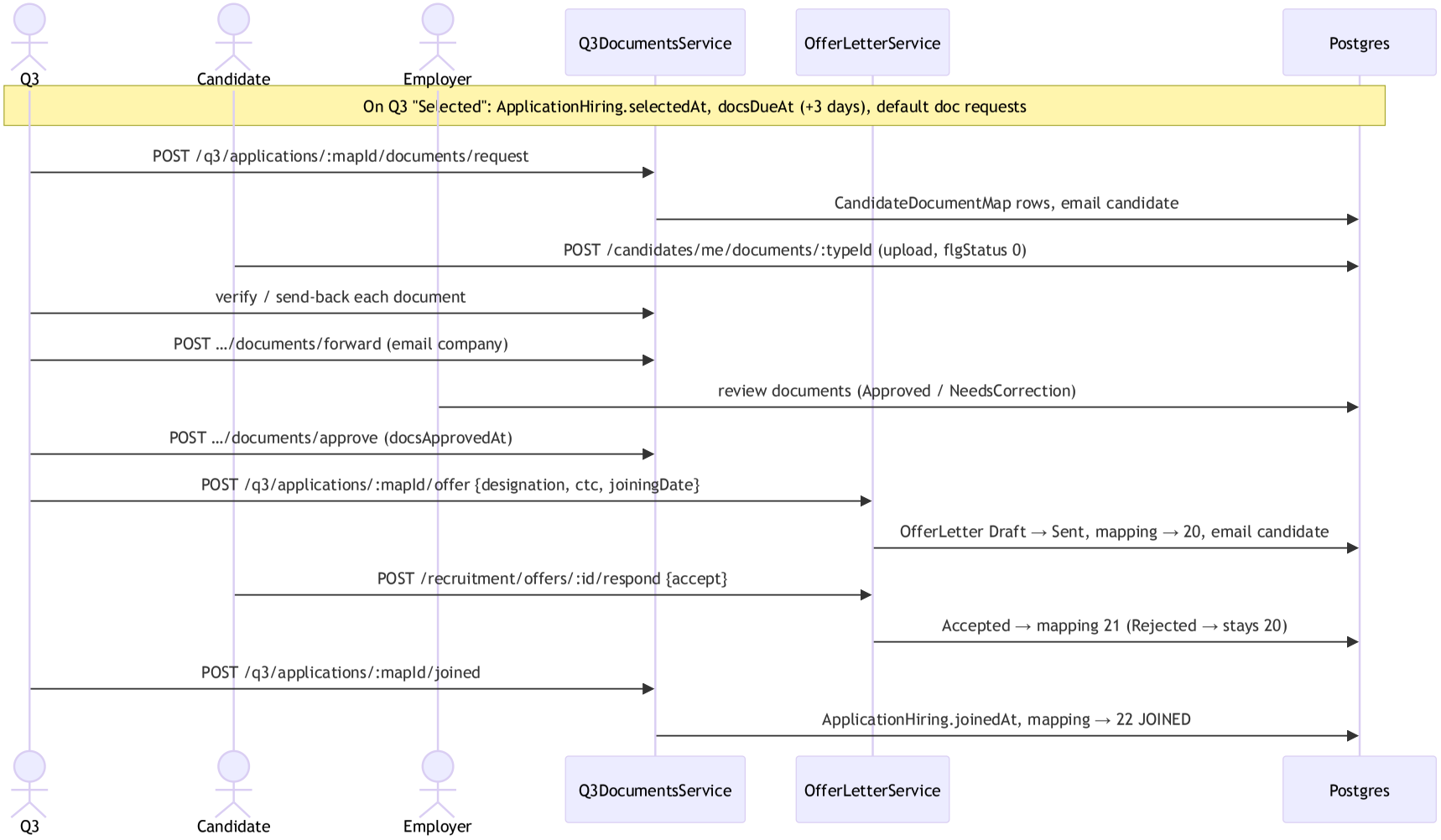
- Q3 can email companies whose review window is about to close (`POST /api/q3/company-review/remind-expiring`).
- Two shortlist outcomes exist depending on path: Q3 round-setup → **2 SHORTLISTED**; employer interview request / recruitment company-review → **16 INTERVIEW_R1**.

12. Interview rounds (Q3 + Employer + Candidate)

Step	Who	Screen	API	Effect
Create round with 3 slots	Employer	Applicant detail (employer portal)	`POST /api/clients/me/applicants/:id/interview-rounds`	Round Pending + 3 slots; mapping → 16 (or 19 for Final); history 7
…or capture shortlist/panel	Q3	`/q3/scheduling/:mapId`	`PUT /api/q3/applications/:mapId/round-setup`	Round 1 Pending, slots; mapping → 2
Ask company for slots	Q3	`/q3/scheduling`	`POST /api/q3/rounds/:roundId/request-slots`	Email HR/company
Contact candidate	Q3	`/q3/scheduling`	`POST /api/q3/applications/:mapId/contact`	Activity log
Confirm slot + send invites	Q3	`/q3/scheduling/:mapId`	`POST /api/q3/rounds/:roundId/confirm` (optional Jitsi link from `/q3/meeting-link`)	Round Scheduled; mapping → 16/17/18/19; emails (IST)
…or candidate picks slot	Candidate	`/candidate/interviews`	`POST /api/recruitment/interview-rounds/:roundId/select-slot`	Round Scheduled; emails (server time zone); no status precondition
Reminder / day-of checklist	Q3	`/q3/interviews`	`/remind` , `/checklist`	Email, flags
Reschedule / no-show	Q3	`/q3/interviews`	`POST /api/q3/rounds/:roundId/reschedule`	Before start → 7 RESCHEDULE_REQUESTED; after start without join → 9 NOT_ATTENDED
Mark conducted	Q3	`/q3/interviews`	`POST /api/q3/rounds/:roundId/conducted`	→ 6 ATTENDED; result due in 24 h
Chase company	Q3	`/q3/decisions`	`POST /api/q3/applications/:mapId/nudge`	Email
Decision	Q3	`/q3/decisions/:mapId`	`POST /api/q3/rounds/:roundId/decision`	NextRound/FinalRound → new round; Hold → `onHoldAt` only; Rejected → 5; Selected (final round only) → 4 + default document requests
…or result	Employer	Applicant detail	`POST /api/clients/me/applicants/:id/interview-rounds/:roundId/result`	Select→next round or Hire (→4); Reject (→5); Hold (→10); candidate email
Decision email	Q3	`/q3/decisions`	`POST /api/q3/rounds/:roundId/decision-mail`	Email

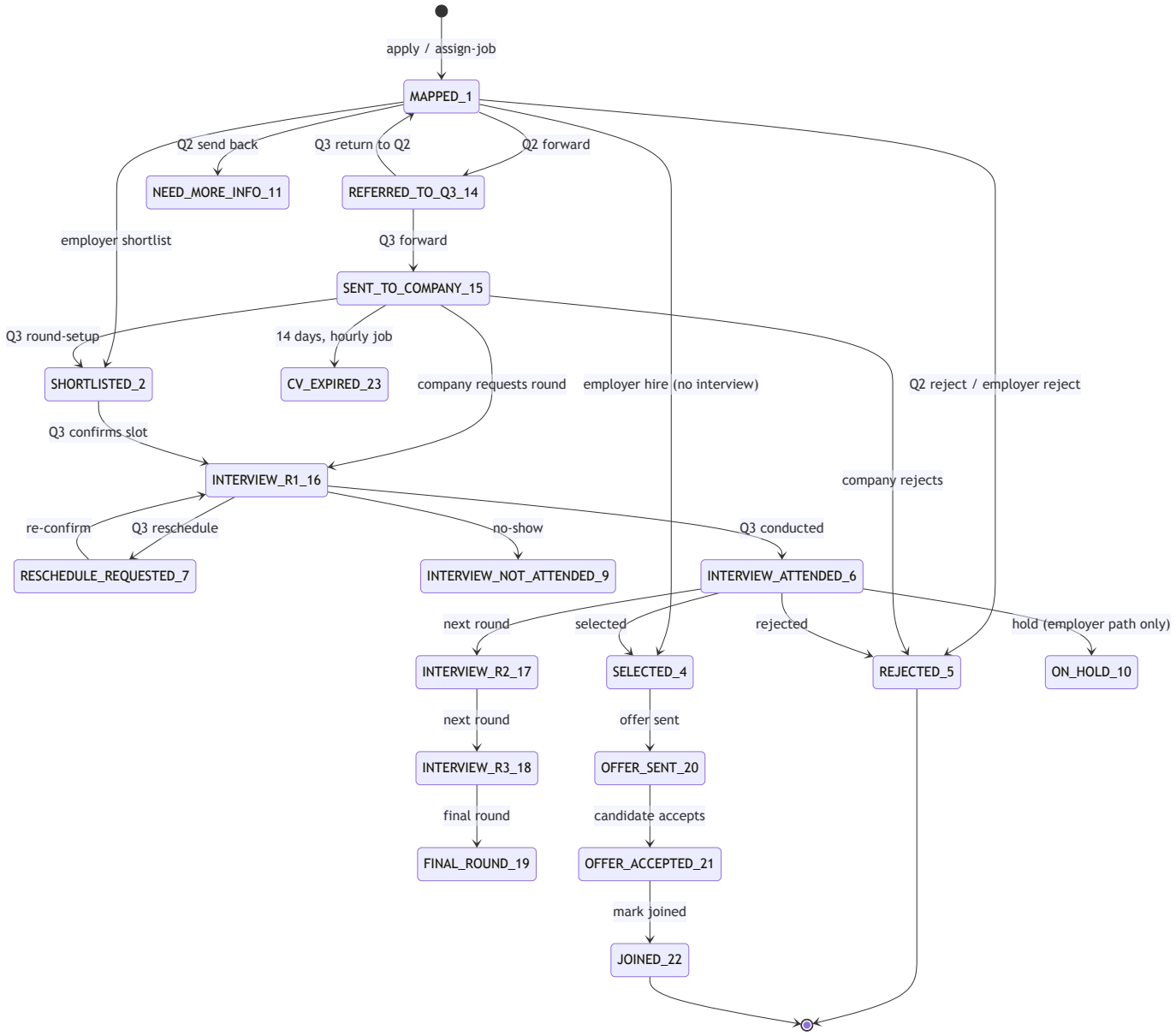
Conflict (confirmed): the employer path (`InterviewRoundService.submitResult`) and the Q3 path (`Q3InterviewsService.decide`) record decisions differently (different fields, different Hold/Select/Final semantics). After an employer "Hire", Q3 still shows the round awaiting a decision and the documents/offer flow does not start. See TESTING_AND_ISSUES.

13. Documents, offer and joining


Q3
Candidate
Employer
Q3DocumentsService
OfferLetterService
Postgres
On Q3 "Selected": ApplicationHiring.selectedAt, docsDueAt (+3 days), default doc requests
POST /q3/applications/:mapId/documents/request
CandidateDocumentMap rows, email candidate
POST /candidates/me/documents/:typeId (upload, flgStatus 0)
verify / send-back each document
POST …/documents/forward (email company)
review documents (Approved / NeedsCorrection)
POST …/documents/approve (docsApprovedAt)
POST /q3/applications/:mapId/offer {designation, ctc, joiningDate}
OfferLetter Draft → Sent, mapping → 20, email candidate
POST /recruitment/offers/:id/respond {accept}
Accepted → mapping 21 (Rejected → stays 20)
POST /q3/applications/:mapId/joined
ApplicationHiring.joinedAt, mapping → 22 JOINED
Q3
Candidate
Employer
Q3DocumentsService
OfferLetterService
Postgres

- Offer creation requires mapping status 4 and **all** of the candidate's uploaded documents verified (across all applications). The Q3 route additionally requires `docsApprovedAt`.
- Alternate (legacy/shared) path: `POST /api/recruitment/offers` + `/offers/:id/send` + `/offers/:id/mark-joined` (Q3, Client, Admin). `mark-joined` there does not set `joinedAt`, so Q3 keeps showing the application in the Offer stage.

14. Application status state machine (JobMapStatus)



Simplified for readability: rounds R2/R3/Final follow the same confirm → conducted → decision loop as R1. The legacy path (`POST /api/recruitment/interviews`) can set 3 INTERVIEW_SCHEDULED from any state. **Statuses never written by any code**: 8 RESCHEDULED, 12 DUPLICATE, 13 WITHDRAWN. **`transitionStatus` itself enforces no from-state**; guards live (inconsistently) in callers.

Every transition writes a `tblJobSubscriberStatus` log row and updates `tblSubscriberJobStatusLatest`; only some write candidate history (`tblSubscriberStatusHistory`) — none of the Q3-path transitions do.

15. Admin operations

Operation	Screen	API	Result
Create staff user	`/admin/users`	`POST /api/admin/users`	Person + login (random password) + emailed set-password link (3-day token); with log email driver the link is returned in the response

Operation	Screen	API	Result
Activate / deactivate / change role	`/admin/users/:userId`	`PATCH /api/admin/users/:userId`, `POST /api/admin/users/bulk-update`	Transaction updates `tblSecUser` + `tblSecMapUserRoles`, deletes refresh sessions; never self, never across candidate/employer/staff
Send password reset	user detail	`POST /api/admin/users/:userId/password-reset`	1-hour link emailed
Approve/reject employer	`/admin/employer-approvals`	`POST /api/admin/employer-registrations/:id/review`	See §5
Close / reopen jobs	`/admin/jobs`, `/admin/jobs/:jobId`	`POST /api/admin/jobs/:jobId/status`, `/bulk-moderate`	Active ↔ Closed, audited per job
Inspect applications	`/admin/applications/:mapId`	`GET /api/admin/applications/:mapId`	Read-only timeline + deep link to owning workspace
Settings	`/admin/settings`	`GET/PATCH /api/admin/settings`	Stored only — **no code reads them**
Blog	`/admin/blog`	`/api/admin/blog-posts`	CRUD
Reports / exports	`/admin/reports`	`/api/exports/*`	CSV/XLSX/PDF
Audit	`/admin/audit`	`GET /api/admin/audit-logs`	Filtered audit browser

16. Notifications summary

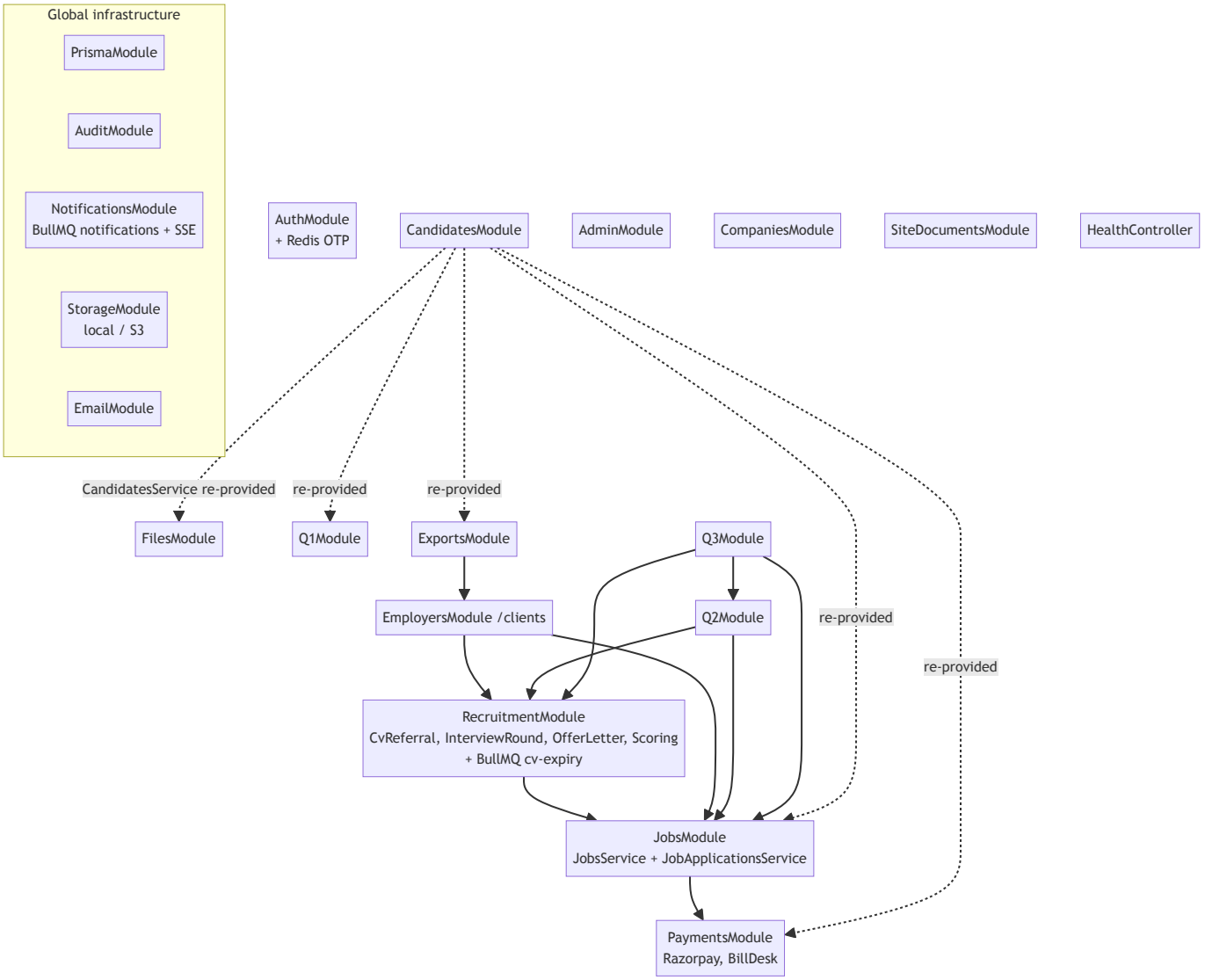
Channel	Implemented	Triggered by
Email (BullMQ → SMTP/SES/log)	Yes	OTP, welcome, password reset, staff set-password, employer approval, legacy interview scheduled, slot selected, round results, offer sent, all Q3 mails (slot requests, invites, reminders, nudges, decisions, document requests/send-backs/forwarding, review-window reminders, return-to-Q2 mails, auto-expiry mails)
Email — not sent	—	application submitted, Q2 reject, forward to company, company reject, offer accepted/rejected, joined, payment receipt, job alerts
SMS	Providers only	**Never called**
In-app / SSE	Endpoints only	**Nothing creates notifications**
WhatsApp	—	Logged as a channel only

Module Dependencies

> Module graph read from the `*.module.ts` files and service constructors in `apps/api/src`, and from feature imports in `apps/web/src`. Commit `8de48cf`.

1. Backend module graph

Global modules (`@Global()`; available to every module without importing): `PrismaModule` (`apps/api/src/prisma`), `AuditModule`, `NotificationsModule`, `StorageModule`, `EmailModule` (`apps/api/src/common/email`). `AppModule` also registers `BullModule.forRoot` (Redis), `JwtModule`, `ThrottlerModule` and the three global guards.



Dashed edges: `CandidatesService` is listed directly in the `providers` of FilesModule, JobsModule, PaymentsModule, Q1Module and ExportsModule instead of being imported from CandidatesModule. Each module therefore gets **its own instance** (stateless, so harmless today, but per-instance caches such as `JobsService.filtersCache` are duplicated). ExportsModule likewise re-provides `JobApplicationsService` and `RecruitmentService`.

2. Module catalogue

Module	Purpose	Main files	Depends on	Used by	Roles	Status
Auth	Login, refresh rotation, OTP registration, employer registration, password reset, `/auth/me`, CSRF token	`modules/auth/auth.controller.ts`, `auth.service.ts`, `otp.service.ts`, `csrf.controller.ts`, `redis.provider.ts`	Prisma, Redis, Email (BullMQ), Audit	Every authenticated feature (tokens), Admin (reset tokens)	Public + Any	Implemented. Gaps: no SMS, CSRF unchecked, tokens not revocable
Candidates	Candidate profile and sections, documents, saved jobs, alerts, prefs, referrals, privacy, activity, recommendations	`modules/candidates/*`	Prisma, Storage, Audit	Files, Jobs (recommendations), Payments, Q1, Exports, Recruitment (`profile`, `resumeFile`)	Candidate	Implemented. Prefs partly saved; referrals/privacy/export not acted on
Files	Upload/download (resume, avatar, generic)	`modules/files/files.controller.ts`, `avatar-url.ts`	Storage, Candidates, Audit	Candidate portal	Candidate, Any, Public	Implemented. Generic download unscoped
Storage	File persistence abstraction	`modules/storage/storage.service.ts`, `drivers/local.driver.ts`, `drivers/s3.driver.ts`	env, Prisma (`tblMstrDocuments` folders)	Files, Candidates, Employers, Q2, Q3, Recruitment, Site documents	—	Implemented. No deletion
Jobs	Public job search, detail, apply; **`JobApplicationsService.transitionStatus`** (the single write path for application status)	`modules/jobs/jobs.service.ts`, `job-application.service.ts`	Payments (subscription gate), Candidates	Employers, Recruitment, Q2, Q3, Exports	Public, Candidate	Implemented
Payments	Plans, Razorpay orders/verify/webhook, BillDesk orders, subscriptions	`modules/payments/*`	Candidates, Prisma, Audit	Jobs (`hasActiveSubscription`)	Public, Candidate	Razorpay implemented; BillDesk settle unreachable
Recruitment	Legacy QC screens + shared hiring services: CV referral lifecycle, interview rounds, offers, scoring, mapping access, staff directory, CV-expiry processor	`modules/recruitment/*`	Jobs, Candidates, Storage, Email, Audit, BullMQ `cv-expiry`	Employers, Q2, Q3, Exports	QC1, QC2, Q3, Client, Candidate, Admin (per route)	Implemented; overlaps with Q3 logic
Employers (`/clients`)	Employer portal API	`modules/employers/*` (`clients/` = deprecated re-exports)	Jobs, Recruitment (InterviewRound, CvReferral), Storage, Audit	Exports	Client (Admin listed, gets 404)	Implemented. Team, invoice payment missing
Q1	Screening workspace + analytics	`modules/q1/*`, `shared/screening.ts`	Candidates, Prisma, Audit	Q2 (writes `SubscriberScreening` on send-back)	QC1, Admin	Implemented
Q2	Matching workspace + analytics	`modules/q2/*`, `shared/matching.ts`	Jobs, Recruitment (Scoring, CvReferral), Storage	Q3 (reuses `Q2Service.factsFor`)	QC2, Admin	Implemented
Q3	Hiring-ops workspace: referrals, company review, scheduling, interviews, decisions, documents, offers, returns, report	`modules/q3/*`, `shared/hiring.ts`	Jobs, Recruitment (CvReferral, OfferLetter), Q2 (`Q2Service` completeness loader, `q3.module.ts:22`), Storage, Email	—	Q3, Admin	Implemented
Admin	Users, staff accounts, employer approval, job moderation, applications (read-only), audit, settings, blog, stats	`modules/admin/*`	Prisma, Email, Audit, `recruitment/staff-directory.ts`	—	Admin	Implemented. Settings not enforced
Exports	CSV/XLSX/PDF reports	`modules/exports/*`	Employers, Candidates, Recruitment, JobApplications	Admin reports page	QC1, QC2, Client, Admin	Implemented; two filter bugs

Module	Purpose	Main files	Depends on	Used by	Roles	Status
Notifications	BullMQ `notifications` queue + worker, email/SMS providers, SSE in-app notifications	`modules/notifications/*`	Redis, provider SDKs	Email service (all mail), web SSE	Any	Email works; SMS never called; in-app never populated
Audit	`tblAuditLog`, login/logout rows	`modules/audit/audit.service.ts`	Prisma	Most modules	—	Implemented (gaps listed in TESTING_AND_ISSUES)
Companies	Public company search/page	`modules/companies/*`	Prisma	Public pages	Public	Implemented
Site documents	Public downloadable documents	`modules/site-documents/*`	Storage	Student guide page (`features/public/pages/StudentGuidePage.tsx`)	Public	Implemented (CLI publishing only)
Health	Liveness/readiness	`modules/health/health.controller.ts`	Prisma, Redis	Docker HEALTHCHECK, CI wait-on	Public	Implemented

3. Shared services and utilities (backend)

Utility	Location	Consumers
`JobApplicationsService.apply` / `transitionStatus`	`modules/jobs/job-application.service.ts`	Jobs, Recruitment, Employers, Q2, Q3 (every status change)
`CvReferralService`	`modules/recruitment/cv-referral.service.ts`	Q2 (forward), Q3 (forward/return/resend), Employers (`markShortlisted`), CV-expiry processor
`InterviewRoundService`	`modules/recruitment/interview-round.service.ts`	Recruitment routes, Employers
`OfferLetterService`	`modules/recruitment/offer-letter.service.ts`	Recruitment routes, Q3 documents
`ScoringService`	`modules/recruitment/scoring.service.ts`	Recruitment, Q2
`MappingAccessService`	`modules/recruitment/mapping-access.service.ts`	Recruitment routes (ownership for Candidate/Client)
`staffDirectory`	`modules/recruitment/staff-directory.ts`	Admin, Q2/Q3 mails
`createStaffLogin`	`modules/admin/staff-account.ts`	Admin, `scripts/create-admin.ts`
`EmailService`	`common/email/email.service.ts`	Auth, Admin, Recruitment, Q3
`AuditService`	`modules/audit/audit.service.ts`	Auth, Candidates, Files, Employers, Jobs, Payments, Recruitment, Q1, Q2, Q3, Admin, Exports
Vocabularies	`shared/roles.ts`, `status.ts`, `screening.ts`, `matching.ts`, `hiring.ts`	API modules; `roles.ts` is mirrored in `apps/web/src/types/roles.ts`

4. Frontend modules and shared code

Frontend area	Folder	API module(s) used	Shared code it relies on
Auth	`features/auth`	Auth	`lib/axios`, `lib/tokenStorage`, `auth.store`
Public & home	`features/public`, `features/home`	Companies, Payments (plans), Site docs	`components/layout/PublicLayout`, `Seo`
Jobs	`features/jobs`	Jobs, Candidates (saved jobs), Payments (gate)	`JobsLayout`, searchable selects
Candidate portal	`features/candidates/portal` (+ legacy `features/candidates/pages`)	Candidates, Files, Jobs, Recruitment (rounds, offers), Notifications	`CandidatePortalLayout`, `profileCompletion.ts`, `useSSE`
Payments	`features/payments`	Payments	Razorpay checkout script
Employer portal	`employer/` (+ public company page in `features/clients`)	Employers (`/clients`), Recruitment (offers/rounds), Exports	`EmployerLayout`, `employer/components/Tables/DataTable`
Q1	`features/q1`	Q1, Recruitment (resume)	`Q1Shell`
Q2	`features/q2`	Q2, Recruitment (resume)	`Q2Shell`

Frontend area	Folder	API module(s) used	Shared code it relies on
Q3	`features/q3`	Q3	`Q3Layout` / `Q3Shell`, toast provider
Legacy recruitment	`features/recruitment`	Recruitment	`DashboardLayout`, `components/ui/Table`
Admin	`features/admin`	Admin, Exports	`AdminShell`, `admin/components/primitives`
Cross-cutting	`components/ui`, `components/layout`, `hooks`, `lib`	—	Used by all areas

5. How the modules communicate

- **Synchronous calls**: the only runtime channel between backend modules is direct NestJS dependency injection. There is no internal HTTP or event bus.
- **Database**: modules share state through common tables. Every status change goes through `tblJobSubscriberMapping.JobMapStatusID`, plus that module's own state tables (`tblSubscriberScreening`, `tblApplicationMatchReview`, `tblCvReferral`, `tblInterviewRound`, `tblApplicationHiring`, `tblOfferLetter`). These parallel state stores can drift; see TESTING_AND_ISSUES W-01…W-07.
- **Asynchronous**: BullMQ on Redis carries two queues, `notifications` (emails) and `cv-expiry` (hourly).
- **Frontend to backend**: REST over `/api` through one axios instance, with TanStack Query caching. Each feature invalidates its own key prefix (`['q1']`, `['q2']`, `['q3']`, `['admin']`, `['employer']`, `['candidate']`).
- **Server to browser push**: an SSE stream at `/api/notifications/stream` exists, but no producer publishes to it.

6. Implementation status summary

Status	Modules / features
Fully implemented	Auth (email OTP), candidate profile, public job search/apply, employer portal core, Q1, Q2, Q3, admin panel, Razorpay payments, exports, storage, email delivery, CV expiry job, audit
Partially implemented	Notifications (email only; in-app and SMS unused), payments (BillDesk path dead; no receipts), candidate privacy/export/deletion/referrals (recorded only), notification preferences (4 of 12 saved), legacy `/recruitment/*` screens (Q3 menu broken), employer analytics (exports truncated)
Not found / placeholder	Application withdrawal, employer team users, messaging, employer notifications/settings pages, company reviews, salary insights, public candidate profile, global search, OAuth, CV parsing, invoice creation, platform-settings enforcement, role 6 "Subscription"

Database Schema

> Derived from `apps/api/prisma/schema.prisma` at commit `8de48cf`. The database is **PostgreSQL 16**, accessed through **Prisma 7** with the `@prisma/adapter-pg` driver adapter. The schema came from a restore of the legacy SQL Server database (`db_aajiveka.bak`). That database had **73 tables and no declared foreign keys**, and 36 of its tables had no primary key. The relations in this schema were worked out from the JOINs in the legacy stored procedures (`db/procs/`) and then checked against the data with an orphan test. See `db/ER-DIAGRAM.md`, `db/SCHEMA_REPORT.md` and `db/foreign-keys.psv` (63 FK edges, each with a confidence rating).

Related docs: PROJECT_OVERVIEW.md covers infra, deploy and environment, COMPLETE_WORKFLOW.md and MASTER_SYSTEM_FLOW.md the pipeline, ROLE_PERMISSION_MATRIX.md the roles, API_DOCUMENTATION.md the endpoints, MODULE_DEPENDENCIES.md the modules, and TESTING_AND_ISSUES.md the tests and known issues.

Verified vs inferred. Statements in this document were checked against the schema, the migrations or the source, unless they are marked *(inferred)*. `schema:N` means `apps/api/prisma/schema.prisma` line N. Other paths are relative to the repo root, and source paths are under `apps/api/src/` unless they say otherwise.

Contents

1. Conventions you must know first
2. Data domains at a glance
3. ER diagram (core entities)
4. Model reference by domain
5. Status and enum value sets
6. Tricky, polymorphic and misnamed columns
7. Data lifecycle
8. Migration timeline
9. Seeding and data migration
10. Data-layer issues

---

1. Conventions you must know first

Convention	Detail
Table and column names	Every table keeps its legacy SQL Server name through `@@map("tblXxx")`, and every column keeps its PascalCase name through `@map("Xxx")`. In raw SQL you must quote identifiers: `"tblClientJobs"."JobID"`.
Prisma field names	Fields are camelCase, and some are mechanically odd: `pIN`, `dOB`, `cVPath`, `tImestampIns`, `uSERID`, `iD`. Look up the actual field name in the schema; don't guess it.
Connection config	Prisma 7 no longer keeps the URL in `schema.prisma`. It lives in `apps/api/prisma.config.ts`, which reads the `DATABASE_URL` env var (and the optional `SHADOW_DATABASE_URL`). The client is created in `apps/api/src/prisma/prisma.client.ts`.
Audit columns	Legacy tables carry `TimestampIns / LoginIDIns / TimestampUpd / LoginIDUpd`. New tables use `CreatedAt / UpdatedAt`, or `TimestampIns @default(now())`.
Status columns	Legacy status columns are **int ids** that point at master tables. New tables (Q1/Q2/Q3, offers, invoices, blog) use **free-text VARCHAR** statuses. No status column has a CHECK constraint, so the vocabularies are enforced only in TypeScript (`shared/status.ts`, `shared/screening.ts`, `shared/matching.ts`, `shared/hiring.ts`).
Enums	There is only one Prisma enum: `PaymentStatus` (schema:1310).
onDelete	Where `onDelete` isn't written, Prisma's default applies: **Restrict** for a required relation, **SetNull** for an optional one, with ON UPDATE CASCADE in both cases. The init migration emitted 27 `ON DELETE RESTRICT` and 39 `ON DELETE SET NULL`. Only two relations are explicitly `Cascade`: `JobApplicationDetail → JobSubscriberMapping` (schema:1538) and `InterviewSlot → InterviewRound` (schema:1639).
Id types	Most PKs are `BigInt` (bigserial). A few columns that hold the same concept are `Int`; see §6 and I20.
Omitted legacy tables	13 dead tables (no rows, and no procedure references them) are left out. They are listed in `db/dead-tables.txt`.

---

2. Data domains at a glance

The schema has 94 models and 1 enum.

Domain	# models	Models
Security, users, sessions	12	SecUser, SecMapUserRoles, SecRoles, SecUserLogin, SecActiveSessions, AuthPasswordReset, ForgotPassword, MstrPerson, PMstNodeTypes, SecMenuHierarchy, SecMenuHierarchyRoles, SecMenuContextMenu
Candidate (subscriber) profile	21	SubscriberRegistration, SubscriberCVDetails, SubscriberCVUploaded, SubscriberProfileExtra, SubscriberEducation, SubscriberEmployer, SubscriberCertificate, SubscriberITSkill, SubscriberProject, SubscriberAccomplishment, SubscriberLanguage, SubscriberTags, SubscriberPrefferedLocations, SubscriberSkills, SubscriberAwards, SubscriberJobAlert, SavedJob, SavedSearch, NotificationPreference, CandidatePrivacy, CandidateReferral
Masters / lookups	25	All `Mstr*` models except MstrPerson (see §4.12)
Companies / employers	5	ClientMstr, ClientContacts, CompanyBranding, EmployerRegistration, ClientInvoice
Jobs	4	ClientJobs, ClientJobSkill, ClientJobs_EducationType, ClientJobs_Gendermapping
Application pipeline core and history	6	JobSubscriberMapping, JobSubscriberStatus, SubscriberStatusHistory, SubscriberJobStatusLatest, JobApplicationDetail, ApplicantNote
Q1 screening	2	SubscriberScreening, ScreeningContactLog
Q2 matching	2	CandidateJobScore, ApplicationMatchReview
Q3 hiring ops / interviews	6	CvReferral, InterviewRound, InterviewSlot, JobInterviewStatus (legacy), ApplicationHiring, Q3Activity
Documents and offers	4	CandidateDocumentMap, CandidateDocumentUploaded, CandidateDocumentStatus, OfferLetter
Payments / subscriptions	3	SubscriptionPlan, PaymentOrder, Subscription
Platform, content, audit	4	AuditLog, SiteDocument, BlogPost, PlatformSetting
Total	**94**	

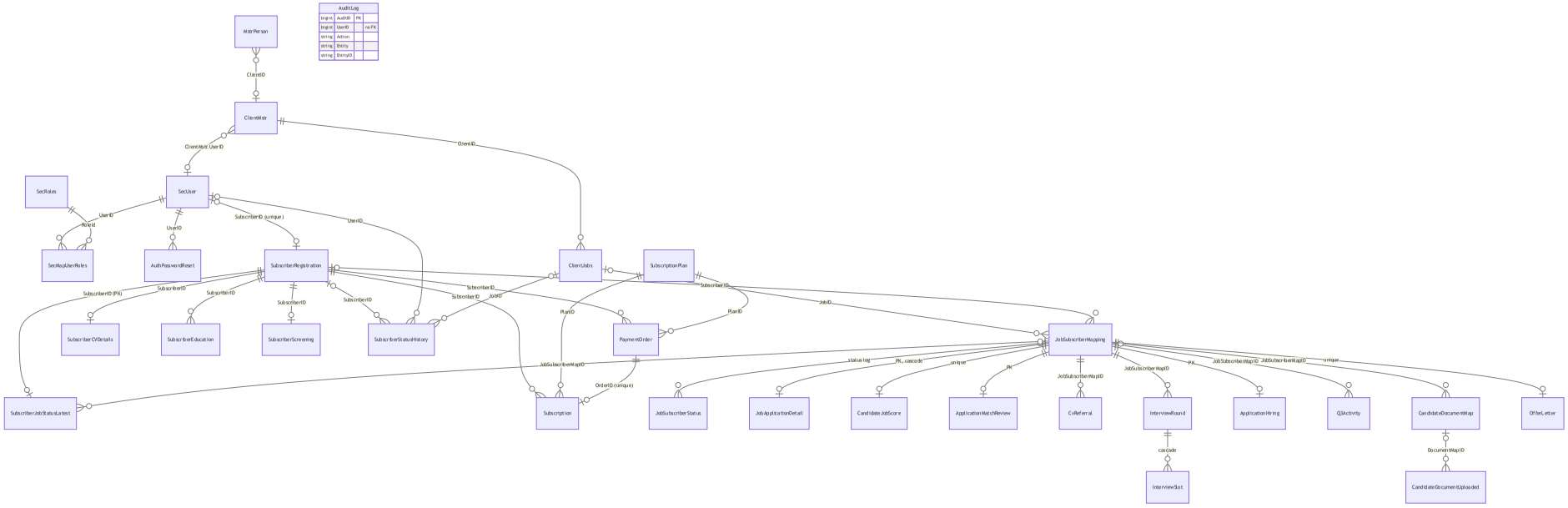
The central table is `JobSubscriberMapping` (tblJobSubscriberMapping). One row is one candidate's application to one job. Almost every workflow table hangs off its `JobSubscriberMapID`. **The root of a candidate is `SubscriberRegistration`**, and **the root of a login is `SecUser`**.

There is **no notifications table**. In-app notifications live in an in-memory map in `modules/notifications/sse-notifications.service.ts:9-26` and are not persisted. Email and SMS go through a BullMQ queue.

---

3. ER diagram (core entities)

The diagram shows 30 core entities with **only the relations that have a real DB foreign key**. Lookups are listed after it, and logical links without an FK are listed in prose below the diagram.


AuditLog
bigint AuditID PK
bigint UserID no FK
string Action
string Entity
string EntityID
MstrPerson
ClientID
ClientMstr
ClientMstr UserID
ClientID
SecRoles
SecUser
RoleId
UserID
UserID
SubscriberID (unique)
UserID
SecMapUserRoles
AuthPasswordReset
SubscriberRegistration
ClientJobs
SubscriptionPlan
SubscriberID (FK)
SubscriberID
SubscriberID
SubscriberID
SubscriberID
SubscriberID
JobID
SubscriberID
PlanID
PlanID
JobID
SubscriberCVDetails
SubscriberEducation
SubscriberScreening
SubscriberStatusHistory
PaymentOrder
JobSubscriberMapping
JobSubscriberMapID
OrderID (unique)
SubscriberJobStatusLatest
Subscription
JobSubscriberStatus
JobApplicationDetail
CandidateJobScore
ApplicationMatchReview
CvReferral
InterviewRound
ApplicationHiring
QJActivity
CandidateDocumentMap
OfferLetter
statusLog
PK, cascade
unique
FK
JobSubscriberMapID
JobSubscriberMapID
FK
JobSubscriberMapID
JobSubscriberMapID
unique
cascade
InterviewSlot
DocumentMapID
CandidateDocumentUploaded

Logical links that have no DB foreign key. The diagram leaves these out on purpose.

From	To	Why there's no FK
`SecUser.NodeID`	`MstrPerson.PersonNodeID` **or** `SubscriberRegistration.SubscriberID`	NodeID is polymorphic (§6.1). An FK was created by the init migration and dropped in `20260713140000`.
`SecActiveSessions.UserID` (Int)	`SecUser.UserID` (BigInt)	The refresh-token session store has no FK and no index.
`AuditLog.UserID`	`SecUser.UserID`	Deliberate, so that audit rows survive user deletion.
`ClientInvoice.ClientID`	`ClientMstr.ClientID`	The FK **exists in the DB** (migration `20261005130000`) but **not in the schema**. That is drift; see I2.
`Q3Activity.InterviewRoundID`	`InterviewRound`	No FK.
`SubscriberStatusHistory.JobSubscriberMapID`	`JobSubscriberMapping`	No FK (it has the column only).
`JobSubscriberStatus.JobMapStatusID`	`MstrJobMappingStatus`	No FK.
`CandidateDocumentStatus.DocUploadID` / `UserID`	`CandidateDocumentUploaded` / `SecUser`	No FKs and no indexes.
`CandidateDocumentMap.SubscriberID`, `CandidateDocumentUploaded.SubscriberID`	`SubscriberRegistration`	No FK and no index.
`SavedSearch.SubscriberID`, `NotificationPreference.SubscriberID` (both Int)	`SubscriberRegistration` (BigInt)	No FK, and the type differs.
`SubscriberLanguage/Skills/Awards.SubscriberID`	`SubscriberRegistration`	No FK.
`EmployerRegistration`	the `ClientMstr` / `SecUser` created on approval	There is no link column at all.
`ClientJobs.StatusID`	(nothing)	This is a job-status flag (1–4), not an FK. The FK to MstrStatus was dropped in `20260713130000`.
All "who did it" BigInt columns on the new tables (`VerifiedBy`, `DecidedBy`, `ReferredByUserID`, `ValidatedByUserID`, `ReturnedBy`, `CreatedBy`, `ScoredBy`, `ContactedBy`, `ReviewedBy`, `CreatedByUserID`), plus the Int columns `ApplicantNote.UpdatedBy` and `PlatformSetting.UpdatedBy`	`SecUser`	No FK.

Core-adjacent tables left out of the diagram (all of them have real FKs to the tables shown):

Table	Relation
SecUserLogin	→ SecUser (SetNull)
ClientContacts, CompanyBranding	→ ClientMstr
ClientJobSkill, ClientJobs_EducationType, ClientJobs_Gendermapping, SavedJob	→ ClientJobs
SubscriberCVUploaded, SubscriberProfileExtra, SubscriberEmployer, SubscriberCertificate, SubscriberITSkill, SubscriberProject, SubscriberAccomplishment, SubscriberTags, SubscriberPrefferedLocations, SubscriberJobAlert, SavedJob, CandidatePrivacy, CandidateReferral, ScreeningContactLog	→ SubscriberRegistration
ApplicantNote, JobInterviewStatus (legacy)	→ JobSubscriberMapping

Lookup / master tables referenced by the core tables:

Lookup	Referenced by
MstrCountry ← MstrState ← MstrCily	ClientMstr.CityID, ClientJobs.JobCityID, SubscriberCVDetails.CityID / CurrentCityID, SubscriberPrefferedLocations, MstrInstitute
MstrDesignation	ClientJobs, SubscriberEmployer
MstrEmpType	ClientJobs.EmployeeTypeID, SubscriberEmployer
MstrWorkMode	ClientJobs
MstrIndustryType	ClientJobs, ClientMstr, SubscriberCVDetails
MstrSkills ← MstrTags ← SubscriberTags	MstrSkills is also referenced by ClientJobSkill and SubscriberCVDetails.SkillID
MstrFunctions ← MstrSubFunctions	SubscriberCVDetails.SubFunctionID

Lookup	Referenced by
MstrEducationType ← MstrCourse	SubscriberEducation.DegreeID → MstrEducationType; SubscriberEducation.CourseTypeID → MstrCourse; ClientJobs_EducationType
MstrInstitute	standalone (State/City FKs only)
MstrJobMappingStatus	JobSubscriberMapping.JobMapStatusID, SubscriberJobStatusLatest
MstrStatus	SubscriberStatusHistory.StatusID only
MstrInterviewMode	JobInterviewStatus, InterviewRound
MstrDocumentType	CandidateDocumentMap, CandidateDocumentUploaded
Standalone / unused	MstrDocuments, MstrGender, MstrCourseType, MstrDegreeType, MstrEducationDegree, MstrDocumentStatus, MstrInviewStatus, PMstNodeTypes

---

4. Model reference by domain

Notation: `→ Target (onDelete)`. Line numbers are where the model starts in `schema.prisma`.

4.1 Security, users, sessions

Model (line) → table	Purpose	Key columns	Relations	Constraints / indexes
SecUser (659) → tblSecUser	The login account for every role	PK `UserID` bigserial; `NodeID` + `NodeType` (polymorphic, §6.1); `UserName` varchar(50); `Password` (argon2id hash); `PwdStatus` (0 = must change on first login); `Active` char(1) `'1'` / `'0'`; `LoginType`; `RoleID` (denormalised; SecMapUserRoles is authoritative); `UserMail`; `SubscriberID` (new)	`SubscriberID` → SubscriberRegistration (SetNull)	`SubscriberID` @unique; indexes on `UserName` and `Active`. **`UserName` is not unique.**
SecMapUserRoles (593) → tblSecMapUserRoles	user ↔ role, with a node per mapping	PK `UserRoleMapID`; `UserID`, `RoleId`, `UserNodeId`, `UserNodeType` (all NOT NULL)	→ SecUser (Restrict); → SecRoles (Restrict)	index userID; **no unique (UserID, RoleId)**
SecRoles (647) → tblSecRoles	Role master	PK `RoleId` int; `RoleName`	—	Seeded: 1 Subscriber, 2 QC1, 3 QC2, 4 Client, 5 Admin, 7 Q3. **There is no row for 6** (Subscription).
SecUserLogin (697) → tblSecUserLogin	Login log (legacy, still written by `audit.recordLogin`, auth.service.ts:163)	PK `LoginID`; `UserID` ?; `LoginTime`; `SessionID`, `IPAddress`, `IEVersion`, `NodeID`, `NodeType`	→ SecUser (SetNull; 647 of 2,225 legacy rows were orphans)	index userID
SecActiveSessions (585) → tblSecActiveSessions	**Refresh-token session store**: one row per issued refresh `jti`	PK `RowID` serial; `SessionID` varchar(50) (= jti); `UserID` **Int**; `StartTime`	**none**	**no indexes**
AuthPasswordReset (1228) → tblAuthPasswordReset	Password-reset and set-password tokens (new)	PK `ResetID`; `TokenHash` varchar(64) (SHA-256); `ExpiresAt`; `UsedAt`	→ SecUser (Restrict)	`TokenHash` @unique; index userID
ForgotPassword (207) → tblForgotPassword	Legacy. It has no token and no expiry, so it can't be used	`forgotPasswordID`, `USERID`	none	**unused in src**
MstrPerson (495) → tblMstrPerson	The "person node" for every non-candidate login (staff, admin, employer HR)	PK `PersonNodeID`; `Descr` (name); `EmailID` NOT NULL (not unique); `NodeType`; `flgActive`; `ClientID`	`ClientID` → ClientMstr (SetNull; legacy sentinel 0 → NULL)	**no index on `EmailID` or `ClientID`**
PMstNodeTypes (570)	Legacy NodeType discriminator (100 = "Subscriber")	PK `NodeType` smallint	—	unused in src
SecMenuHierarchy (628), SecMenuHierarchyRoles (639), SecMenuContextMenu (606)	Legacy ASP.NET menu tables	—	none	unused in src

4.2 Candidate (subscriber) profile

Model (line) → table	Purpose	Key columns	Relations	Constraints / indexes
SubscriberRegistration (945) → tblSubscriberRegistration	Root candidate identity	PK `SubscriberID` bigserial; `RegistrationCountryCode` ; `RegistrationMobileNo` varchar(15); `RegistrationIPNo` ; `RegistrationDateTime` ; `flgCVUploaded` ; `flgstatus` smallint? (the QC registration decision, see §5.4)	parent of about 25 relations	**no unique on the mobile number**
SubscriberCVDetails (732) → tblSubscriberCVDetails	1:1 core CV and profile row	PK/FK `SubscriberID` ; `FullName` ; `MobileNo1` ; `EmailID` ; `EmailVerified` / `EmailVerifiedAt` ; `OnboardedAt` (NULL means onboarding isn't done); `DOB` ; `Gender` ; `TotalExp` (years); `CurrentCTC` ; `CityID` , `CurrentCityID` , `SkillID` , `SubFunctionID` , `IndustryTypeID` ; `NoticePeriod` ; `PhotoName` ; `CVPath`	→ SubscriberRegistration (Restrict); city ×2, industry, skill, subFunction (SetNull)	indexes skillID, currentCityID, industryTypeID, emailID. **`EmailID` is deliberately not unique** (schema:776-780); uniqueness is enforced in `AuthService.assertEmailAvailable` .
SubscriberCVUploaded (784)	1:1 latest CV file	PK/FK `SubscriberID` ; `LatestCVPath` ; `CVName` ; `SizeBytes` ; `PageCount`	→ SubscriberRegistration (Restrict)	—
SubscriberProfileExtra (1039)	1:1 single-valued profile sections (new)	headline, summary, `TotalExpMonths` , department/role, desired job/employment type (CSV), `PreferredShift` , `PreferredWorkModes` (CSV), `PreferredSalary` , `PreferredJobRoles` (CSV), `KeySkills` (CSV display copy), personal and diversity fields	→ SubscriberRegistration (Restrict)	—
SubscriberEducation (827)	Education rows	`CourseTypeID` (**= MstrCourse.DegreeID**), `DegreeID` (**= MstrEducationType.EducationTypeID**), `InstituteName` , `StartYear` , `PassingYear` , `Specialization` , `CourseMode` , `Marks`	→ MstrCourse, MstrEducationType (SetNull); → SubscriberRegistration (Restrict)	index subscriberID
SubscriberEmployer (867)	Employment history	`Employer` , `DesignationID` , `EmployeeTypeID` , `JoiningDate` , `ReleavingDate` , `Salary` , `NoticePeriodDays` , `flgCurrent`	→ SubscriberRegistration, MstrDesignation, MstrEmpType (all SetNull)	index subscriberID
SubscriberCertificate (802)	Certifications	`CertificateName` , `CertificateUrl` , `CertificationID` , valid-from/till, `flgNeverExpires`	→ SubscriberRegistration (Restrict)	**no index on SubscriberID**
SubscriberITSkill (1114)	IT skills (new)	`SkillName` , `Version` , `LastUsedYear` , `ExpYears/Months`	→ SubscriberRegistration (Restrict)	index
SubscriberProject (1134)	Projects (new)	title, client, status, dates, site, team size	→ SubscriberRegistration (Restrict)	index
SubscriberAccomplishment (1168)	Online profiles, work samples, publications, presentations, patents	`Kind` discriminator: ONLINE_PROFILE / WORK_SAMPLE / PUBLICATION / PRESENTATION / PATENT	→ SubscriberRegistration (Restrict)	index (subscriberID, kind)
SubscriberLanguage (910)	Languages	`LanguageID` (dangling: no master table exists), `LanguageName` , flgRead/Write/Speak, `ProficiencyID` 1–3	**no FK**	index subscriberID
SubscriberTags (1013)	Candidate ↔ master tag (searchable key skills)	composite PK (SubscriberID, TagID)	→ SubscriberRegistration, MstrTags (Restrict)	—
SubscriberPrefferedLocations (931)	Preferred cities (*sic* "Preffered")	composite PK (SubscriberID, CityID)	→ SubscriberRegistration, MstrCily (Restrict)	—
SubscriberSkills (981), SubscriberAwards (718)	Legacy	—	no FKs	**unused in src**

Model (line) → table	Purpose	Key columns	Relations	Constraints / indexes
SubscriberJobAlert (1195)	Job alerts (new)	`Keyword`, `Location`, `Frequency`	→ SubscriberRegistration (Restrict)	index
SavedJob (1210)	Bookmarked jobs	—	→ SubscriberRegistration, ClientJobs (Restrict)	**@@unique(subscriberID, jobID)**
SavedSearch (1404)	Saved searches	`SubscriberID` **Int**; `Filters` (JSON as TEXT)	**no FK**	index
NotificationPreference (1416)	Channel and per-topic opt-ins	PK `SubscriberID` **Int**; email/push/sms bools; `JobAlertFrequency`; 8 topic bools	**no FK**	—
CandidatePrivacy (1495)	Privacy flags and export/deletion requests	PK/FK `SubscriberID`; `ShowCurrentEmployer`; `AllowRecruiterMessages`; `ExportRequestedAt`; `DeletionRequestedAt`	→ SubscriberRegistration (Restrict)	—
CandidateReferral (1470)	"Refer a friend"	`ReferralID`; `Name`, `Email`, `Mobile`; `Status` Invited → Signed Up → Applied → Rewarded; `RewardPaise`; `RewardPaid`	→ SubscriberRegistration (Restrict)	index subscriberID. There is no link to the invitee's eventual SubscriberID.

4.3 Companies / employers

Model (line) → table	Purpose	Key columns	Relations	Constraints / indexes
ClientMstr (174) → tblClientMstr	A company	PK `ClientID`; `ClientName`; `CityID` NOT NULL; `IndustryTypeID`; `UserID`; logo, website, descr	→ MstrCily (Restrict); → MstrIndustryType (SetNull); `UserID` → SecUser (SetNull)	indexes cityID, userID, industryTypeID
ClientContacts (71)	Company contacts (HR row uses roleID 4)	`ClientID`, `RoleID`, names, phones	→ ClientMstr (Restrict)	index
CompanyBranding (1453)	1:1 branding	tagline, cover, culture, `Benefits` (JSON TEXT)	PK/FK → ClientMstr (Restrict)	—
EmployerRegistration (1551) → tblEmployerRegistration	Pending employer self-registrations awaiting admin approval	`Status` Pending/Approved/Rejected; `EmailCompany` NOT NULL; company fields; `ReviewedBy` (BigInt, no FK); notes	**none**	index status; `EmailCompany` **not unique**
ClientInvoice (1646) → tblClientInvoice	Invoices the admin raises to a company	`InvoiceNo`; `InvoiceDate`; `DueDate`; `Amount/Tax/Total` decimal(14,2); `Status` Unpaid/Paid/Cancelled; `PaidAt`; `CreatedByUserID`	**none in the schema** (but the DB has an FK, see I2)	`InvoiceNo` @unique; index clientID

4.4 Jobs

Model (line) → table	Key columns	Relations	Constraints / indexes
ClientJobs (98) → tblClientJobs	PK `JobID`; `ClientID`, `DesignationID`, `EmployeeTypeID`, `WorkModeID`, `JobCityID` (all NOT NULL); `IndustryTypeID` ?; **`StatusID`** int? = job status 1–4 (§5.2), **not an FK**; `MinExp` / `MaxExp` / `MaxEmp`; `MinCTC` / `MaxCTC` NOT NULL; text sections `JobDescr`, `JobCandidateProfile`, `KeyResponsibilities`, `PreferredQualifications`, `InterviewProcess`; `EducationDetail`, `ReportTo`, `TeamSize`, `Department`, `SubDepartment`	→ ClientMstr, MstrDesignation, MstrEmpType, MstrCily, MstrWorkMode (Restrict); → MstrIndustryType (SetNull)	indexes clientID, statusID, designationID, jobCityID, timestampIns, industryTypeID
ClientJobSkill (86)	`JobSkillID` PK; job ↔ skill	→ ClientJobs, MstrSkills (Restrict)	**no unique (JobID, SkillID)**
ClientJobs_EducationType (153)	composite PK (JobID, EducationTypeID)	Restrict	—
ClientJobs_Gendermapping (164)	composite PK (JobID, Gender char(1))	Restrict	Gender has no FK to MstrGender

4.5 Application pipeline core and history

Model (line) → table	Purpose	Key columns	Relations	Constraints / indexes
JobSubscriberMapping (233) → tblJobSubscriberMapping	**The application**: one candidate on one job	PK `JobSubscriberMapID` ; `JobID` ?; `SubscriberID` ?; `MapDate` ; `JobMapStatusID` ? (1–23, §5.1)	job, subscriber, status → all SetNull	indexes jobID, subscriberID, jobMapStatusID. **No unique (JobID, SubscriberID)**.
JobSubscriberStatus (266) → tblJobSubscriberStatus	**Per-application status log**; one row per transition	PK `StatusID` (an identity, *not* a status id); `JobMapStatusID` (no FK); `MappedbyUserID` int; `comments`	→ JobSubscriberMapping (SetNull)	**no index on JobSubscriberMapID**
SubscriberStatusHistory (992) → tblSubscriberStatusHistory	**Candidate journey history** (MstrStatus ids)	`SubscriberID` , `JobID` , `ClientID` , `JobSubscriberMapID` (no FK), `StatusID` , `UserID` , `comments` varchar(5000)	subscriber, job, client, status, user → SetNull	index subscriberID only
SubscriberJobStatusLatest (891) → tblSubscriberJobStatusLatest	**Latest-application snapshot per candidate** (PK SubscriberID)	`ClientID` , `JobID` , `JobSubscriberMapID` , `JobMapStatusID` (legacy sentinel 0 → NULL), `flgClose` , `flgApprovedByQC`	all SetNull	PK subscriberID
JobApplicationDetail (1519) → tblJobApplicationDetail	Snapshot of the submitted apply form	PK = `JobSubscriberMapID` ; name, email, phone, salary text, notice, links, resume filename, cover letter	→ mapping (**Cascade**)	—
ApplicantNote (1441) → tblApplicantNote	Employer notes on an applicant	`NoteID` ; `UpdatedBy` Int	→ mapping (Restrict)	**no index on JobSubscriberMapID**

4.6 Q1 screening

Model (line) → table	Purpose	Key columns	Relations	Indexes
SubscriberScreening (1834) → tblSubscriberScreening	1:1 Q1 screening state per candidate	`Status` (§5.5); `Priority` High/Medium/Low; 7 `Chk*` booleans; `RelevantExpMonths` ; follow-up date/time; `ContactAttempts` ; `NextAttemptAt` ; `NotInterestedReason` ; `FirstReviewedAt` ; `VerifiedAt/By`	→ SubscriberRegistration (Restrict)	status, priority, followUpDate
ScreeningContactLog (1889) → tblScreeningContactLog	Append-only Q1 contact log	`Kind` Contact/ProfileUpdateRequest/CvRequest; `Channel` Call/WhatsApp/Email/Message; `Reason` ; `MissingItems` (CSV); `InternalNote` ; `ContactedAt/By`	→ SubscriberRegistration (Restrict)	subscriberID, contactedAt

4.7 Q2 matching

Model (line) → table	Purpose	Key columns	Relations	Constraints / indexes
CandidateJobScore (1706) → tblCandidateJobScore	7 sub-scores and a weighted `TotalScore` (weights 30/25/20/10/5/5/5)	`ScoredBy`	→ mapping (Restrict)	`JobSubscriberMapID` @unique. A TotalScore index exists only in the migration.
ApplicationMatchReview (1731) → tblApplicationMatchReview	Q2 review state	PK = `JobSubscriberMapID` ; `Status` (§5.6); 7 `Inc*` criteria flags (all false means score on all seven); `DecisionNote` ; `DecidedAt/By`	→ mapping (Restrict)	status, decidedAt

4.8 Q3 hiring ops and interviews

Model (line) → table	Purpose	Key columns	Relations	Constraints / indexes
CvReferral (1668) → tblCvReferral	Q2 → Q3 → company referral of a CV	`Status` and `CompanyReviewStatus` (§5.7); `ReferredByQ2At` , `ReferredByUserID` ; `ValidatedByQ3At/ByUserID` ; `SentToCompanyAt` ; `ExpiresAt` (14-day clock); `CompanyShortlistedAt` ; return fields `ReturnedAt/By` , `ReturnReason` , `ReturnNote` , `Q2MailSentAt` , `Q2AcknowledgedAt` , `ReReferredAt`	→ mapping (Restrict)	indexes mapping, status, expiresAt, returnedAt. **1:N**: no unique on the mapping, and a re-referral creates a new row.
InterviewRound (1577) → tblInterviewRound	Multi-round interviews (R1… Final) per application	`RoundNumber` ; `RoundName` ; interviewer/HR/guest names and emails; `Interviewers` (JSON TEXT); `InterviewModeID` ; `MeetingLink` ; `Location` ; `ScheduledAt` ; `DurationMins` (default 45); `IsFinal` ; `Status` / `Result` / `Decision` (§5.8); many lifecycle timestamps (slots shared/requested, invites sent, reminder, candidate confirmed/joined, conducted, result due, decided, decision mail)	→ mapping (Restrict); → MstrInterviewMode (SetNull)	indexes mapping, scheduledAt, status. **No unique (JobSubscriberMapID, RoundNumber).**
InterviewSlot (1631) → tblInterviewSlot	3–4 offered slots per round	`SlotDateTime` , `IsSelected` , `IsUnavailable`	→ InterviewRound (**Cascade**)	index

Model (line) → table	Purpose	Key columns	Relations	Constraints / indexes
JobInterviewStatus (214) → tblJobInterviewStatus	**Legacy** flat interview record, still written by `/recruitment/interviews`	`InterviewTime`, `InterviewScheduledOn`, `InterviewModeID`, `InterviewLocation`, `PraposedTime`	→ mapping (SetNull); → MstrInterviewMode (SetNull)	index
ApplicationHiring (1783) → tblApplicationHiring	Q3 per-application hire state	PK = `JobSubscriberMapID`; `SelectedAt`, docs `Requested/Due/Forwarded/Approved`, `JoinedAt`, `OnHoldAt`	→ mapping (Restrict)	—
Q3Activity (1802) → tblQ3Activity	Append-only Q3 action log	`Kind` (Contact, Nudge, Reminder, SlotRequest, Invites, DecisionMail, DocRequest, DocVerified, DocSentBack, DocsForwarded, Q2Mail); `Channel` Call/Mail/WhatsApp; `Note`; `CreatedBy`; `InterviewRoundID` (no FK)	→ mapping (Restrict)	mapping, createdAt

4.9 Documents and offers

Model (line) → table	Purpose	Key columns	Relations	Notes
CandidateDocumentMap (21) → tblCandidateDocumentMap	Which documents are requested for an application	`JobSubscriberMapID`; `DocumentTypeID`; `SubscriberID`; `flgStatus`	→ mapping (SetNull); → MstrDocumentType (SetNull)	`SubscriberID` has no FK and no index
CandidateDocumentUploaded (49) → tblCandidateDocumentUploaded	Uploaded document files	`DocumentMapID`; `DocumentTypeID`; `SubscriberID`; `DocumentPath` (storage key); `FileName`; `SizeBytes`; **`flgStatus`** (§5.9)	→ CandidateDocumentMap (SetNull); → MstrDocumentType (SetNull)	`SubscriberID` has no FK and no index
CandidateDocumentStatus (38) → tblCandidateDocumentStatus	Review log per upload	`DocUploadID`, `StatusID`, `UserID`, `comments`	**none**	no indexes
OfferLetter (1764) → tblOfferLetter	One offer per application	`OfferDetails` (JSON TEXT); `Status` Draft/Sent/Accepted/Rejected/Expired; `SentAt`; `CandidateResponseAt`; `JoiningDate`	→ mapping (Restrict)	`JobSubscriberMapID` @unique

4.10 Payments / subscriptions

Model (line) → table	Purpose	Key columns	Constraints
SubscriptionPlan (1293) → tblSubscriptionPlan	12 candidate plans (3 tiers × 1/3/6/12 months)	`Tier` entry/mid/senior; `TierLabel`; `Months`; `PriceINR` (whole rupees); `Active`	@@unique(tier, months)
PaymentOrder (1320) → tblPaymentOrder	One payment attempt	`OrderRef`; `SubscriberID` → SubscriberRegistration (Restrict); `PlanID` → plan (Restrict); `AmountINR` (copied from the plan); `Status` **enum PaymentStatus**; **`BDOrderID`** (holds the BillDesk *or* Razorpay order id, §6); `TransactionID`; `AuthStatus`; `PaymentMethod`; `ErrorDescription`; `RawResponse`	`OrderRef` @unique, `TransactionID` @unique; indexes subscriberID, status; **no index on BDOrderID**
Subscription (1353) → tblSubscription	What a paid order buys	`StartsAt` / `EndsAt`; FKs to subscriber, plan and order (Restrict)	`OrderID` @unique, so activation is idempotent

4.11 Platform, content, audit

Model (line) → table	Purpose	Key columns	Constraints
AuditLog (1247) → tblAuditLog	Generic audit trail (new; the legacy app had none)	`UserID` (no FK); `Action` varchar(60); `Entity`; `EntityID` varchar; `IPAddress`; `UserAgent`; `Detail` (JSON TEXT); `TimestampIns`	indexes userID, timestampIns; **no (Entity, EntityID) index**
SiteDocument (1269) → tblSiteDocument	Public files by slug (e.g. `aajivika-book`)	`Slug`; `StorageKey`; `MimeType`; `SizeBytes`; `FileName`; `Active`	`Slug` @unique
BlogPost (1385) → tblBlogPost	Blog CMS	`Slug`; `Status` (free text, default `'Draft'`); `Category`; `Author`; all TEXT	`Slug` @unique; indexes status, createdAt
PlatformSetting (1376) → tblPlatformSettings	Admin key/value settings	PK `Key`; `Value`; `UpdatedBy` Int	Read and written only by admin settings (admin.service.ts:1097, :1116); **nothing enforces them**

4.12 Masters / lookups

All of these use int serial PKs with a `Descr` column unless noted. Seed counts come from `db/seed/*.psv`.

Model → table	Rows (seed)	Notes
MstrCountry	1	India
MstrState	36	`CountryID` → MstrCountry (SetNull)
MstrCily (*sic*)	764	Actually a **district** list. `StateID` → MstrState. Extra cities come from migration `20260810210000` and seed `ensureJobImportMasters`.
MstrDesignation	65	plus about 60 from migration `20260810210000`
MstrSkills	85	plus more from `20260810210000`
MstrTags	7	`SkillID` → MstrSkills (Restrict)
MstrFunctions / MstrSubFunctions	37 / 1566	sub → function (SetNull)
MstrIndustryType	81	column `IndustryType`
MstrEmpType	4	Full Time, Part Time, Internship, Contract
MstrWorkMode	5	Work From Home, Work From Office, In-office, Hybrid, Remote
MstrGender	3	M, F, T (column `Gendor`). Unused.
MstrEducationType	4 → ~90	Qualification level. Migration `20260816120000` adds `Category`, @@unique(descr) and the India qualifications.
MstrCourse	8 + generated	Course within a qualification; @@unique(educationTypeID, degreeName)
MstrEducationDegree	8	A byte-identical copy of MstrCourse. Unused, but still seeded.
MstrCourseType, MstrDegreeType	0	Legacy, unused
MstrInstitute	generated	UGC/AICTE institution list; @@unique(searchKey); `SearchText` has a pattern index that exists only in the migration; queried with raw SQL (candidates.service.ts:338)
MstrDocumentType	13	1 CV, 2 PAN, 3 Aadhar, 4 Passport, 5 Police Verification, 6 Driving Licence, 7–11 marksheets / degree / internship / photo / bank details, 19 CompanyLogo, 20 candidatephoto
MstrDocuments	13	Parallel to DocumentType (same ids), with FolderName / RootFolder used for storage keys
MstrDocumentStatus, MstrInviewStatus	—	Legacy, unused
MstrInterviewMode	4	1 In-person, 2 Telephonic, 3 Video, 4 Walk-in
MstrJobMappingStatus	11 → 23	Application status labels. **Ids 10 and 11 differ by environment** (§6).
MstrStatus	29	Candidate journey statuses (§5.3); `RoleID` is the role that sets each one

---

5. Status and enum value sets

Two separate **integer** status spaces exist, and they are easy to confuse:

- `tblMstrJobMappingStatus` is an **application's** progress.
- `tblMstrStatus` is a **candidate's** journey.

The Q1, Q2 and Q3 workspaces deliberately use **string** status spaces of their own. Constants live in `apps/api/src/shared/`.

5.1 JobMapStatus (`tblJobSubscriberMapping.JobMapStatusID`)

Source: `shared/status.ts:7–33`.

Id	Constant	Written?
1	MAPPED	yes (apply; Q3 return-to-Q2)
2	SHORTLISTED	yes
3	INTERVIEW_SCHEDULED	yes (legacy `/recruitment/interviews`)
4	SELECTED	yes
5	REJECTED	yes

Id	Constant	Written?
6	INTERVIEW_ATTENDED	yes
7	RESCHEDULE_REQUESTED	yes
8	RESCHEDULED	**never written**
9	INTERVIEW_NOT_ATTENDED	yes
10	ON_HOLD	yes, but only by recruitment `submitResult` (Hold). A Q3 Hold doesn't write it.
11	NEED_MORE_INFO	yes (Q2 send-back)
12	DUPLICATE	**never written**
13	WITHDRAWN	**never written**; there is no application-withdrawal endpoint
14	REFERRED_TO_Q3	yes
15	SENT_TO_COMPANY	yes
16 / 17 / 18	INTERVIEW_R1 / R2 / R3	yes
19	FINAL_ROUND	yes
20	OFFER_SENT	yes
21	OFFER_ACCEPTED	yes
22	JOINED	yes
23	CV_EXPIRED	yes (hourly BullMQ `cv-expiry` job)

"Never written" means a grep of `apps/api/src` finds no writer. "Duplicate" and "Withdrawn" exist only as *registration* decisions (§5.4), and those never touch an application's status. For the full transition table, see COMPLETE_WORKFLOW.md.

5.2 Job status (`tblClientJobs.StatusID`)

Source: `shared/status.ts:41–44`. Values: **1 Active, 2 Closed, 3 Archived, 4 Draft**. This is a flag, not an FK. Code treats any value other than 1 as not active.

5.3 Candidate journey (`SubscriberStatus` / `tblMstrStatus`)

`tblSubscriberStatusHistory.StatusID` uses these ids. `shared/status.ts:46–58` names the ones the code writes:

Id	Constant		Id	Constant
1	ACCOUNT_CREATED		7	INTERVIEW_SCHEDULED
2	CANDIDATE_NOT_INTERESTED		13	SELECTED
3	CV_CREATED		14	REJECTED
4	CV_APPROVED		15	DOCUMENT_MAPPING (15–27 are the legacy document flow)
5	MAPPED_TO_JOB		28	Position Closed (seed only)
6	SHORTLISTED		29	NOT_ATTENDED

5.4 Registration decision (`SubscriberRegistration.flgstatus`)

Written by `POST /recruitment/candidates/:id/decision` (recruitment.service.ts:407-449).

Decision	`flgstatus`	History row
(pending / not yet decided)	0	—
Approved	1	4 CV_APPROVED
Rejected, Withdrawn	2	2 CANDIDATE_NOT_INTERESTED
Duplicate	0	2
OnHold, NeedMoreInfo	0	3 CV_CREATED

5.5 Q1 screening (`tblSubscriberScreening.Status`)

Source: `shared/screening.ts`. Values: `New`, `Screening`, `Incomplete`, `FollowUp`, `NoResponse`, `Verified`, `NotInterested`. Priority is `High`, `Medium` or `Low`; it is derived from profile completeness and status, then stored. The contact log uses `Kind` Contact/ProfileUpdateRequest/CvRequest and `Channel` Call/WhatsApp/Email/Message.

5.6 Q2 match review (`tblApplicationMatchReview.Status`)

Source: `shared/matching.ts:18–27` . Values: `New` , `Forwarded` , `SentBackToQ1` , `Rejected` . Only Q3's return-to-Q2 and CV expiry reset a review to `New` .

5.7 CvReferral (`tblCvReferral`)

Column	Values	Notes
`Status`	`Referred` , `SentToCompany` , `Expired` , `Returned` ; plus `Validated`	**`Validated` is listed in the schema comment (schema:1677) but never written.** A company rejection sets `Returned` .
`CompanyReviewStatus`	`Pending` , `Selected` , `Rejected`	`markCompanyRejected` (cv-referral.service.ts:259) has no caller.
`ReturnReason`	`CV missing` , `Incomplete profile` , `Role mismatch` , `Duplicate referral` , `Auto-expired` (expiry job only)	`shared/hiring.ts` `ReturnReason`

5.8 InterviewRound

Source: `shared/hiring.ts:116–146` .

Column	Values	Notes
`Status`	`Pending` , `Scheduled` , `Completed` , `Cancelled` , `NoShow` , `RescheduleRequested`	**`Cancelled` is never written**; it is only read.
`Result`	`Pending` , `Passed` , `Failed` , `Hold`	Older field. Q3 derives it from `Decision` via `RESULT_FOR_DECISION` .
`Decision`	`NextRound` , `FinalRound` , `Hold` , `Rejected` , `Selected`	Q3 only. Recruitment `submitResult` never sets it.

5.9 Document review (`tblCandidateDocumentUploaded.flgStatus`)

Value	Meaning	Written by
0	Uploaded, awaiting review	candidate upload
1	Verified	recruitment review, Q3 verify, employer review
2	Rejected / sent back	recruitment `documents/review` , Q3 `send-back`
3	Needs correction	**employer** document review only

2 vs 3 inconsistency. Two different values mean "needs correction" depending on who wrote them. `shared/hiring.ts:260–265` `documentState()` treats both 2 and 3 as `NeedsCorrection` . The employer's own list (`employers.service.ts` ~2495) maps 1 → Verified, 2 → Rejected and everything else → Pending, so a correction the employer requests shows up **as "Pending" on the employer's screen**. The three writers also differ in side effects: recruitment writes no `CandidateDocumentStatus` row and no audit, while Q3 and employer reviews write a status row.

5.10 PaymentStatus enum (schema:1310)

`CREATED` , `PENDING` , `SUCCESS` , `FAILED` , `ABORTED` . This is the only DB enum.

5.11 Other string statuses

Table.column	Values
EmployerRegistration.Status	`Pending` → `Approved` / `Rejected`
OfferLetter.Status	`Draft` , `Sent` , `Accepted` , `Rejected` , `Expired`
ClientInvoice.Status	`Unpaid` , `Paid` , `Cancelled`
CandidateReferral.Status	`Invited` , `Signed Up` , `Applied` , `Rewarded`
BlogPost.Status	free string; default `Draft` , `publishedAt` set on the first `Published`
SecUser.Active	`'1'` / `'0'` (char)
SecUser.PwdStatus	0 = must change on first login; 1 = normal

6. Tricky, polymorphic and misnamed columns

6.1 `SecUser.NodeID` / `NodeType` (polymorphic)

- For a **candidate**, `NodeID` = `tblSubscriberRegistration.SubscriberID`.
- For **every other role**, `NodeID` = `tblMstrPerson.PersonNodeID`.
- Ids 1, 2 and 3 exist in both tables, so the orphan check falsely "confirmed" an FK to MstrPerson. The init migration created it and `20260713140000_secuser_nodeid_is_polymorphic` dropped it. **`db/ER-DIAGRAM.md` still draws this FK as confirmed**, so it is stale on this point.
- To find a candidate, code uses the explicit **`SecUser.SubscriberID`** column (unique FK), not NodeID. It was added in `20260713110000`, and `migrate-data.ts` backfills it for role-1 legacy users.
- Email login for non-candidates resolves `MstrPerson.EmailID` (case-insensitive) → `SecUser.NodeID` with `NodeType != 100` (auth.service.ts:189-199).
- **NodeType values the rebuild actually writes:**

NodeType	Written for
100	candidates (auth.service.ts registration path) **and also** staff/admin (`admin/staff-account.ts:40,56`) and all seeded demo users (seed.ts)
200	approved employers (admin.service.ts ~1363)

So 100 no longer means "Subscriber", which breaks email login for staff. See I1.

6.2 Misnamed columns and ids that look like FKs but aren't

Column	What it really holds
`tblSubscriberEducation.CourseTypeID`	`tblMstrCourse.DegreeID` (re-pointed in `20260730170000`)
`tblSubscriberEducation.DegreeID`	`tblMstrEducationType.EducationTypeID` (re-pointed in `20260730180000`). `db/foreign-keys.psv` and `db/ER-DIAGRAM.md` still list the old targets.
`tblClientJobs.StatusID`	Job status flag 1–4. The schema comment at schema:112 still says "FK -> MstrStatus (confirmed)", which is wrong.
`tblJobSubscriberStatus.StatusID`, `tblMstrDocumentStatus.StatusID`	Their own identity PKs, not status ids
`tblClientMstr.UserID`	NULL on **every legacy row**. The real login → company path is `SecUser.NodeID → MstrPerson.PersonNodeID → MstrPerson.ClientID`. Employer approval *does* set it, after the user is created (admin.service.ts ~1378).
`tblPaymentOrder.BDOrderID`	Commented as "BillDesk's order id", but the current Razorpay flow stores the **Razorpay order id** here and looks orders up by it (payments.service.ts:74, 116-118, 203).
`tblSecActiveSessions.SessionID`	The refresh-token `jti`. The legacy session table was repurposed as the server-side refresh session store (auth.service.ts:783-785).
`tblMstrJobMappingStatus` ids 10 and 11	`db/seed` says "Interview passed to candidate / to Client". Migration `20260901120000` inserts them as On Hold / Need More Info. Both inserts use `ON CONFLICT DO NOTHING`, so whichever runs first wins. A **fresh** DB gets the API meaning; a **legacy-restored** DB keeps the legacy labels (schema:1829-1833).
`tblMstrPerson.ClientID`, `tblSubscriberJobStatusLatest.*`	The legacy data used sentinel `0` for "none". `migrate-data.ts` maps 0 to NULL.
`tblSubscriberLanguage.LanguageID`	Dangling: there is no language master table.
`tblMstrCily`	"City" master that is actually districts; misspelled.

6.3 Denormalised copies, CSV and JSON-in-TEXT

Kind	Columns
CSV	SubscriberProfileExtra `DesiredJobType`, `DesiredEmploymentType`, `PreferredWorkModes`, `PreferredJobRoles`, `KeySkills`, `WorkPermitCountries`; `ScreeningContactLog.MissingItems`
JSON stored as TEXT (not `jsonb`)	`InterviewRound.Interviewers`, `CompanyBranding.Benefits`, `SavedSearch.Filters`, `OfferLetter.OfferDetails`, `AuditLog.Detail`

Kind	Columns
Copies that can disagree	`SubscriberTags` vs `ProfileExtra.KeySkills`; `InterviewRound.InterviewerName/Email` (mirrors the first entry of `Interviewers`); `SecUser.RoleID` vs `SecMapUserRoles`; `SubscriberRegistration.flgstatus` vs `SubscriberScreening.Status`; `JobSubscriberMapping.JobMapStatusID` vs the workspace status strings; MstrCourse vs MstrEducationDegree
Split values	`SubscriberCVDetails.TotalExp` (years) + `SubscriberProfileExtra.TotalExpMonths` (the months part)
Sentinel NULL	`SubscriberCVDetails.OnboardedAt` NULL sends the candidate to onboarding. Legacy rows were backfilled to the migration time.

---

7. Data lifecycle

7.1 Where records originate

Origin	Tables written	Transactional?	Evidence
Candidate registration (OTP verify)	SubscriberRegistration, SubscriberCVDetails (`EmailVerified=true`), SubscriberStatusHistory (1), SecUser (`SubscriberID` set, NodeType 100), SecMapUserRoles (role 1); then AuditLog and SecActiveSessions	**yes**, for the core rows	auth.service.ts:409
Employer self-registration	EmployerRegistration (`Pending`) only; no login is created, and nothing is audited	single insert	auth.service.ts:733-764
Employer approval (admin)	Claims EmployerRegistration, then creates MstrPerson, ClientMstr, SecUser (NodeType 200, PwdStatus 0), SecMapUserRoles, and cross-links ClientMstr.UserID and MstrPerson.ClientID	**yes**	admin.service.ts:1314
Staff / admin creation (admin panel, `create-admin.ts`)	MstrPerson, SecUser (NodeType 100), SecMapUserRoles; then AuthPasswordReset and AuditLog	**yes**	admin/staff-account.ts:35
Apply (candidate self-apply or QC "assign job")	JobSubscriberMapping (1), JobSubscriberStatus, SubscriberJobStatusLatest (upsert), SubscriberStatusHistory (5), JobApplicationDetail	**yes**, but the duplicate check runs *before* the transaction	job-application.service.ts:70-78, :77
Any application status change (`transitionStatus`)	JobSubscriberMapping.JobMapStatusID, a JobSubscriberStatus log row, SubscriberJobStatusLatest (only if it still points at this mapping), and optionally SubscriberStatusHistory	**yes** within the call; callers usually write their workspace row (CvReferral, InterviewRound…) and the audit row **outside** it	job-application.service.ts:169-227
Job create / edit / delete (employer)	ClientJobs, ClientJobSkill, …	**yes**	employers.service.ts:682, :1139, :1284
Q3 document verify / send-back / approve, slot updates, reschedule	CandidateDocumentUploaded + CandidateDocumentStatus; ApplicationHiring; InterviewRound + InterviewSlot	**yes**, as batch `$transaction([...])`. The follow-on status transition and Q3Activity rows are separate.	q3-documents.service.ts:197, 229, 293; q3-interviews.service.ts:281, 482; q3-referrals.service.ts:368
Payment settlement	PaymentOrder → SUCCESS, Subscription	**yes**	payments.service.ts:128, 233, 441
Profile section edits, Q1/Q2 actions, CvReferral forward/return, offers, registration decisions	individual tables	**no** (single statements, or sequential statements with no transaction). For example, the Q3 "create and send offer" call is not atomic.	`$transaction` is absent from q1, q2, recruitment, cv-referral and offer-letter services *(verified by grep)*

Concurrency on Q3 hand-offs is handled with **conditional `updateMany` claims** (e.g. `WHERE Status='Referred'`) rather than transactions or locks.

7.2 Status log vs history vs latest

Table	Grain	What writes it	Use
tblJobSubscriberMapping.JobMapStatusID	per application (current)	apply, `transitionStatus`, and the unused-by-UI `PATCH /recruitment/pipeline/:id`	**The source of truth** for an application's stage
tblJobSubscriberStatus	one row per application **transition**	apply and **every** `transitionStatus` call	Per-application audit of status moves; the employer "undo" reads it to find the previous status

Table	Grain	What writes it	Use
tblSubscriberStatusHistory	per **candidate** journey event (MstrStatus ids)	registration (1), apply (5), legacy interview schedule (7), `createRound` (7), `submitResult` Failed (14) / Select (13), registration decisions (4/2/3), legacy no-show (29), employer decisions	Candidate timeline. **No Q3-path transition writes it** (forward, return, expiry, confirm, conducted, decide, offer, joined), so it is incomplete for new-flow applications.
tblSubscriberJobStatusLatest	one row per candidate (PK SubscriberID)	apply (upsert, job-application.service.ts:96); `transitionStatus` updates it only when it still points at the same mapping (:195-203)	Read by the recruitment candidate list for the "job status" column (recruitment.service.ts:52-89). It reflects only the candidate's **most recent** application.

7.3 Audit log (`tblAuditLog`)

`AuditService.record` is called from about 80 sites. Login and logout are also logged to `tblSecUserLogin`.

Audited (action prefixes)	Not audited *(verified against the action list, plus the notes for specific paths)*
`auth.*`, `register.*`, `password.*`	Employer self-registration
`admin.*` (users, employers, settings, blog)	Job **create/edit** (only `job.status` and `job.deleted` are audited)
`job.applied`, `job.status`, `job.deleted`	Candidate profile section edits (education, employment, skills…)
`candidate.*` (uploads, privacy, deletion/export requests, decision)	Recruitment `documents/review`, Q3 document verify/send-back (these write CandidateDocumentStatus, but no audit row)
`cv.*`, `q1.*`, `q2.*`, `q3.*`, `interview.*`, `offer.*`, `applicant.*`	Q1 profile view, which moves New → Screening
`payment.*`, `export.*`, `company.*`, `file.uploaded`	Scheduled jobs (CV expiry writes actor `0`)

Caveats:

- `cv.*` actions record `EntityID = 0` and put the ids in `Detail`, so they can't be found by entity.
- There is no `(Entity, EntityID)` index.
- Audit rows are written **after** and outside the business transaction, so a crash in between loses the audit row.

7.4 Deletes: soft vs hard

Data	Behaviour
Users (admin "delete")	**Soft**: `bulkDeleteUsers` sets `SecUser.Active='0'` and deletes their sessions (admin.service.ts:655-681)
Candidate account deletion	**Soft**: sets `CandidatePrivacy.DeletionRequestedAt`, `Active='0'` and deletes sessions (candidates.service.ts:2145). The code comment promises "a retention job after the grace period", but **no retention or purge job exists** (grep)
Jobs	Close/Archive via `StatusID` (soft). **Hard** delete is allowed only when the job has no applicants, and it removes skills, education, gender maps, saved jobs and the job in one transaction (employers.service.ts:1275-1289)
Profile sub-rows (education, employer, certificate, IT skill, project, accomplishment, language, tags, preferred locations)	**Hard** `deleteMany`, scoped by SubscriberID (candidates.service.ts:753-1119)
Saved jobs, saved searches	Hard
Resume	Hard-deletes the `SubscriberCVUploaded` row and clears `CVPath` (candidates.service.ts:1930)
Interview slots	Hard (replacing unselected slots: interview-round.service.ts:164, q3-referrals.service.ts:369). A round delete cascades to its slots.
Blog posts	Hard (admin.service.ts:1207)
Sessions	Hard (logout, refresh rotation, password reset, deactivation)
Applications, referrals, rounds, offers, audit, payments	**Never deleted** by application code
Stored files (CVs, documents, logos, photos)	**Never deleted from storage.** Both storage drivers implement `delete()` (`modules/storage/drivers/*.ts`), but nothing calls it. Removing a resume or replacing a file leaves the old object in place.

---

8. Migration timeline

There are 34 migrations in `apps/api/prisma/migrations/`. All of them are applied by `prisma migrate deploy` when the API container starts.

#	Migration	Purpose
1	`20260713091218_init_from_restored_backup`	Legacy schema (61 tables) with derived PKs and FKs (27 RESTRICT, 39 SET NULL). Includes four wrong FKs that later migrations drop or re-point.
2	`20260713101118_add_password_reset_and_audit_log`	tblAuthPasswordReset, tblAuditLog
3	`20260713110000_link_secuser_to_subscriber`	`SecUser.SubscriberID` (unique, FK SET NULL)
4	`20260713120000_add_payments`	`PaymentStatus` enum, plans, orders, subscriptions
5	`20260713130000_drop_clientjobs_status_fk`	Drops ClientJobs.StatusID → MstrStatus
6	`20260713140000_secuser_nodeid_is_polymorphic`	Drops SecUser.NodeID → MstrPerson
7	`20260726051427_init`	(misleading name) tblSavedJob with unique (SubscriberID, JobID)
8	`20260726120000_add_admin_extended_models`	PlatformSettings, BlogPost, SavedSearch, NotificationPreference, ApplicantNote, CompanyBranding, plus ~25 indexes on legacy FK columns
9	`20260730120000_add_email_verification`	CVDetails EmailVerified/At, plus an EmailID index
10	`20260730170000_fix_education_coursetype_fk`	CourseTypeID → MstrCourse.DegreeID
11	`20260730180000_education_degree_is_education_type`	**Data-changing**: NULLs invalid DegreeIDs, then re-points the FK to MstrEducationType
12	`20260730190000_add_onboarding_completion`	`OnboardedAt` ; **backfills all existing rows to now**
13	`20260731090000_widen_registration_mobile_no`	Mobile varchar(10) → (15)
14	`20260804090000_add_profile_sections`	Education and certificate fields; ProfileExtra, ITSkill, Project, Accomplishment
15	`20260810200000_employer_job_fields`	Extra ClientJobs fields; EmpType and WorkMode rows
16	`20260810210000_expand_job_masters`	Idempotent designation, skill and city inserts (cities are skipped on a fresh DB)
17	`20260811120000_add_referrals_and_privacy`	tblCandidateReferral, tblCandidatePrivacy
18	`20260813090000_add_preferred_work_modes`	ProfileExtra PreferredWorkModes
19	`20260813100000_add_job_application_detail`	tblJobApplicationDetail (ON DELETE CASCADE)
20	`20260813120000_granular_notification_prefs`	8 per-topic notification booleans
21	`20260815090000_education_years_and_specialization`	Education StartYear, Specialization
22	`20260815100000_add_key_skills_text`	ProfileExtra KeySkills, backfilled from SubscriberTags
23	`20260815110000_job_posting_sections`	ClientJobs KeyResponsibilities, PreferredQualifications
24	`20260815120000_referral_privacy_fk_actions`	Re-creates the referral/privacy FKs to match the schema (fixes drift from #17)
25	`20260816120000_india_education_masters`	Education master overhaul: Category, uniques, tblMstrInstitute, and generated inserts (~3,200 lines, from `prisma/tools/build-education-sql.ts`)
26	`20260825120000_total_experience_months`	ProfileExtra TotalExpMonths
27	`20260826090000_site_document`	tblSiteDocument
28	`20260901120000_figma_flow_gap_models`	EmployerRegistration, InterviewRound, InterviewSlot, CvReferral, CandidateJobScore, OfferLetter; SecRoles 7 'Q3'; JobMappingStatus 10–23 (**no setval**)
29	`20260912000000_q1_screening_workspace`	SubscriberScreening, ScreeningContactLog
30	`20260912100000_q2_matching_workspace`	ApplicationMatchReview, plus a TotalScore index

#	Migration	Purpose
31	`20260912110000_cv_size_and_pages`	CVUploaded SizeBytes, PageCount
32	`20261005120000_interview_round_guest_location`	InterviewRound GuestName/Email, Location
33	`20261005130000_client_invoices`	tblClientInvoice (with an inline FK that the schema doesn't declare)
34	`20261018090000_q3_hiring_ops`	CvReferral return fields; InterviewRound Q3 columns; slot `IsUnavailable`; document FileName/SizeBytes; ApplicationHiring, Q3Activity; document-type and interview-mode masters (with setval)

---

9. Seeding and data migration

9.1 `apps/api/prisma/seed.ts`

Run it with `npm run db:seed --workspace apps/api`. `main()` (seed.ts:418-433) runs these steps in order:

1. **`seedMasterData()`** loads each `db/seed/<table>.psv` (pipe-separated, with a header row) using `INSERT … ON CONFLICT DO NOTHING`, then calls `setval` on each PK sequence. It covers roles, geography, status masters, skills/tags/functions, designation, industry, emp type, work mode, gender, education, document types, job-mapping status and interview mode.
2. **`seedIndiaEducation()`** loads qualifications, courses and institutes from `prisma/data/india-education.ts` and `india-institutes.ts` with `createMany skipDuplicates`.
3. **`ensureJobImportMasters()`** adds 9 metro cities (with hard-coded StateIDs) and the 4 employment types if they are missing.
4. **`seedPlans()`** upserts the 12 `SubscriptionPlan` rows.
5. **Only if `SEED_DEMO_USERS=1`**, two more steps run: `seedDemoUsers()` and `seedDemoJob()`.
    - It creates a demo company, `aajiveka`, if no ClientMstr exists.
    - It creates one login per role:

Username	Role
`anuj`	1 candidate (with registration and CV rows)
`qc1`	2
`qc2`	3
`anuj@aajiveka.com`	4 Client
`admin`	5
`q3`	7

    - It creates one active demo job.
    - The demo passwords are deliberately weak (see seed.ts). **Never set this flag on a shared or production database.** Without the flag, **no logins exist**; use `create-admin.ts`.

9.2 Scripts (`apps/api/scripts/`, run with `npx tsx` from `apps/api`)

Script	What it does
`create-admin.ts --username --name --email [--force]`	Creates the first admin through `createStaffLogin` (MstrPerson + SecUser + role 5, random password hash). It refuses if an active admin already exists unless `--force` is passed. It issues a single-use 24 h reset token and prints a set-password link; it never takes or prints a password. Writes audit `admin.user_created`.
`publish-site-document.ts --file --slug --title`	Uploads a file through StorageModule (`local` or `s3`) and upserts a `tblSiteDocument` row
`seed-demo-applicants.ts`	Dev only. Creates demo applicants for the demo employer, with mapping statuses 1–5 and history rows.
`seed-applicant-profile-resume.ts`	Dev only. Fills in the demo candidate's profile and writes a generated PDF CV into local storage.
`prisma/tools/build-education-sql.ts`	Generates the INSERTs for migration #25

9.3 Legacy data migration (`apps/api/prisma/migrate-data.ts`)

Run it with `npm run db:migrate-legacy`. It needs a restored SQL Server container (see `db/README.md`) and a Postgres database that is already migrated.

Flow:

1. Connect to MSSQL using the `MSSQL_SA_PASSWORD` env var.
2. Read FK edges from `../../db/foreign-keys.psv` (a cwd-relative path).
3. Topologically sort the tables. The `ClientMstr.UserID → SecUser → MstrPerson → ClientMstr` cycle is broken by deferring a column.
4. **TRUNCATE every public table** (except `_prisma_migrations`) with `RESTART IDENTITY CASCADE`.
5. For each table, copy the rows in 250-row chunks:
    - sentinel `0` becomes NULL;
    - an orphan FK becomes NULL if the column is nullable, otherwise the row is skipped and counted;
    - `tblSecUser.Password` (plaintext in the legacy DB) is hashed with argon2id.
6. Run the deferred-column pass, then `setval`.
7. Backfill `SecUser.SubscriberID` from `NodeID` for role-1 users.
8. Print the orphan and skipped-row counts. Known orphans: `SecUserLogin.UserID` 647 / 2,225; `SubscriberStatusHistory.SubscriberID` and `.UserID` 12 / 70 each.

Risks:

Risk	Detail
Destructive	It wipes **all** post-launch data (payments, subscriptions, audit, Q1/Q2/Q3 tables) with no confirmation or environment guard.
Committed credential	A hard-coded fallback SA password is committed in the source (migrate-data.ts:32; the value is deliberately not reproduced here).
Wrong FK targets	`foreign-keys.psv` still lists the old SubscriberEducation targets, so valid `CourseTypeID` / `DegreeID` values may be NULLed as "orphans" *(inferred)*.
Not runnable in production	`mssql` is a devDependency, so the script can't run inside the production image *(inferred from package.json)*.
Environment-dependent labels	JobMappingStatus ids 10 and 11 keep their legacy labels on a restored DB.

---

10. Data-layer issues

Classes:

- **CI**: Confirmed issue
- **PI**: Potential issue (needs runtime verification)
- **MI**: Missing implementation
- **IR**: Improvement recommendation

10.1 Schema/code and schema/migration drift

#	Class	Issue	Evidence
I1	CI	Staff and admin accounts are written with `NodeType 100` (the "Subscriber" discriminator), but non-candidate **email** login filters `nodeType != 100`. QC1, QC2, Q3 and Admin users can log in by **username only**. Employers (200) are unaffected.	admin/staff-account.ts:40, :56; seed.ts:270, :288; auth.service.ts:189-199
I2	CI	`tblClientInvoice` has a DB FK to tblClientMstr that the schema doesn't declare. `prisma migrate dev` would detect drift and generate a DROP.	migration 33; schema:1646-1664
I3	CI	Some indexes and FK actions exist only in migrations: `tblMstrInstitute_SearchText_pattern_idx`, `tblCandidateJobScore_TotalScore_idx`, and MstrInstitute's State/City FKs (NO ACTION vs Prisma's SetNull).	migrations 25 and 30
I4	CI	Payment provider drift. Razorpay is the live flow, but its order id goes into `BDOrderID` (commented as BillDesk), and the `RAZORPAY_*` env vars are forwarded by no compose file.	payments.service.ts:32-89, 74, 116-118, 203; schema:1330
I5	CI	Stale relational docs. `db/ER-DIAGRAM.md` shows SecUser → MstrPerson as confirmed and the old SubscriberEducation targets; `foreign-keys.psv` has the old education edges; the schema:112 comment is wrong; `db/SCHEMA_REPORT.md` says only 5 roles exist.	see §6
I6	CI	Role 6 "Subscription" is defined in code but has no `tblSecRoles` row, so mapping a user to role 6 would violate the FK.	db/seed/tblSecRoles.psv; roles.ts:8

#	Class	Issue	Evidence
I7	CI	The document `flgStatus` codes don't round-trip: employer "needs correction" = 3 and Q3/recruitment sent-back = 2, and the employer list renders 3 as "Pending".	shared/hiring.ts:254-265; employers.service.ts ~2495, ~2542
I8	MI	JobMapStatus 8, 12 and 13, CvReferral `Validated` and InterviewRound `Cancelled` are never written. There is no application-withdrawal path, and `markCompanyRejected` is never called.	shared/status.ts; cv-referral.service.ts:259
I9	MI	Tables with no consumer. `ClientInvoice` has no create path (read only, employers.service.ts:2325). `PlatformSetting` isn't enforced. Unused models: ForgotPassword, SubscriberSkills, SubscriberAwards, MstrEducationDegree, MstrCourseType, MstrDegreeType, MstrDocumentStatus, MstrInviewStatus, MstrGender, MstrCountry, PMstNodeTypes, SecMenu×3.	grep of src
I10	MI	No persisted notification inbox. In-app notifications are an in-memory map, which is lost on restart and not shared across API replicas.	sse-notifications.service.ts:9-26
I11	MI	Candidate deletion promises a retention purge job that doesn't exist. Because every SubscriberRegistration child is `RESTRICT`, a purge would need an ordered delete or an anonymisation step.	candidates.service.ts:2134-2165
I12	MI	Stored files are never deleted (no caller of the storage `delete()`).	modules/storage/drivers/*.ts

10.2 Missing FKs, indexes and constraints

#	Class	Issue	Evidence
I13	PI	No unique `(JobID, SubscriberID)` on tblJobSubscriberMapping. The duplicate check is a `findFirst` before the transaction, so a double submit can create two applications. Add a unique index after de-duplicating legacy data.	job-application.service.ts:70-78
I14	PI	`tblSecActiveSessions` has no index on `SessionID` or `UserID`, and no FK. Every refresh and logout deletes by SessionID, which means a sequential scan. `UserID` is Int, not BigInt.	schema:585-590; auth.service.ts:222, 795
I15	IR	Columns that should be FKs: listed in §3 ("Logical links that have no DB foreign key").	—
I16	IR	Missing indexes on hot columns: ApplicantNote / JobSubscriberStatus `.JobSubscriberMapID`; CandidateDocumentMap/Uploaded `.SubscriberID`, `.JobSubscriberMapID`, `.DocumentMapID`; CandidateDocumentStatus `.DocUploadID`; SubscriberCertificate `.SubscriberID`; MstrPerson `.EmailID` (case-insensitive login lookup) and `.ClientID`; PaymentOrder `.BDOrderID`; AuditLog `(Entity, EntityID)`; SubscriberStatusHistory `.JobID/.ClientID`. Postgres doesn't index FK columns automatically.	schema
I17	IR	Missing uniqueness: `SecUser.UserName`; `SecMapUserRoles(UserID, RoleId)`; `ClientJobSkill(JobID, SkillID)`; `InterviewRound(JobSubscriberMapID, RoundNumber)`; `EmployerRegistration.EmailCompany`; `CvReferral` per mapping (by design 1:N). `SubscriberCVDetails.EmailID` is checked only in code, which is race-prone.	staff-account.ts:30-32; schema:776-779
I18	IR	No CHECK constraints on any string status vocabulary or numeric range (scores 0–100, ProficiencyID 1–3).	§5

10.3 Nullability and types

#	Class	Issue
I19	IR	`JobSubscriberMapping.JobID`, `SubscriberID` and `JobMapStatusID` are nullable with SetNull, so an application with no job or no candidate can exist. This was needed for the legacy load; consider NOT NULL once the data is clean. Other risky nullables: `ClientJobs.StatusID`, `SecUser.UserName/Password/Active`, `SubscriberEmployer.SubscriberID`.
I20	IR	Mixed id types for the same concept: SubscriberID is **Int** in SavedSearch and NotificationPreference; UserID is **Int** in SecActiveSessions, ApplicantNote.UpdatedBy and PlatformSetting.UpdatedBy; BigInt elsewhere.
I21	IR	JSON is stored as TEXT (not `jsonb`), and BlogPost uses unbounded TEXT columns.

10.4 Data consistency

#	Class	Issue	Evidence
I22	PI	JobMappingStatus ids 10 and 11 mean different things on a fresh DB and on a legacy-restored DB, so the labels users see can differ between environments.	schema:1829-1833; shared/screening.ts:4-10
I23	PI	Migration 28 inserts explicit ids (SecRoles 7, JobMappingStatus 10–23) without `setval`. A later serial insert could collide unless the seed's setval loop has run *(inferred)*.	migration 28
I24	PI	Migrations 11 and 12 are irreversible: 11 NULLs invalid education DegreeIDs, and 12 marks every pre-existing candidate as onboarded.	migrations 11, 12
I25	CI	Candidate history is incomplete. No Q3-path transition writes `tblSubscriberStatusHistory`, and `tblSubscriberJobStatusLatest` tracks only the latest application. Use `JobSubscriberMapping` plus `JobSubscriberStatus` as the source of truth.	§7.2

#	Class	Issue	Evidence
I26	IR	Denormalised duplicates can drift apart (see §6.3), notably JobMapStatusID vs the Q2/Q3 workspace status strings, which several duplicate code paths update inconsistently (see COMPLETE_WORKFLOW.md).	—
I27	IR	Audit rows are written outside business transactions, use `EntityID 0` for `cv.*` actions, and don't cover job create/edit or document reviews.	§7.3

10.5 Seed and migration tooling

#	Class	Issue	Evidence
I28	CI	`migrate-data.ts` commits a fallback SQL Server SA password. Remove the default and require `MSSQL_SA_PASSWORD`.	migrate-data.ts:32
I29	PI	`migrate-data.ts` TRUNCATEs every table, including new-only ones, with no guard. Running it against a live DB wipes all post-launch data.	migrate-data.ts:125-127
I30	IR	Demo users have trivially guessable passwords. This is gated by `SEED_DEMO_USERS=1`, but nothing stops the flag being used on a production DB.	seed.ts:186, 284

API Documentation

> Source of truth: the NestJS controllers under `apps/api/src/modules/**`. Every endpoint below was read in code at commit `8de48cf`. Interactive Swagger is served by the running API at **`/api/docs`** (`apps/api/src/main.ts`).
>
> Legend — **Auth**: `Public` = `@Public()` (no token); `Any` = any authenticated role; otherwise the roles from `@Roles(...)`. Role ids: 1 Candidate (Subscriber), 2 QC1, 3 QC2, 4 Client (Employer), 5 Admin, 6 Subscription, 7 Q3.

Contents

1. Cross-cutting behaviour
2. Auth
3. Candidates
4. Files
5. Jobs (public search + apply)
6. Employers / Clients
7. Recruitment (legacy pipeline + shared hiring actions)
8. Q1 screening
9. Q2 matching
10. Q3 hiring operations
11. Admin
12. Payments
13. Exports
14. Notifications
15. Companies, site documents, health
16. Background jobs
17. External integrations
18. Frontend calls with no backend route

---

1. Cross-cutting behaviour

Concern	Implementation	File
Global prefix	`/api`	`apps/api/src/main.ts`
Authentication	`JwtAuthGuard` is a global `APP_GUARD`. Every route needs `Authorization: Bearer <access token>` unless it is `@Public()`. The token must carry `type: 'access'`.	`apps/api/src/common/guards/jwt-auth.guard.ts`
Authorization	`RolesGuard` (global). A method-level `@Roles` **replaces** the class-level one (`getAllAndOverride`). A route with no `@Roles` admits any authenticated role.	`apps/api/src/common/guards/roles.guard.ts`
Rate limit	`ThrottlerGuard` (global), `THROTTLE_LIMIT_PER_MIN` per client IP (default 60). Tighter `@Throttle` overrides on auth, uploads and payments. `trust proxy = 1` so the IP is nginx's forwarded address.	`apps/api/src/app.module.ts`, `main.ts`
Validation	Global `ValidationPipe` with `whitelist`, `forbidNonWhitelisted`, `transform`, `enableImplicitConversion`. Unknown body fields → 400.	`main.ts`
Error shape	`AllExceptionsFilter`: `{statusCode, message, error, timestamp, path, ...extra}` (extra carries e.g. `retryAfterSeconds`, `attemptsRemaining`). Non-HTTP exceptions → 500. **Prisma errors are not mapped** (FK / unique violations surface as 500).	`apps/api/src/common/filters/http-exception.filter.ts`
Security headers	`helmet()`; CORS from `CORS_ORIGIN` (comma list) with credentials.	`main.ts`
BigInt	Serialised as JSON numbers (`BigInt.prototype.toJSON`).	`main.ts`
Request log	`METHOD url status ms` for every route except `/api/health`.	`apps/api/src/common/middleware/request-logger.middleware.ts`
Raw body	`/api/payments/webhook` receives the body as raw text (for signature verification).	`main.ts`
CSRF	`GET /api/auth/csrf-token` issues an `XSRF-TOKEN` cookie and the web client sends `X-CSRF-Token`, but **no server code validates it** (see TESTING_AND_ISSUES).	`apps/api/src/modules/auth/csrf.controller.ts`

Pagination conventions differ per module: most list endpoints take `page` (≥1) and `pageSize` (≤100) and return either `{rows,total}` or `{items,total,page,pageSize,pageCount}`.

---

2. Auth — `/api/auth`

Controller `apps/api/src/modules/auth/auth.controller.ts`, service `auth.service.ts`, OTP `otp.service.ts` (Redis).

Method + path	Auth	Throttle	Request	Response	Errors
POST `/auth/login`	Public	5/min	`{userName, password}` — userName may be the login name, the mobile, or an email	`{user, accessToken, refreshToken}`	401 invalid credentials
POST `/auth/refresh`	Public	global	`{refreshToken}`	new `{user, accessToken, refreshToken}` (rotation; old refresh token is single-use)	401 invalid / revoked / user inactive
POST `/auth/logout`	Public	global	`{refreshToken?}`	204	none (bad tokens silently ignored)
POST `/auth/register`	Public	5/min	`{fullName, email, mobile, password (8–72), countryCode?}`; Indian mobiles must match `^[6-9]\d{9}$`	`{otpRequired, registrationToken, email, expiresInSeconds:600, resendAfterSeconds:60, maxAttempts:5, devCode?}` — `devCode` only outside production	409 email/mobile taken; 429 cooldown; 503 email enqueue failed
POST `/auth/verify-otp`	Public	10/min	`{registrationToken (64 hex), code (6 digits)}`	`{user (isOnboarded:false), accessToken, refreshToken}`	400 wrong / expired / too many attempts; 409 taken meanwhile
POST `/auth/resend-otp`	Public	5/min	`{registrationToken}`	same as register (minus token)	400 expired; 429; 503
POST `/auth/forgot-password`	Public	5/min	`{userName}` (username or mobile, **not email**)	always `{message}`	—
POST `/auth/reset-password`	Public	global	`{token, newPassword (min 8)}`	`{message}`; revokes all sessions	400 invalid / used / expired
POST `/auth/register-employer`	Public	3/min	`{companyName, emailCompany, emailHR?, contactNumberCompany?, contactNumberHR?, location?, aboutCompany?, industryType?, website?, companyLogo?}`	`{id, status:'Pending'}` — no login is created	409 duplicate pending/approved registration
GET `/auth/me`	Any	global	—	`AuthUser {userId, userName, fullName, email, roleId, isOnboarded}`	401
GET `/auth/csrf-token`	Public	global	—	`{csrfToken}` + cookie	—

Tokens: access and refresh tokens share one `jti` and are signed with separate secrets (`JWT_ACCESS_SECRET`, `JWT_REFRESH_SECRET`; TTL `JWT_ACCESS_TTL` default 15m, `JWT_REFRESH_TTL` default 7d). `tblSecActiveSessions` is an allow-list for refresh tokens only; the access-token guard does not consult it.

DB writes: login → `tblSecActiveSessions`, `tblSecUserLogin`, `tblAuditLog`. verify-otp → one `$transaction` creating `tblSubscriberRegistration`, `tblSubscriberCVDetails`, `tblSubscriberStatusHistory` (status 1), `tblSecUser`, `tblSecMapUserRoles`. register-employer → `tblEmployerRegistration` (status `Pending`).

---

3. Candidates — `/api/candidates`

Controller `apps/api/src/modules/candidates/candidates.controller.ts`, class-level `@Roles(Subscriber)`. Service `candidates.service.ts`. Every route resolves the candidate through `tblSecUser.SubscriberID` and returns **404 "This login is not linked to a candidate profile"** when it is null (true for migrated legacy candidates).

All rows are scoped by `subscriberID` (updates use `updateMany` with the owner filter, so they are IDOR-safe; an unknown id is a silent no-op that still returns 200).

Method + path	Purpose	Main tables
GET `/candidates/me`	Profile summary (identity, CTC, notice, skills, education, experience, resume meta)	`tblSubscriberCVDetails`, `tblSubscriberProfileExtra`, education, employer, tags, CV uploaded
POST `/candidates/me/complete-onboarding`	Sets `onboardedAt` once; returns `{isOnboarded:true}`	`tblSubscriberCVDetails`
GET `/candidates/me/cv-masters`	Lookup lists for the editor	master tables
GET `/candidates/me/institutes?q&stateId&limit`	Institute typeahead (parameterised raw SQL)	`tblMstrInstitute`
GET `/candidates/me/cv-edit`	Full editable profile (11 tables)	subscriber tables
PUT `/candidates/me/personal`	Name, email (no re-verification), mobile, DOB, gender, address, city. First save also writes `CV_CREATED` history	CV details, registration, status history
PUT `/candidates/me/professional`	Function, experience, CTC, notice, industry, preferred cities, tags	CV details, profile extra, preferred locations, tags
PUT / DELETE `/candidates/me/education[/:id]`	Education rows (degree/course consistency, year ranges, marks)	`tblSubscriberEducation`
PUT / DELETE `/candidates/me/employment[/:id]`	Employment history	`tblSubscriberEmployer`
PUT / DELETE `/candidates/me/certificates[/:id]`	Certificates	`tblSubscriberCertificate`
PUT `/candidates/me/headline` · `/summary` · `/key-skills` · `/career-profile` · `/personal-details` · `/diversity`	Profile sections	`tblSubscriberProfileExtra` (+ tags for key skills)
PUT / DELETE `/candidates/me/it-skills` · `/projects` · `/accomplishments` · `/languages[/:id]`	Child sections	matching subscriber tables
GET `/candidates/me/applied-jobs`	The candidate's applications with status history and latest interview	`tblJobSubscriberMapping` + status tables
GET `/candidates/me/documents`	Requested / uploaded documents	`tblCandidateDocumentMap`, `tblCandidateDocumentUploaded`
POST `/candidates/me/documents/:documentTypeId`	Upload a requested document (multipart `file`) — 400 if not requested	storage + `tblCandidateDocumentUploaded`
GET / POST `/candidates/me/job-alerts`	Stored job alerts (no update/delete; **no sender**)	`tblSubscriberJobAlert`
GET `/candidates/me/saved-jobs`, `/saved-job-ids`; POST / DELETE `/candidates/me/saved-jobs/:jobId`	Saved jobs	`tblSavedJob`
POST `/candidates/me/change-password`	Verify current, set new (min 8). Does not revoke other sessions	`tblSecUser`
GET `/candidates/me/dashboard`	`{appliedCount, savedCount, interviewCount, profileCompletion}`	counts
GET / PUT `/candidates/me/notification-prefs`	Preferences. **PUT persists only 4 of 12 fields**	`tblNotificationPreference`
GET / POST / DELETE `/candidates/me/saved-searches[/:id]`	Saved searches	`tblSavedSearch`
GET `/candidates/me/recommendations`	Up to 20 active jobs matching skill tag, city or industry	jobs
GET `/candidates/me/activity?cursor`	Merged activity timeline (page 20)	5 sources
GET / POST `/candidates/me/referrals`	Refer-a-friend records (no invite is sent)	`tblCandidateReferral`
GET / PUT `/candidates/me/privacy`	Privacy flags (stored, not enforced elsewhere)	`tblCandidatePrivacy`
POST `/candidates/me/export` (202)	Records an export request (no processor)	`tblCandidatePrivacy`
POST `/candidates/me/delete-account` (202)	Records deletion request, deactivates the login, deletes sessions (transaction)	privacy, `tblSecUser`, sessions

4. Files — `/api/files`

Controller `apps/api/src/modules/files/files.controller.ts`; storage in `apps/api/src/modules/storage/*`.

Method + path	Auth	Purpose
POST `/files/upload/:documentTypeId`	**Any**	Generic upload into a document-type folder; not linked to any record; unused by the web app

Method + path	Auth	Purpose
GET `/files/:folder/:sub/:name`	**Any (no ownership check)**	Download any stored file by key (local driver). See TESTING_AND_ISSUES §1
POST `/files/resume`	Candidate, 10/min	Upload own resume (also records size and PDF page count)
GET `/files/resume`	Candidate	Download own resume
DELETE `/files/resume`	Candidate	Clears DB references (stored object is not deleted)
POST `/files/avatar`	Candidate, 10/min	Upload profile photo
GET `/files/avatar/:subscriberId`	Public, 300/min	Serve a candidate photo (jpg/png only), cached 1h

Upload rules (`storage.service.ts`): ≤10 MB (checked after buffering), MIME in pdf/jpeg/png/doc/docx and extension must match; key = `<RootFolder>/<FolderName>/<userId>-<timestamp>-<random>.<ext>` . Driver: `STORAGE_DRIVER=local` (disk, served by the API) or `s3` (private bucket, 300 s presigned URLs).

---

5. Jobs — `/api/jobs`

Controller `apps/api/src/modules/jobs/jobs.controller.ts` ; services `jobs.service.ts` , `job-application.service.ts` .

Method + path	Auth	Purpose / notes
GET `/jobs/filters`	Public	Filter option lists (5-min in-process cache)
GET `/jobs`	Public	Structured search over **Active** jobs: designation, industry, location, workMode, employmentType, min/max exp & CTC, comma lists (`workModes` , `employmentTypes` , `locations` , `states` , `skills`), `postedWithin` (24h/7d/30d), `sortBy` , `page` , `pageSize` (default 10). Returns `{rows, total}`
GET `/jobs/search`	Public	Postgres full-text search (`to_tsvector` / `to_tsquery` , ts_rank) over Active jobs; empty `q` falls back to `/jobs`
GET `/jobs/suggestions?q`	Public	Up to 5 typeahead suggestions (designation / company / skill)
GET `/jobs/recommended`	Candidate	Delegates to candidate recommendations
GET `/jobs/:id`	Public	Job detail. **Not filtered by status** (Draft/Closed/Archived are readable)
POST `/jobs/:id/apply`	Candidate	Apply. Requires an **active paid subscription** (403 otherwise), job Active (404), not already applied (400). Body: optional application-form snapshot (name, email, phone, experience, location, expected salary, notice, LinkedIn, portfolio, resume file name, cover letter). Returns `{jobSubscriberMapId, reference:'AJ-000123'}`

Apply transaction (`JobApplicationsService.apply`): creates `tblJobSubscriberMapping` (status 1 MAPPED), a `tblJobSubscriberStatus` log row, upserts `tblSubscriberJobStatusLatest` , writes `tblSubscriberStatusHistory` (5 MAPPED_TO_JOB) and optionally `tblJobApplicationDetail` ; then audit `job.applied` . No email is sent.

---

6. Employers — `/api/clients`

Controller `apps/api/src/modules/employers/employers.controller.ts` , class-level `@Roles(Client, Admin)` ; service `employers.service.ts` . (`apps/api/src/modules/clients/*` only re-exports these and is not imported by `AppModule` .)

Ownership: every `me/*` call resolves the company via `tblSecUser.NodeID → tblMstrPerson.ClientID` (`clientIdFor`) and returns 404 when missing. Jobs are checked with `ownedJob` ; applications by `mapping.job.clientID` . Admin is listed in `@Roles` but has no linked company, so **Admin receives 404 on every `me/*` route**.

Method + path	Purpose	Notes
GET / PATCH `/clients/me`	Company profile (+ HR contact)	PATCH accepts any string as `companyLogo` (see issues)
POST `/clients/me/logo`	Upload logo (JPEG/PNG)	
GET `/clients/masters`	Lookups for job forms	
GET `/clients/me/jobs`	List own jobs with counts by status and filters `q, city, status, page, pageSize`	Draft/Archived hidden unless filtered

Method + path	Purpose	Notes
POST `/clients/me/jobs?draft=`	Create job (Active, or Draft with `draft=true`)	`$transaction` (job + skills); no audit; no min≤max checks
GET / PATCH / DELETE `/clients/me/jobs/:id`	Read / edit / delete (delete only if no applications)	
POST `/clients/me/jobs/:id/status`	`{status: Active	Closed
POST `/clients/me/jobs/:id/deactivate` · `/archive` · `/activate`	Shortcuts to the above	
POST `/clients/me/jobs/:jobId/duplicate`	Copy as a new **Active** job	Drops two text fields
POST `/clients/me/jobs/bulk-upload?commit=`	CSV import: validate preview, then commit	Per-row transactions
GET `/clients/me/applicants`	All applications on own jobs (filters status, q, jobId, city, minExp, maxNotice)	In-memory pagination; includes pre-screening applicants with contact details
GET `/clients/me/applicants/:id`	Full applicant profile and timeline	
GET `/clients/me/applicants/:id/resume?inline=`	Applicant CV	
POST `/clients/me/applicants/:id/decision`	`{decision: Shortlisted	Hired
POST `/clients/me/applicants/:id/decision/undo`	Revert Shortlisted/Rejected	
GET / POST `/clients/me/applicants/:id/interview-rounds`	List / request first round (exactly 3 distinct future slots, interviewer + HR details, mode)	Q3 then confirms a slot
POST `/clients/me/applicants/:id/interview-rounds/:roundId/result`	`{decision: Select	Reject
GET / PUT `/clients/me/applicants/:id/notes`	Private notes (PUT appends)	
GET `/clients/me/applicants/:id/documents`	Candidate documents	Scoped to the candidate, not the application
POST `/clients/me/applicants/:id/documents/review`	`{docUploadId, status: Approved	NeedsCorrection, comments?}`
GET `/clients/me/document-types`	Requestable document types	
POST `/clients/me/applicants/:id/document-request`	Request documents (only when status = SELECTED)	
GET `/clients/me/analytics` · `/analytics/export`	Funnel / job performance, CSV export	
GET `/clients/me/billing` · `/billing/export`	Invoices (read-only), CSV export	
GET / PATCH `/clients/me/branding`	Branding (unused by web)	
POST `/clients/me/interviews/generate-link`	Returns a placeholder `meet.aajiveka.com` URL	Not a real meeting
GET `/clients/:id/logo`	**Public** company logo	Streams the stored key
GET `/clients/:id/public`	**Public** company card	

---

7. Recruitment — `/api/recruitment`

Controller `apps/api/src/modules/recruitment/recruitment.controller.ts`, class-level `@Roles(QC1, QC2, Admin)`; method-level `@Roles` override it. Services: `recruitment.service.ts`, `cv-referral.service.ts`, `interview-round.service.ts`, `offer-letter.service.ts`, `scoring.service.ts`, `mapping-access.service.ts` (ownership checks for Candidate and Client only).

Method + path	Roles	Purpose
GET `/recruitment/candidates?search&status&page&pageSize`	QC1, QC2, Admin	Candidate list
GET `/recruitment/candidates/:id`	QC1, QC2, Admin	Candidate detail + registration status + latest score
GET `/recruitment/candidates/:id/resume`	QC1, QC2, Admin	Candidate CV
POST `/recruitment/candidates/:id/decision`	QC1, Admin	Registration decision: Approved / Rejected / OnHold / NeedMoreInfo / Duplicate / Withdrawn (does **not** change any application status)
POST `/recruitment/candidates/:id/assign-job`	QC1, QC2, Admin	Staff-created application (same as apply, without the subscription gate)

Method + path	Roles	Purpose
POST `/recruitment/candidates/:id/documents`	QC2, Admin	Assign document types to collect
GET `/recruitment/qc1/stats`	QC1, Admin	Legacy QC1 counters
GET `/recruitment/jobs`	QC1, QC2, Admin	Active jobs
GET `/recruitment/document-types`	QC1, QC2, Admin	Candidate-uploadable document types
GET `/recruitment/interviews` , `/interviews/eligible`	QC1, QC2, Admin	Legacy interview list (tblJobInterviewStatus)
POST `/recruitment/interviews`	QC1, QC2, Admin	Legacy schedule → status 3, email
POST `/recruitment/interviews/:id/status`	QC1, QC2, Admin	Completed → 6, Cancelled → 9
GET `/recruitment/interview-modes`	QC1, QC2, Q3, Client, Admin	Lookup
GET `/recruitment/documents` · `/documents/:documentId/file`	QC1, QC2, Admin	All candidate documents (unscoped) / download
POST `/recruitment/documents/review`	QC1, QC2, Admin	Verified / Rejected
PATCH `/recruitment/pipeline/:id`	QC1, QC2, Admin	Sets any status with no log/history/audit. **Unused by the web app**
POST `/recruitment/applications/:id/score`	QC1, Admin	Compute and store the 7-factor match score
POST `/recruitment/referrals`	QC2, Admin	Refer applications to Q3 (→ 14)
GET `/recruitment/referrals?status`	QC2, Q3, Admin	All referrals
POST `/recruitment/referrals/forward`	Q3, Admin	Forward to company (→ 15, 14-day window)
POST `/recruitment/referrals/:id/company-review`	Client, Admin	Company shortlist (creates round 1 with 3–4 slots) or reject
POST `/recruitment/interview-rounds`	Q3, Client, Admin	Create a round (any roundNumber)
GET `/recruitment/interview-rounds/:mapId`	QC2, Q3, Client, Candidate, Admin	Rounds + slots (Client/Candidate ownership-checked)
POST `/recruitment/interview-rounds/:roundId/slots`	Q3, Client, Admin	Replace unselected slots (resets round to Pending)
POST `/recruitment/interview-rounds/:roundId/select-slot`	Candidate, Q3, Admin	Pick a slot → round Scheduled, emails
POST `/recruitment/interview-rounds/:roundId/result`	Q3, Client, Admin	Passed / Failed / Hold (+ next round or Select)
POST `/recruitment/offers`	Q3, Client, Admin	Create offer (needs SELECTED and all docs verified)
POST `/recruitment/offers/:id/send`	Q3, Client, Admin	Send offer → 20, email
POST `/recruitment/offers/:id/respond`	Candidate, Admin	Accept (→ 21) / reject
POST `/recruitment/offers/:id/mark-joined`	Q3, Client, Admin	→ 22 JOINED
GET `/recruitment/offers/:mapId`	QC2, Q3, Client, Candidate, Admin	Offer for an application

---

8. Q1 screening — `/api/q1`

Controller `apps/api/src/modules/q1/q1.controller.ts` , `@Roles(QC1, Admin)` . Services `q1.service.ts` , `q1-analytics.service.ts` . Screening state lives in `tblSubscriberScreening` (New, Screening, Incomplete, FollowUp, NoResponse, Verified, NotInterested).

Method + path	Purpose
GET `/q1/stats` , `/q1/nav-counts`	Dashboard counters
GET `/q1/candidates?tab&search&priority&experience&cvOnly&page&pageSize`	Screening queue (tabs: all, new, screening, incomplete, follow-up, no-response, verified, not-interested)
GET `/q1/candidates/:id`	Candidate profile + completeness + screening row (opening moves New → Screening)
GET `/q1/candidates/:id/contact-log`	Last 50 contact entries
PATCH `/q1/candidates/:id/checklist` · `/checklist/all`	Toggle 7 checklist items

Method + path	Purpose
POST `/q1/candidates/:id/verify`	Mark Verified (no precondition)
POST `/q1/candidates/:id/contact` · `/request-update` · `/request-cv`	Log a contact attempt (Call/WhatsApp/Email/Message) — **nothing is actually sent**
POST `/q1/candidates/:id/status`	FollowUp (date/time) / NoResponse (attempts) / NotInterested (reason)
GET `/q1/analytics`	Screening analytics

---

9. Q2 matching — `/api/q2`

Controller `apps/api/src/modules/q2/q2.controller.ts`, `@Roles(QC2, Admin)`. Services `q2.service.ts`, `q2-analytics.service.ts`. Review state in `tblApplicationMatchReview` (New, Forwarded, SentBackToQ1, Rejected).

Method + path	Purpose
GET `/q2/stats`, `/q2/nav-counts`, `/q2/sla`	Counters (the "SLA" is a backlog count, not time-bound)
GET `/q2/jobs?search`	Active jobs with applicant/relevant counts (lazily scores up to 25 applications per call)
GET `/q2/jobs/:jobId?relevantOnly`	Ranked applicants for a job
GET `/q2/applicants?tab&bucket&search&page&pageSize`	All applicants (tabs all/relevant/notRelevant; buckets queue/forwarded/sentBack/rejected)
GET `/q2/applications/:mapId`	Application detail with 7-criterion score breakdown and skill coverage (creates review row, stamps first view)
PATCH `/q2/applications/:mapId/criteria`	Include/exclude a criterion from the displayed total
POST `/q2/applications/:mapId/forward`	Refer to Q3 (→ 14)
POST `/q2/applications/:mapId/send-back`	Send back to Q1 (→ 11 NEED_MORE_INFO, screening → Incomplete)
POST `/q2/applications/:mapId/reject`	Reject (→ 5)
GET `/q2/analytics`	Matching analytics

Score weights (`apps/api/src/shared/matching.ts`, duplicated in `scoring.service.ts`): skills 30%, experience 25%, job role 20%, education 10%, location / salary / notice 5% each. Relevant = score ≥ 60.

---

10. Q3 hiring operations — `/api/q3`

Controller `apps/api/src/modules/q3/q3.controller.ts`, `@Roles(Q3, Admin)`. Services `q3-read.service.ts`, `q3-referrals.service.ts`, `q3-interviews.service.ts`, `q3-documents.service.ts`, `q3-mail.ts`. Only applications with at least one `tblCvReferral` row are visible.

Method + path	Purpose
GET `/q3/nav-counts`, `/q3/dashboard`, `/q3/pipeline?stage&search&company`, `/q3/report?quarter`, `/q3/queues/:name`	Read models (stage derived by `stageOf` in `shared/hiring.ts`)
GET `/q3/referrals`	Referred CVs grouped by job batch
POST `/q3/referrals/forward`	Forward referrals to the company (all must have a CV)
POST `/q3/referrals/return`	Return to Q2 with reason (+ optional email to Q2)
GET `/q3/company-review?tab`	14-day company review board
POST `/q3/company-review/remind-expiring`	Email companies with CVs about to expire
GET `/q3/applications/:mapId/shortlist`	Shortlist summary
PUT `/q3/applications/:mapId/round-setup`	Capture company shortlist + round panel + slots (→ 2 SHORTLISTED) or edit a pending round
GET `/q3/applications/:mapId/scheduling`	Scheduling workspace
POST `/q3/applications/:mapId/contact`	Log a candidate contact
POST `/q3/rounds/:roundId/slots/:slotId/availability?unavailable`	Mark a slot (un)available
POST `/q3/rounds/:roundId/request-slots`	Email the company for slots
POST `/q3/meeting-link`	Generate a Jitsi link
POST `/q3/rounds/:roundId/confirm`	Confirm a slot → round Scheduled, status R1..R3/Final, invitation emails

Method + path	Purpose
GET `/q3/interviews?week`	Weekly interview board
POST `/q3/rounds/:roundId/remind` · `/checklist` · `/reschedule` · `/conducted`	Interview-day actions
GET `/q3/applications/:mapId/rounds`	Rounds and decisions
POST `/q3/applications/:mapId/nudge`	Chase the company for a result
POST `/q3/rounds/:roundId/decision`	NextRound / FinalRound / Hold / Rejected / Selected (Selected only on final round; creates document requests)
POST `/q3/rounds/:roundId/decision-mail`	Send the decision email
GET `/q3/applications/:mapId/documents`	Document checklist + offer
POST `/q3/documents/:docUploadId/verify` · `/send-back`	Verify or send back a document
GET `/q3/documents/:docUploadId/file?preview`	View/download a document
POST `/q3/applications/:mapId/documents/request` · `/forward` · `/approve`	Request docs (due in 3 days) → forward to company → approve
POST `/q3/applications/:mapId/offer`	Generate and send offer (needs docs approved) → 20
POST `/q3/applications/:mapId/joined`	Mark joined (offer must be Accepted) → 22
GET `/q3/returned?tab`, `/q3/returned/:referralId/mail`; POST `/q3/returned/:referralId/resend`	Returned-to-Q2 tracker

11. Admin — `/api/admin`

Controller `apps/api/src/modules/admin/admin.controller.ts`, `@Roles(Admin)`. Service `admin.service.ts`, `staff-account.ts`. The admin panel is **read-only over the hiring pipeline** by design; pipeline actions stay in the Q1/Q2/Q3/employer workspaces (which also admit Admin).

Method + path	Mutates	Purpose
GET `/admin/stats`	—	Platform KPIs, signups (30 d), top designations/cities, recent audit
GET `/admin/users?page&pageSize&q&roleId&isActive`	—	User search
POST `/admin/users`	yes	Create staff login (QC1, QC2, Q3, Admin) + emailed set-password link
GET `/admin/users/:userId`	—	User detail (logins, sessions, audit, applications, company)
POST `/admin/users/:userId/password-reset`	yes	Email a 1-hour reset link
PATCH `/admin/users/:userId`	yes	Active flag and/or role (never self; never across candidate/employer/staff lines); revokes refresh sessions
POST `/admin/users/bulk-update` · `/bulk-delete`	yes	Bulk active/role; bulk soft-delete (bulk-delete unused by web)
GET `/admin/jobs`, `/admin/jobs/:jobId`	—	Jobs and job detail
POST `/admin/jobs/:jobId/status` · `/admin/jobs/bulk-moderate`	yes	`close` / `reopen` (Active ↔ Closed only)
GET `/admin/applications`, `/admin/applications/:mapId`	—	Applications by status/stage with timeline and next-step deep link
GET `/admin/companies`	—	Companies with job/application counts
GET `/admin/audit-logs?action&entity&entityId&userId`	—	Audit log browser
GET `/admin/employer-registrations?status`	—	Registration queue
POST `/admin/employer-registrations/:id/review`	yes	Approve (creates company + login + emails a temporary password) or reject
GET / PATCH `/admin/settings`	yes	18 platform settings — **stored but read by no other code**
GET / POST `/admin/blog-posts`; PATCH / DELETE `/admin/blog-posts/:id`	yes	Blog CMS

12. Payments — `/api/payments`

Controller `apps/api/src/modules/payments/payments.controller.ts`; services `payments.service.ts`, `razorpay.service.ts`, `billdesk.service.ts`.

Method + path	Auth	Purpose
GET `/payments/plans`	Public	Active subscription plans (12 seeded: 3 tiers × 1/3/6/12 months)

Method + path	Auth	Purpose
POST `/payments/create-order`	Candidate, 10/min	Create a Razorpay order `{planId}` → `{orderRef, razorpayOrderId, amount, currency, keyId, planName}`
POST `/payments/verify`	Candidate, 10/min	Verify Razorpay checkout signature → order SUCCESS + `tblSubscription` (transaction, idempotent)
POST `/payments/webhook`	Public (HMAC)	Razorpay webhook (`x-razorpay-signature`); handles `payment.captured` / `payment.failed`
GET `/payments/history`	Candidate	Own orders
POST `/payments/orders`	Candidate, 10/min	Create a **BillDesk** order (legacy path) → `{orderRef, amountInr, redirectUrl}`
GET `/payments/orders/:orderRef`	Candidate	Order status (used by `/payment/return`)
GET `/payments/subscription`	Candidate	Current subscription

Effect of payment: only a `tblSubscription` row (start = end of current subscription or now; stacks). It gates **candidate self-apply** only; the user's role is not changed. **`PaymentsService.settle()` (BillDesk settlement) has no route**, so BillDesk orders can never complete. No receipt email is sent.

---

13. Exports — `/api/exports`

Controller `apps/api/src/modules/exports/exports.controller.ts` . Query `format=csv|xlsx|pdf` (default csv). Every export writes an `export.<name>` audit row.

Method + path	Roles	Data	Known defects
GET `/exports/candidates?search&status`	QC1, QC2, Admin	Up to 5,000 candidates	not used by web
GET `/exports/applicants`	Client, Admin	Employer applicants	**only 10 rows** (default page size)
GET `/exports/users?roleId&isActive&from&to`	Admin	Users	date filter = users with a login in range
GET `/exports/jobs?status&company&from&to`	Admin	Jobs	`status` , `company` ignored
GET `/exports/applications?jobId&from&to`	Admin	Applications with contact details	
GET `/exports/payments?from&to`	Admin	Payment orders	

---

14. Notifications — `/api/notifications`

Controller `apps/api/src/modules/notifications/notifications.controller.ts` (any authenticated role).

Method + path	Purpose
GET `/notifications/stream`	Server-Sent Events stream (`connected` event + 30 s heartbeat). Web client uses `fetch` streaming with the bearer header
GET `/notifications?page&pageSize&type&unread`	In-app notifications
PATCH `/notifications/:id/read` , `/notifications/read-all`	Mark read
DELETE `/notifications/:id`	Delete

The store is **in-memory and nothing in the codebase creates notifications**, so these endpoints always return empty data.

---

15. Companies, site documents, health

Method + path	Auth	Purpose	File
GET `/companies/search?q`	Public	Company typeahead (20)	`modules/companies/companies.controller.ts`
GET `/companies/:id`	Public	Company page incl. contact details and active job count	same
GET `/site-docs`	Public	Published downloadable documents	`modules/site-documents/site-documents.controller.ts`

Method + path	Auth	Purpose	File
GET `/site-docs/:slug`, `/site-docs/:slug/download`	Public	Serve a document (inline for pdf/images; S3 → 302 presigned)	same
GET `/health`	Public, unthrottled	`{status ok	degraded, env, db, redis, disk, timestamp}` — always HTTP 200

Site documents are written only by the CLI `apps/api/scripts/publish-site-document.ts`.

---

16. Background jobs (BullMQ)

Queue	Job	Schedule	Processor	Effect
`notifications`	`email`, `sms`	on demand (5 attempts, exponential backoff from 2 s)	`apps/api/src/modules/notifications/notifications.worker.ts` (in-process)	Delivers through the configured provider
`cv-expiry`	`expire-stale-cvs`	every hour (scheduler upserted in `RecruitmentModule.onModuleInit`)	`apps/api/src/modules/recruitment/cv-expiry.processor.ts` → `CvReferralService.expireStale`	Expires CVs not reviewed by the company within 14 days (→ 23 CV_EXPIRED), puts them back in Q2's queue, emails Q2

There are no other cron jobs, webhooks consumers or retention processors.

---

17. External integrations

Integration	Env vars (names only)	Status
PostgreSQL 16 (Prisma 7, `@prisma/adapter-pg`)	`DATABASE_URL`	Required
Redis (OTP, BullMQ)	`REDIS_URL`	Required
Email: SMTP (nodemailer) / AWS SES / log	`EMAIL_DRIVER`, `SMTP_HOST`, `SMTP_PORT`, `SMTP_USER`, `SMTP_PASSWORD`, `SMTP_SECURE`, `SMTP_FROM`, `SMTP_FROM_NAME`, `SES_FROM_EMAIL`, `AWS_REGION`	Default `log` (emails are logged, not sent)
SMS: 2Factor / MSG91 / AWS SNS / log	`SMS_DRIVER`, `TWOFACTOR_API_KEY`, `MSG91_AUTH_KEY`, `MSG91_TEMPLATE_ID`, `SNS_SENDER_ID`, `SNS_DLT_ENTITY_ID`, `SNS_DLT_TEMPLATE_ID`	Providers implemented, **but no code sends SMS**
Storage: local disk / AWS S3	`STORAGE_DRIVER`, `STORAGE_LOCAL_ROOT`, `S3_BUCKET`, `AWS_REGION`	Implemented
Razorpay	`RAZORPAY_KEY_ID`, `RAZORPAY_KEY_SECRET`, `RAZORPAY_WEBHOOK_SECRET`	Wired to the UI
BillDesk	`BILLDESK_BASE_URL`, `BILLDESK_CLIENT_ID`, `BILLDESK_MERCHANT_ID`, `BILLDESK_SECRET_KEY`	Order creation only; settlement unreachable; never run against the real gateway
Jitsi	—	Q3 generates `meet.jit.si` links (no API)
Google Analytics / web vitals	web-side (`apps/web/src/lib/ga4.ts`, `webVitals.ts`)	Client-side only

---

18. Frontend calls with no backend route

Verified by grepping `apps/web/src` against the controllers:

Frontend call	Caller	Result
GET `/candidates/:id/profile`	`apps/web/src/features/candidates/pages/PublicProfilePage.tsx`	404
GET / POST `/companies/:id/reviews`	`apps/web/src/features/public/pages/CompanyReviewsPage.tsx`	404
GET / POST `/clients/:id/reviews`	`apps/web/src/features/public/components/CompanyReviews.tsx` (rendered on `/companies/:id`)	404
GET `/jobs/salary-insights`	`apps/web/src/features/public/pages/SalaryInsightsPage.tsx`	400 (captured by `/jobs/:id` → ParseIntPipe)
POST `/clients/me/campaigns/bulk`	`apps/web/src/features/clients/components/BulkEmailUpload.tsx` (orphan component, not routed)	404

See FRONTEND_BACKEND_MAPPING.md for the page-by-page mapping.

Frontend ↔ Backend Mapping

> **Commit:** `8de48cf` · **Scope:** `apps/web/src` (React 18 + Vite + React Router + TanStack Query) mapped onto `apps/api/src/modules/**` (NestJS).
>
> **How to read this doc.** Web paths are relative to `apps/web/src/` unless they start with `apps/`. API paths omit the global `/api` prefix. "Backend" cells name the controller file and the service class it delegates to; full request/response contracts live in API_DOCUMENTATION.md, roles in ROLE_PERMISSION_MATRIX.md, the business flow in COMPLETE_WORKFLOW.md, and test coverage in TESTING_AND_ISSUES.md. Folder layout is in PROJECT_STRUCTURE.md; see also PROJECT_OVERVIEW.md, MODULE_DEPENDENCIES.md, DATABASE_SCHEMA.md, MASTER_SYSTEM_FLOW.md.
>
> **Evidence.** Everything here was read in code (static analysis). Nothing was executed against a running system. Statements that were reasoned rather than read are tagged *(inferred)*. Behaviour that can only be confirmed by running the app is classed **Potential issue (needs runtime verification)**.

Contents

1. How the web talks to the API
2. Layout shells
3. Route tables
4. Key forms and actions
5. Dead or broken UI
6. Duplicate screens: legacy `/recruitment/*` vs `/q1` `/q2` `/q3`
7. Mobile responsiveness (static evidence only)

---

1. How the web talks to the API

1.1 Request path

```
Browser ──> api (axios, lib/axios.ts, baseURL = env.apiBaseUrl = '/api')
   dev:     Vite dev/preview server proxies /api → VITE_API_PROXY ?? http://127.0.0.1:4000   (apps/web/vite.config.ts:168-181)
   deploy:  nginx `location /api/ { proxy_pass http://api:4000; }`, SPA fallback try_files → /index.html (deploy/nginx.conf:11-12, 42-43)
   ──> NestJS: global JwtAuthGuard (Bearer header only) → RolesGuard (@Roles, method overrides class) → ThrottlerGuard → ValidationPipe
```

1.2 Client plumbing

Piece	File	Behaviour (verified)
Env	`lib/env.ts`	`apiBaseUrl = VITE_API_BASE_URL ?? '/api'` ; `useMocks = VITE_USE_MOCKS === '1'` ; `oauthEnabled = VITE_OAUTH_ENABLED === '1'`
Axios instance	`lib/axios.ts` (`api`)	`timeout 30 s` , JSON. **Request:** adds `Authorization: Bearer <access>` ; on non-GET adds `X-CSRF-Token` from `<meta name=csrf-token>` or cookie `XSRF-TOKEN` (the API issues but never validates it — see API_DOCUMENTATION.md §1). **Response:** 429 → wait `Retry-After` (default 1 s), retry once; 401 → one shared `refreshAccessToken()` (bare `axios.post('/api/auth/refresh', {refreshToken})`), store new tokens, replay; refresh failure → `tokenStorage.clear()` + `window.location.assign('/login')` ; ≥500 → `captureError` . `getErrorMessage(err, fallback)` extracts the API `message` . No `paramsSerializer` (arrays go out as `key[]=a`).
Token storage	`lib/tokenStorage.ts`	Access token in a module variable (memory only, lost on reload). Refresh token in `localStorage['aaj.refresh']` .
Auth store	`features/auth/auth.store.ts` (Zustand)	`{user, isAuthenticated, isLoading}` . `setSession` stores both tokens; `updateUser(patch)` ; `logout()` → `POST /auth/logout {refreshToken}` (errors swallowed) → clear tokens. `bootstrap()` (called once from `app/providers.tsx`): if a refresh token exists → `GET /auth/me` (401 → interceptor refresh → replay). `AuthUser = {userId, userName, fullName, email, roleId, isOnboarded?}` . **`logout()` does not clear the Query cache, offline queue or `aaj.lastOrderRef` .**
Query client	`lib/queryClient.ts`	`staleTime 60 s` , `retry 1` , `refetchOnWindowFocus false` . Central `queryKeys` + `invalidate*` helpers exist, but most feature hooks use inline keys (e.g. `['candidate','saved-jobs']`). `queryClient.clear()` is never called.

Piece	File	Behaviour (verified)
Providers	`main.tsx`, `app/providers.tsx`	`ErrorBoundary > AppProviders (QueryClientProvider + ToastProvider + ReactQueryDevtools) > App (RouterProvider + OfflineBanner + PWA prompts)`; `useOfflineSync()` at root (queue in `lib/offlineQueue.ts`, localStorage). `purgeLegacyApiCache()` deletes the old `api-cache` Cache Storage.
Route guard	`routes/ProtectedRoute.tsx`	`isLoading` → `FullPageLoader`; unauthenticated → `/login` with `state.from`; role not in `allow` → `ROLE_HOME[roleId]`; Subscriber with `!isOnboarded` on any path other than `/candidate/onboarding` → onboarding.
Role homes	`types/roles.ts:29-37`	Subscriber `/candidate/profile`, QC1 `/q1/dashboard`, QC2 `/q2/dashboard`, Client `/company`, Admin `/admin`, Subscription `/pricing`, Q3 `/q3/dashboard`.
Login doors	`features/auth/loginPortals.ts`	`/login?as=candidate
SSE	`features/notifications/notifications.api.ts` (`useNotificationStream`)	`fetch` streaming of `GET /notifications/stream` with the bearer header. The API store is in-memory and nothing creates notifications (API_DOCUMENTATION.md §14).
MSW mocks	`main.tsx:49-56`, `mocks/handlers.ts`, `mocks/data.ts`	Started **only** when `VITE_USE_MOCKS=1` (dynamic import, `onUnhandledRequest: 'bypass'`, failure swallowed). Off by default; the CI `e2e-mock` job sets it. Unmocked calls (saved jobs, referrals, payments, employer analytics/billing/rounds, notifications …) fall through to the real `/api`.
Analytics	`lib/analytics.ts`, `components/AnalyticsProvider`	DEV: console only. Prod: `sendBeacon(VITE_ANALYTICS_ENDPOINT)` only if set. Every route is a child of `<AnalyticsProvider/>` (`routes/router.tsx:161`).

1.3 Files the browser downloads

The API authenticates **only** through the `Authorization` header (`apps/api/src/common/guards/jwt-auth.guard.ts:34-35`). A plain `<a href="/api/...">` or `window.open(...)` therefore sends no credentials and gets 401. The working pattern is "GET as blob through `api`, then object URL" (e.g. `features/recruitment/recruitment.api.ts:62-76`, `features/q3/q3.api.ts:265-271`, employer `useApplicantResumeBlob`, admin `features/admin/components/ExportButton.tsx`). Exceptions are listed in §5.

---

2. Layout shells

Shell	File	Used by (routes)	Notes
none (full bleed)	—	`/brochure`	Static brochure rebuild
`PublicLayout`	`components/layout/PublicLayout.tsx`	`/`, `/login`, `/register`, `/register/employer`, `/forgot-password`, `/reset-password`, `/search`, `/candidates/:id`, `/companies/:id`, `/companies/:id/reviews`, `/pricing`, `/about`, `/contact`, `/help`, `/blogs`, `/blogs/:slug`, `/career`, `/testimonial`, `/privacy`, `/terms`, `/subscription`, `/resume`, `/student`, `/salary-insights`, `/payment/return`	Public `Navbar` (no signed-in state) + footer; mounts `CommandPalette`
`JobsLayout`	`features/jobs/components/JobsLayout.tsx`	`/jobs`, `/jobs/:id` (public); `/candidate/applications`, `/candidate/saved-jobs`, `/jobs/:id/apply` (Subscriber)	Renders `PublicLayout` for anonymous / non-candidate users and `CandidateChrome` (portal `PortalHeader`, via `usePortalProfile`) for Subscribers
`CandidatePortalLayout`	`features/candidates/portal/CandidatePortalLayout.tsx`	`/candidate/profile`, `/candidate/onboarding`, `/candidate/tracker`, `/candidate/interviews`, `/candidate/documents`, `/candidate/referrals`, `/candidate/account`, `/candidate/notifications`, `/candidate/all-notifications`, `/candidate/resume-builder`	`PortalHeader` + `ProfileHero` + `PortalSidebar` (`ContactCard`, `ProfileAsideCards`, `ModuleTiles`)
`DashboardLayout` (legacy)	`components/layout/DashboardLayout.tsx`	`/candidate/subscription`; all `/recruitment/*`	Shared `Sidebar` driven by `components/layout/menu.ts`, breadcrumbs, `CommandPalette`
`Q1Shell`	`features/q1/components/Q1Shell.tsx`	all `/q1/*` (each page renders the shell itself)	Sidebar badges (`GET /q1/nav-counts`), SLA card, `NotificationBell`, quick search
`Q2Shell`	`features/q2/components/Q2Shell.tsx`	all `/q2/*` (rendered per page)	Collapsible sidebar (`aajiveka.q2.sidebarCollapsed`), SLA card (`/q2/nav-counts`, `/q2/sla`)

Shell	File	Used by (routes)	Notes
`Q3Layout` + `Q3Shell`	`features/q3/components/Q3Layout.tsx` (route element, toast context), `Q3Shell.tsx` (rendered per page)	all `/q3/*`	`GET /q3/nav-counts`
`AdminShell`	`features/admin/components/AdminShell.tsx`	all `/admin/*` (rendered per page; 13/13 pages)	Collapsible sidebar, modal mobile drawer
`EmployerLayout`	`employer/layouts/EmployerLayout.tsx` (+ `employer/components/Header/EmployerHeader.tsx`, `Sidebar/EmployerSidebar.tsx`)	all `/company/*`	Forces light mode; sidebar badge from `GET /clients/me/applicants?page=1&pageSize=1`; `messages` / `notifications` badges hard-coded `0`
`AuthShell`	`features/auth/components/AuthShell.tsx`	wraps auth page content inside `PublicLayout`	—

The Q1/Q2/Q3/Admin groups sit outside `DashboardLayout` on purpose (router comments, `routes/router.tsx:255–332`): `DashboardLayout` would stack a second breadcrumb row.

---

3. Route tables

Sources: `routes/router.tsx`, `employer/routes/employerRoutes.tsx`. Guard legend: **Public** = no guard; **Auth** = generic `<ProtectedRoute/>`; role lists are the `allow` prop. `*` → `features/misc/NotFoundPage`.

Backend abbreviations: `auth.ctl → AuthService` means `apps/api/src/modules/auth/auth.controller.ts` → `auth.service.ts`. Same pattern for every module.

3.1 Public & auth

Route	Guard	Page component	Key components / hooks	API calls	Backend
`/`	Public	`features/home/HomePage.tsx`	`JobSearchBar` (`useJobFilters`)	GET `/jobs/filters` *(JobSearchBar on home: inferred)*	`jobs.ctl → JobsService`
`/brochure`	Public	`features/home/brochure/BrochureHomePage.tsx`	static sheets, `brochure/tracking.ts`	none	—
`/login`	Public	`features/auth/pages/LoginPage.tsx`	`SocialLoginButtons`, `AuthShell`, `loginPortals.ts`	POST `/auth/login`; POST `/auth/logout` on wrong portal	`auth.ctl → AuthService`
`/register`	Public	`features/auth/pages/RegisterPage.tsx`	`OtpVerification`	POST `/auth/register`, `/auth/verify-otp`, `/auth/resend-otp`	`auth.ctl → AuthService` + `OtpService` (Redis)
`/register/employer`	Public	`features/auth/pages/EmployerRegisterPage.tsx`	—	POST `/auth/register-employer`	`auth.ctl → AuthService`
`/forgot-password`	Public	`features/auth/pages/ForgotPasswordPage.tsx`	—	POST `/auth/forgot-password {userName}`	`auth.ctl → AuthService`
`/reset-password?token=`	Public	`features/auth/pages/ResetPasswordPage.tsx`	—	POST `/auth/reset-password`	`auth.ctl → AuthService`
`/search`	Public	`features/jobs/pages/SearchResultsPage.tsx`	tabs jobs/companies/candidates	GET `/search` — **no such route**	none (404)
`/candidates/:id`	Public	`features/candidates/pages/PublicProfilePage.tsx`	—	GET `/candidates/:id/profile` — **no such route**	none (404)
`/companies/:id`	Public	`features/clients/pages/CompanyPublicPage.tsx`	`features/public/components/CompanyReviews.tsx`	GET `/clients/:id/public`; reviews widget GET/POST `/clients/:id/reviews` — **no such route**	`employers.ctl → EmployersService` (public card); reviews: none
`/companies/:id/reviews`	Public	`features/public/pages/CompanyReviewsPage.tsx`	review form	GET/POST `/companies/:id/reviews` — **no such route**	none (404)

Route	Guard	Page component	Key components / hooks	API calls	Backend
`/salary-insights`	Public	`features/public/pages/SalaryInsightsPage.tsx`	—	GET `/jobs/salary-insights` — captured by `GET /jobs/:id` + `ParseIntPipe` → 400	`jobs.ctl` (wrong handler)
`/pricing`, `/subscription`, `/resume`	Public	`features/public/pages/PricingPage.tsx`, `SubscriptionPage.tsx`, `ResumePage.tsx`	`PricingPlans` (`usePlans`) → `SubscribeButton` → `RazorpayCheckout`; `PlanComparison` (static)	GET `/payments/plans`; on subscribe see §3.4	`payments.ctl` → `PaymentsService`
`/about`, `/contact`, `/help`, `/career`, `/testimonial`, `/privacy`, `/terms`	Public	`features/public/pages/{About,Contact,Help,Career,Testimonial}Page.tsx`, `LegalPage.tsx` (variant)	static data files	none (Contact submit is simulated, §5)	—
`/blogs`, `/blogs/:slug`	Public	`features/public/pages/BlogsPage.tsx`, `BlogDetailPage.tsx`	`features/public/blogs.data.ts` (hard-coded)	none	— (admin CMS has no public read route)
`/student`	Public	`features/public/pages/StudentGuidePage.tsx`	—	link to `/api/site-docs/aajivika-book` (hard-coded `/api`)	`site-documents.ctl`
`*`	Public	`features/misc/NotFoundPage.tsx`	—	none	—

3.2 Jobs

Route	Guard	Page component	Key components / hooks	API calls	Backend
`/jobs`	Public (`JobsLayout`)	`features/jobs/pages/JobSearchPage.tsx`	`JobResultCard`, `useInfinitePublicJobs`, client-side `matchScore`; Subscriber only: `useCvEditProfile({enabled})`, `useSavedJobIds`, `useSaveJob`, `useUnsaveJob`	GET `/jobs?designation&employmentTypes[]&workModes[]&locations[]&minExp&maxExp&sortBy&page&pageSize`; Subscriber: GET `/candidates/me/cv-edit`, `/candidates/me/saved-job-ids`, POST/DELETE `/candidates/me/saved-jobs/:jobId`	`jobs.ctl` → `JobsService`; `candidates.ctl` → `CandidatesService`
`/jobs/:id`	Public (`JobsLayout`)	`features/jobs/pages/JobDetailPage.tsx`	`MatchBadge`, `useJob`	GET `/jobs/:id`; Subscriber: cv-edit, saved-job-ids, save/unsave	same
`/jobs/:id/apply`	Auth → Subscriber (`JobsLayout`)	`features/jobs/pages/JobApplyPage.tsx`	`useJob`, `useCandidateProfile`, `useCvEditProfile`, `useApplyToJob`, `useUploadResume`	GET `/jobs/:id`, `/candidates/me`, `/candidates/me/cv-edit`; POST `/jobs/:id/apply`; POST `/files/resume`	`jobs.ctl` → `JobsService` (apply transaction in `JobApplicationsService`, per API doc); `files.ctl` → `StorageService`

Hooks in `features/jobs/jobs.api.ts` with no live caller found: `useFullTextJobSearch` (GET `/jobs/search`), `useJobSuggestions` (GET `/jobs/suggestions`) *(inferred from grep of live pages)*.

3.3 Candidate portal

All rows: guard **Auth → Subscriber** (+ onboarding redirect). Hooks: `features/candidates/candidate.api.ts` unless noted (`portal.api.ts` = `features/candidates/portal/portal.api.ts`; `recruitment.api.ts` = `features/recruitment/recruitment.api.ts`; `notifications.api.ts` = `features/notifications/notifications.api.ts`). Backend for every `/candidates/me/*` call: `candidates.ctl → CandidatesService`.

Route	Layout	Page component	Key components / hooks	API calls	Backend (other than CandidatesService)
`/candidate/applications`	`JobsLayout`	`features/candidates/portal/pages/ApplicationsPage.tsx`	`useAppliedJobs` (refetch 60 s); per row `InterviewSlotPicker` (`useInterviewRounds`, `useSelectSlot`) and `OfferResponseCard` (`useOffer`, `useRespondToOffer`) from `recruitment.api.ts`	GET `/candidates/me/applied-jobs`; per application GET `/recruitment/interview-rounds/:mapId`, GET `/recruitment/offers/:mapId`; POST `/recruitment/interview-rounds/:roundId/select-slot {slotId}`; POST `/recruitment/offers/:id/respond {accept}`	`recruitment.ctl` → `MappingAccessService` (ownership) → `InterviewRoundService` / `OfferLetterService`
`/candidate/saved-jobs`	`JobsLayout`	`portal/pages/SavedJobsPage.tsx`	`useSavedJobs`, `useUnsaveJob`	GET `/candidates/me/saved-jobs`; DELETE `/candidates/me/saved-jobs/:jobId`	—
`/candidate/profile`	Portal	`portal/pages/ProfileOverviewPage.tsx`	`useCvEditProfile`, `useCvMasters`, `useDashboard`; `ProfileHero` (photo modal, `useDownloadResume`)	GET `/candidates/me`, `/candidates/me/cv-edit`, `/candidates/me/cv-masters`, `/candidates/me/dashboard`; GET `/files/resume` (blob); POST `/files/avatar`	`files.ctl`
`/candidate/onboarding?step=`	Portal	`portal/pages/ProfileWizardPage.tsx` + `portal/wizard/*Step.tsx`	see §4.2	PUT section endpoints; POST `/candidates/me/complete-onboarding`	`files.ctl` (resume step)
`/candidate/tracker`	Portal	`portal/pages/ApplicationTrackerPage.tsx`	`useAppliedJobs`	GET `/candidates/me/applied-jobs`	—
`/candidate/interviews`	Portal	`portal/pages/InterviewsPage.tsx`	`useAppliedJobs`; "Add to Calendar" = Google Calendar URL	GET `/candidates/me/applied-jobs`	—
`/candidate/documents`	Portal	`portal/pages/DocumentsPage.tsx`	`useCandidateDocuments`, `useUploadCandidateDocument`	GET `/candidates/me/documents`; POST multipart `/candidates/me/documents/:documentTypeId`	—
`/candidate/referrals`	Portal	`portal/pages/ReferPage.tsx`	`useReferrals`, `useCreateReferral` (`portal.api.ts`)	GET/POST `/candidates/me/referrals`	—
`/candidate/account?tab=`	Portal	`portal/pages/AccountSettingsPage.tsx`	`useCandidateProfile`, `useCvEditProfile`, `useUpdatePersonal`, `useUploadAvatar`, `useChangePassword`; `usePrivacySettings`, `useUpdatePrivacy`, `useRequestDataExport`, `useRequestAccountDeletion` (`portal.api.ts`)	PUT `/candidates/me/personal`; POST `/files/avatar`; POST `/candidates/me/change-password`; GET/PUT `/candidates/me/privacy`; POST `/candidates/me/export`; POST `/candidates/me/delete-account`	`files.ctl`
`/candidate/notifications`	Portal	`portal/pages/EmailPreferencesPage.tsx`	`useNotificationPrefs`, `useUpdateNotificationPrefs`	GET/PUT `/candidates/me/notification-prefs`	—
`/candidate/all-notifications`	Portal	`features/candidates/pages/NotificationsPage.tsx`	`useNotifications` (infinite), `useMarkRead`, `useMarkAllRead`, `useDeleteNotification`, `useNotificationStream` (`notifications.api.ts`)	GET `/notifications?page&type&unread`; PATCH `/notifications/:id/read`, `/notifications/read-all`; DELETE `/notifications/:id`; GET `/notifications/stream` (SSE)	`notifications.ctl` → `NotificationsService`, `SseNotificationsService`
`/candidate/resume-builder`	Portal	`portal/pages/ResumeBuilderPage.tsx`	`useCvEditProfile`, `useCvMasters`; html2canvas + jsPDF in browser	GET cv-edit, cv-masters only	—
`/candidate/subscription`	`DashboardLayout`	`features/payments/pages/MembershipPage.tsx`	`useSubscription`, `usePaymentHistory`, `PricingPlans`	see §3.4	`payments.ctl` → `PaymentsService`

Commented-out candidate routes (page files still exist; `routes/router.tsx:57–62, 141–152, 373–389`): `/candidate/profile/advanced`, `/cv-manager`, `/applied-jobs`, `/resume`, `/job-alerts`, `/change-password`, `/assessments(/:id)`, `/messages`, `/saved-searches`, `/skill-gap`, `/interview-calendar`, `/activity`, `/career-path`, `/compare-companies`, `/dashboard`.

3.4 Payments

Hooks: `features/payments/payments.api.ts`. Backend: `payments.ctl → PaymentsService` (+ `RazorpayService`).

Route / entry point	Guard	Component	Hooks	API calls	Notes
`/pricing`, `/subscription`, `/resume`, `/candidate/subscription`	Public (first three), Subscriber (last)	`features/public/components/PricingPlans.tsx` → `features/payments/components/SubscribeButton.tsx` → `RazorpayCheckout`	`usePlans`, `useCreateRazorpayOrder`, `useVerifyPayment`	GET `/payments/plans`; POST `/payments/create-order {planId}`; POST `/payments/verify {razorpayOrderId, razorpayPaymentId, razorpaySignature}`	Guest → toast + `/login` with `from`. Non-Subscriber roles get 403 from create-order. Loads `checkout.razorpay.com` script. Stores `aaj.lastOrderRef` in localStorage
`/candidate/subscription`	Subscriber	`features/payments/pages/MembershipPage.tsx`	`useSubscription`, `usePaymentHistory`	GET `/payments/subscription`, GET `/payments/history`	—
`/payment/return?ref=`	Public (polls only when authenticated)	`features/payments/pages/PaymentReturnPage.tsx`	`useOrder`, `useVerifyPayment`	GET `/payments/orders/:orderRef` every 2 s up to 60 s; POST `/payments/verify` if `razorpay_*` query params are present	BillDesk `useCreateOrder` (POST `/payments/orders`) exists; settlement has no API route (API_DOCUMENTATION.md §12)
Gate on apply	Subscriber	`JobApplyPage.tsx`	`useApplyToJob`	POST `/jobs/:id/apply` → 403 without subscription	UI redirects to `/candidate/subscription?returnTo=/jobs/:id/apply`

3.5 Employer portal

Guard for the whole tree: `ProtectedRoute allow={[Role.Client, Role.Admin]}` (`routes/router.tsx:438–442`), sitting **outside** the generic `<ProtectedRoute/>` group (still redirects unauthenticated users to `/login`). Layout `EmployerLayout`. Hooks: `employer/services/employer.api.ts`. Backend: `employers.ctl` (`@Controller('clients')`, `@Roles(Client, Admin)`) → `EmployersService`, unless noted. Page files under `employer/pages/`.

Route	Page component	Key components / hooks	API calls	Backend (other)
(layout)	`employer/layouts/EmployerLayout.tsx`	`EmployerHeader`, `EmployerSidebar`, `useApplicants`	GET `/clients/me/applicants?page=1&pageSize=1`	—
`/company`	`Dashboard/DashboardPage.tsx`	`useCompanyAnalytics`, `useApplicants` ×3	GET `/clients/me/analytics`; GET `/clients/me/applicants` (pageSize 6; status=Interview; status=OnHold)	—
`/company/profile`	`CompleteProfile/CompleteProfilePage.tsx`	`useCompanyProfile`, `useCompanyMasters`, `useUpdateCompanyProfile`, `useUploadCompanyLogo`	GET `/clients/me`, GET `/clients/masters`, PATCH `/clients/me`, POST `/clients/me/logo`; `<img src="/api/clients/:id/logo">` (hard-coded `/api`)	—
`/company/jobs`, `/company/jobs/drafts`, `/company/jobs/archived`	`ManageJobs/JobList/JobListPage.tsx` (status from path) + `JobViewModal.tsx`	`useCompanyJobs`, `useCompanyJob`, `useSetJobStatus`, `useArchiveJob`, `useUnarchiveJob`, `useDeleteJob`	GET `/clients/me/jobs?page&pageSize&q&city&status`; GET `/clients/me/jobs/:id`; POST `.../:id/status`, `.../:id/archive`, `.../:id/activate`; DELETE `/clients/me/jobs/:id`	—
`/company/post-job`, `/company/jobs/:id/edit`	`ManageJobs/AddJob/AddJobPage.tsx` + `JobPreviewModal.tsx`	`useCompanyMasters`, `useCompanyJob`, `usePostJob`, `useUpdateJob`	GET `/clients/masters`; GET `/clients/me/jobs/:id`; POST `/clients/me/jobs[?draft=true]`; PATCH `/clients/me/jobs/:id[?draft=]`	—
`/company/bulk-import`	`ManageJobs/BulkImport/BulkImportPage.tsx`	`useUploadBulkJobs`; template built client-side (`downloadJobImportTemplate`)	POST `/clients/me/jobs/bulk-upload?commit=false	true` (multipart CSV)

Route	Page component	Key components / hooks	API calls	Backend (other)
`/company/applicants`, `/company/applicants/shortlisted`, `/interviews`, `/hired`, `/rejected`	`Applicants/ApplicantList/ApplicantListPage.tsx` (status from path)	`useCompanyJobs`, `useApplicants`, `useDecideApplicant`, `useUndoApplicantDecision`; `employer/components/Tables/DataTable.tsx`	GET `/clients/me/jobs?pageSize=100`; GET `/clients/me/applicants?page&pageSize&status&q&jobId&city&minExp&maxNotice`; POST `.../applicants/:id/decision`, `.../decision/undo`; interview icon → `/company/applicants/:id?schedule=1`	—
`/company/applicants/:id`	`Applicants/ApplicantProfile/ApplicantProfilePage.tsx` + `InterviewProcessPanel.tsx`, `ScheduleInterviewForm.tsx`, `RequestDocumentsForm.tsx`, `ApplicantDocumentsPanel.tsx`	`useApplicant` / `useApplicantDetails`, `useApplicantResumeBlob`, `useApplicantNotes`, `useSaveApplicantNote`, `useDecideApplicant`, `useUndoApplicantDecision`, `useApplicantInterviewRounds`, `useScheduleInterview`, `useRecordInterviewResult`, `useApplicantDocuments`, `useReviewApplicantDocument`, `useRequestableDocumentTypes`, `useRequestApplicantDocuments`; `useOffer`, `useCreateOffer`, `useSendOffer` (`recruitment.api.ts`)	GET `/clients/me/applicants/:id`; GET `.../resume?inline=1` and `.../resume` (blobs); GET/PUT `.../notes`; POST `.../decision`, `.../decision/undo`; GET/POST `.../interview-rounds`; POST `.../interview-rounds/:roundId/result`; GET `.../documents`; POST `.../documents/review`; GET `/clients/me/document-types`; POST `.../document-request`; GET `/recruitment/offers/:mapId`, POST `/recruitment/offers`, `/recruitment/offers/:id/send`, `/recruitment/offers/:id/mark-joined`	`EmployersService` → `InterviewRoundService.submitResult` (result); `recruitment.ctl` → `MappingAccessService` → `OfferLetterService` (offers)
`/company/compare-candidates`	`CompareCandidates/ComparePage.tsx` (max 4)	`useApplicants`, `useApplicant`, `useDecideApplicant`	GET `/clients/me/applicants?pageSize=100`; GET `/clients/me/applicants/:id` per selection; POST decision	—
`/company/analytics`	`Analytics/AnalyticsPage.tsx`	`useCompanyAnalytics`	GET `/clients/me/analytics`; GET `/clients/me/analytics/export` (CSV blob); summary CSV built client-side	—
`/company/billing`	`Billing/BillingPage.tsx`	`useCompanyBilling`	GET `/clients/me/billing`; GET `/clients/me/billing/export`	—
`/company/messages`	`Messages/MessagesPage.tsx`	—	none ("coming soon")	—
`/company/notifications`	`Notifications/NotificationsPage.tsx`	—	none ("No notifications yet")	—
`/company/settings`	`Settings/SettingsPage.tsx`	—	none (static, links to profile)	—

Admin can enter this tree and has `/company/profile`, `/company/jobs`, `/company/applicants` in its legacy sidebar menu (`components/layout/menu.ts:95-105`), but the API returns 404 on every `/clients/me/*` route for a login with no linked company (API_DOCUMENTATION.md §6).

Employer hooks defined but imported nowhere: `useDeactivateJob` (POST `/clients/me/jobs/:id/deactivate`), `useBulkDecideApplicants`, `useDuplicateJob` (POST `/clients/me/jobs/:jobId/duplicate`). API routes with no web caller: GET/PATCH `/clients/me/branding`, POST `/clients/me/interviews/generate-link`, GET `/exports/applicants`.

3.6 Q1 screening (`/q1/*`)

Guard: Auth → `[QC1, Admin]` (`routes/router.tsx:263`). Shell `Q1Shell`. Hooks: `features/q1/q1.api.ts`; every mutation invalidates `['q1']`. Backend: `q1.ctl` (`@Roles(QC1, Admin)`) → `Q1Service` (analytics → `Q1AnalyticsService`).

Route	Page component	Key components / hooks	API calls
`/q1`	redirect → `/q1/dashboard`	—	—
`/q1/dashboard`	`features/q1/pages/Q1DashboardPage.tsx`	`CandidateQueue` (tabs), stat cards; `useQ1Stats`, `useQ1Queue`	GET `/q1/stats`; GET `/q1/candidates?tab&search&priority&experience&cvOnly&page&pageSize`
`/q1/candidates`	`Q1CandidatesPage.tsx`	`CandidateQueue showTabs`, seeded from `?search=`	GET `/q1/candidates`

Route	Page component	Key components / hooks	API calls
`/q1/candidates/:id`	`Q1CandidateProfilePage.tsx`	`ScreeningRail`, `ContactCandidateModal`, `UpdateStatusModal`; `useQ1Profile`	GET `/q1/candidates/:id` (**side effect: New → Screening**); PATCH `.../checklist`, `.../checklist/all`; POST `.../verify`, `.../contact`, `.../status`, `.../request-update`, `.../request-cv`; `window.open('/api/recruitment/candidates/:id/resume')` (broken, §5)
`/q1/follow-ups`, `/q1/no-response`, `/q1/verified`, `/q1/not-interested`	`Q1FollowUpsPage.tsx`, `Q1NoResponsePage.tsx`, `Q1VerifiedPage.tsx`, `Q1NotInterestedPage.tsx`	`CandidateQueue` pinned to a tab	GET `/q1/candidates?tab=…`
`/q1/analytics`	`Q1AnalyticsPage.tsx`	`useQ1Analytics`	GET `/q1/analytics` → `Q1AnalyticsService`
(shell)	`features/q1/components/Q1Shell.tsx`	`useQ1NavCounts`, `components/layout/NotificationBell`	GET `/q1/nav-counts`

`useQ1ContactLog` (GET `/q1/candidates/:id/contact-log`) is defined but rendered nowhere.

3.7 Q2 matching (`/q2/*`)

Guard: Auth → `[QC2, Admin]` (`routes/router.tsx:285`). Shell `Q2Shell`. Hooks: `features/q2/q2.api.ts`; mutations invalidate `['q2']` except the criterion toggle (writes into cache). Backend: `q2.ctl` (`@Roles(QC2, Admin)`) → `Q2Service` (analytics → `Q2AnalyticsService`).

Route	Page component	Key components	API calls
`/q2`	redirect → `/q2/dashboard`	—	—
`/q2/dashboard`	`features/q2/pages/Q2DashboardPage.tsx`	stat cards, `EmployerJobsTable`	GET `/q2/stats`; GET `/q2/jobs?search`
`/q2/jobs`	`Q2EmployerJobsPage.tsx`	`EmployerJobsTable` (300 ms debounce)	GET `/q2/jobs`
`/q2/jobs/:jobId`	`Q2JobApplicantsPage.tsx`	ranked list, "Relevant only" toggle	GET `/q2/jobs/:jobId?relevantOnly`
`/q2/applicants`	`Q2AllApplicantsPage.tsx`	`ApplicantsTable` bucket=queue	GET `/q2/applicants?tab&bucket&search&page&pageSize`
`/q2/forwarded`	`Q2ForwardedPage.tsx`	`ApplicantsTable` bucket=forwarded	GET `/q2/applicants?bucket=forwarded`
`/q2/sent-back`	`Q2SentBackPage.tsx`	`ApplicantsTable` bucket=sentBack	GET `/q2/applicants?bucket=sentBack`
`/q2/applications/:mapId`	`Q2ApplicantDetailPage.tsx`	candidate panel, JD panel, criteria checkboxes, 3 decision buttons	GET `/q2/applications/:mapId`; PATCH `.../criteria`; POST `.../forward`, `.../send-back`, `.../reject`; `<a href="/api/recruitment/candidates/:subscriberId/resume">` (broken, §5)
`/q2/analytics`	`Q2AnalyticsPage.tsx`	—	GET `/q2/analytics` → `Q2AnalyticsService`
(shell)	`features/q2/components/Q2Shell.tsx`	—	GET `/q2/nav-counts`, GET `/q2/sla`

Q2 has no "map candidate to job" action; it reviews applications that already exist (candidate apply, or legacy `POST /recruitment/candidates/:id/assign-job`).

3.8 Q3 hiring operations (`/q3/*`)

Guard: Auth → `[Q3, Admin]`, wrapped in `Q3Layout` (`routes/router.tsx:305-309`). Hooks: `features/q3/q3.api.ts`; writes invalidate `['q3']` (meeting-link excepted). Backend: `q3.ctl` (`@Roles(Q3, Admin)`) → one of four services; the route → service pairing below was read from `apps/api/src/modules/q3/q3.controller.ts`.

Route	Page component	Key components	API calls → service
`/q3`	redirect → `/q3/dashboard`	—	—
`/q3/dashboard`	`features/q3/pages/Q3DashboardPage.tsx`	pipeline table + stage tabs, `ExportReport` (jsPDF on demand, `q3-report.ts`)	GET `/q3/dashboard`, `/q3/pipeline`, `/q3/report?quarter` → `Q3ReadService`; POST `/q3/applications/:mapId/contact`, `/q3/rounds/:r/decision-mail`, `/q3/applications/:mapId/nudge` → `Q3InterviewsService`

Route	Page component	Key components	API calls → service
`/q3/referred`	`Q3ReferredCvsPage.tsx`	batch picker, `CvRow`, `ReturnToQ2Modal`	GET `/q3/referrals`; POST `/q3/referrals/forward`, `/q3/referrals/return` → `Q3ReferralsService`
`/q3/company-review`	`Q3CompanyReviewPage.tsx`	`ShortlistPanel`, `RoundSetupModal`, `RemindExpiringButton`	GET `/q3/company-review?tab`, `/q3/applications/:mapId/shortlist`; PUT `.../round-setup`; POST `/q3/company-review/remind-expiring` → `Q3ReferralsService`
`/q3/scheduling`, `/q3/scheduling/:mapId`	`Q3SchedulingPage.tsx`	`QueueSwitcher`, stepper, slot list, `ConfirmCard`, contact-log form	GET `/q3/queues/scheduling` → `Q3ReadService`; GET `.../scheduling`; POST `/q3/rounds/:r/slots/:s/availability`, `/q3/rounds/:r/request-slots`, `/q3/meeting-link` (Jitsi), `/q3/rounds/:r/confirm`, `.../contact` → `Q3InterviewsService`
`/q3/interviews`	`Q3InterviewsPage.tsx`	week view (refetch 60 s)	GET `/q3/interviews?week`; POST `/q3/rounds/:r/remind
`/q3/decisions`, `/q3/decisions/:mapId`	`Q3RoundsPage.tsx`	`RecordDecisionModal`	GET `/q3/queues/decisions` → `Q3ReadService`; GET `.../rounds`; POST `/q3/rounds/:r/decision`, `.../nudge` → `Q3InterviewsService`
`/q3/documents[/:mapId]`, `/q3/offers[/:mapId]`	`Q3DocumentsPage.tsx` (mode from pathname)	`DocumentRow`, `DocumentPreviewModal`, `SendBackModal`, `GenerateOfferModal`, approve/join confirm modals	GET `/q3/queues/documents
`/q3/returned`	`Q3ReturnedPage.tsx`	tabs, `ReturnMailModal`	GET `/q3/returned?tab`, `/q3/returned/:referralId/mail`; POST `.../resend` → `Q3ReferralsService`
(shell)	`features/q3/components/Q3Shell.tsx`	—	GET `/q3/nav-counts` → `Q3ReadService`

3.9 Legacy `/recruitment/*`

All inside `DashboardLayout`. Hooks: `features/recruitment/recruitment.api.ts`. Backend: `recruitment.ctl` (class `@Roles(QC1, QC2, Admin)`, method `@Roles` replaces it) → `RecruitmentService` unless noted. Page files under `features/recruitment/pages/`.

Route	Guard	Page component	API calls	Backend (other)
`/recruitment/candidates`	QC1, QC2, Admin	`CandidatesListPage.tsx`	GET `/recruitment/candidates?search&status&page&pageSize`	—
`/recruitment/candidates/:id`	QC1, QC2, Admin	`CandidateDetailsPage.tsx`	GET `/recruitment/candidates/:id`; POST `.../decision`; GET `/recruitment/jobs`; POST `.../assign-job`; GET `/recruitment/document-types`; POST `.../documents`; GET `.../resume` (blob, OK); POST `/recruitment/referrals`; POST `/recruitment/applications/:id/score`	`CvReferralService` (referrals); `ScoringService` (score)
`/recruitment/documents`	QC1, QC2, Admin	`DocumentReviewPage.tsx`	GET `/recruitment/documents`; GET `/recruitment/documents/:id/file` (blob); POST `/recruitment/documents/review`	—
`/recruitment/interviews`	QC1, QC2, Admin	`InterviewsPage.tsx`	GET `/recruitment/interviews`, `/interviews/eligible`, `/interview-modes`; POST `/recruitment/interviews`, `/recruitment/interviews/:id/status`	—
`/recruitment/qc1`	QC1, Admin	`QC1DashboardPage.tsx`	GET `/recruitment/qc1/stats`	—
`/recruitment/referrals`	QC2, Admin	`ReferralsPage.tsx`	GET `/recruitment/referrals`	`CvReferralService`
`/recruitment/q3`	Q3, Admin	`Q3DashboardPage.tsx` + `InterviewRoundsPanel`	GET `/recruitment/referrals`; POST `/recruitment/referrals/forward`; GET `/recruitment/interview-rounds/:mapId`, `/interview-modes`; POST `/recruitment/interview-rounds`, `.../:roundId/result`	`CvReferralService`; `MappingAccessService` → `InterviewRoundService`
`/recruitment/q3/candidates`	Q3, Admin	`CandidatesListPage.tsx`	GET `/recruitment/candidates` → **403 for Q3**	—
`/recruitment/q3/interviews`	Q3, Admin	`InterviewsPage.tsx`	GET `/recruitment/interviews`, `/interviews/eligible` → **403 for Q3**	—
`/recruitment/q3/documents`	Q3, Admin	`DocumentReviewPage.tsx`	GET `/recruitment/documents` → **403 for Q3**	—

Unused recruitment hooks: `useAddRoundSlots` (POST `/recruitment/interview-rounds/:id/slots`), `useCompanyReviewReferral` (POST `/recruitment/referrals/:id/company-review`). `PATCH /recruitment/pipeline/:id` has no web caller.

3.10 Admin (`/admin/*`)

Guard: Auth → `[Admin]` (`routes/router.tsx:334`). Shell `AdminShell`. Hooks: `features/admin/admin.api.ts` (mutations invalidate `['admin']`); Settings and Blog use inline `useQuery`/`useMutation`. Backend: `admin.ctl` (`@Roles(Admin)`) → `AdminService` (+ `staff-account.ts`); exports → `exports.ctl`. Page files under `features/admin/pages/`.

Route	Page component	Key components	API calls
`/admin`	`AdminDashboardPage.tsx`	stat cards, activity feed	GET `/admin/stats`
`/admin/users`	`AdminUsersPage.tsx`	`CreateStaffModal`, `ExportButton`	GET `/admin/users?page&q&roleId&isActive`; POST `/admin/users`; POST `/admin/users/bulk-update`; GET `/exports/users?format=csv`
`/admin/users/:userId`	`AdminUserDetailPage.tsx`	`ConfirmDialog`, `SetPasswordNotice`	GET `/admin/users/:id`; PATCH `/admin/users/:id {isActive
`/admin/employer-approvals`	`AdminEmployerApprovalsPage.tsx`	`ReviewModal`	GET `/admin/employer-registrations?status`; GET `/clients/masters` (`employers.ctl`); POST `/admin/employer-registrations/:id/review`
`/admin/companies`	`AdminCompaniesPage.tsx`	—	GET `/admin/companies?page&q`
`/admin/jobs`	`AdminJobsPage.tsx`	bulk bar, `ExportButton`	GET `/admin/jobs?page&q&status&clientId`; POST `/admin/jobs/:id/status` (one) or `/admin/jobs/bulk-moderate` (several, chosen in `admin.api.ts:193-199`); GET `/exports/jobs`
`/admin/jobs/:jobId`	`AdminJobDetailPage.tsx`	—	GET `/admin/jobs/:id`; POST `.../status`
`/admin/applications`	`AdminApplicationsPage.tsx`	`ExportButton`	GET `/admin/applications?page&q&statusId&stage&jobId`; GET `/exports/applications`
`/admin/applications/:mapId`	`AdminApplicationDetailPage.tsx`	`NextStepLink` (deep link to Q1/Q2/Q3 owner)	GET `/admin/applications/:id` (read-only)
`/admin/blog`	`BlogCmsPage.tsx`	editor	GET/POST `/admin/blog-posts`; PATCH/DELETE `/admin/blog-posts/:id`
`/admin/settings`	`AdminSettingsPage.tsx`	form	GET/PATCH `/admin/settings` (stored, not enforced)
`/admin/reports`	`ReportsPage.tsx`	`ExportButton`	GET `/admin/stats`; GET `/exports/users
`/admin/audit`	`AdminAuditLogPage.tsx`	—	GET `/admin/audit-logs?page&action&entity&userId`

`POST /admin/users/bulk-delete` has no admin UI.

---

4. Key forms and actions

Column "Wired?": **yes** = calls a real endpoint and handles the result; **no-op** = nothing reaches the server; **broken** = reaches the server but cannot succeed or mis-reports.

4.1 Auth

Action	Component	Validation	API	Success	Failure	Wired?
Login	`features/auth/pages/LoginPage.tsx`	zod `loginSchema` (userName, password min 1; `auth.types.ts:22-26`)	POST `/auth/login`	wrong door → revoke + toast; else `setSession`, toast, `navigate(from ?? ROLE_HOME)`	toast with API message	yes
Register candidate	`RegisterPage.tsx`	zod `registerSchema`: fullName 2-100, email, ISO2 country, mobile 4-15 digits (IN `^[6-9]\d{9}$`), password 8-72	POST `/auth/register`	stores challenge, toast "OTP sent" (non-prod shows `devCode`)	toast API message	yes

Action	Component	Validation	API	Success	Failure	Wired?
OTP verify / resend	`features/auth/components/OtpVerification.tsx`	6 digits, auto-submit, countdown	POST `/auth/verify-otp`, `/auth/resend-otp`	`setSession` → `/candidate/onboarding`	inline aria-live error; 429 uses `retryAfterSeconds`	yes
Register employer	`EmployerRegisterPage.tsx`	RHF only: companyName, emailCompany required + regex	POST `/auth/register-employer` (blank fields sent as `''`)	"under review" screen	toast API message	**broken** when HR email is left blank (`''` fails `@IsOptional()` `@IsEmail()`)
Forgot password	`ForgotPasswordPage.tsx`	zod userName min 1	POST `/auth/forgot-password`	success Alert	**no onError, no error UI**	yes (silent on failure)
Reset password	`ResetPasswordPage.tsx`	zod token, password ≥8, confirm match	POST `/auth/reset-password`	toast + `/login`	generic toast (token errors not distinguished)	yes
Social login	`features/auth/components/SocialLoginButtons.tsx`	—	`window.location.href='/auth/google`	linkedin"	—	—
Logout	`ProfilePanel.tsx`, `EmployerHeader.tsx`, `DashboardLayout.tsx`	—	POST `/auth/logout`	tokens cleared; route guard redirects	errors swallowed	yes (cache not cleared, §5)

4.2 Candidate

Action	Component	Validation	API	Success	Failure	Wired?
Wizard: Personal	`portal/wizard/PersonalStep.tsx`	zod: fullName ≥2, mobile ≥10 chars, optional email/LinkedIn URL, experience unless fresher	PUT `/candidates/me/personal`, `/headline`, `/professional`, `/accomplishments` (only changed parts)	invalidates `['candidate','cv-edit']`, `['candidate','profile','me']`; next step	inline "Could not save"	yes (blank gender sent as `'M'`)
Wizard: Summary / Experience / Projects	`SummaryStep.tsx`, `ExperienceStep.tsx`, `ProjectsStep.tsx`	company / title required	PUT `/summary`; PUT/DELETE `/employment`, `/projects`	next step	inline	yes
Wizard: Education	`EducationStep.tsx`	zod `educationDraftSchema` (`wizard/validation.ts:32-78`)	GET `/candidates/me/institutes` (250 ms debounce); PUT/DELETE `/education`	next step	per-field	yes
Wizard: Skills / Preferences	`SkillsStep.tsx`, `PreferencesStep.tsx`	non-empty entries	PUT `/key-skills`, PUT/DELETE `/languages`, `/it-skills`; PUT `/career-profile`, `/professional`	next step	inline	yes
Wizard: Resume	`ResumeStep.tsx`	PDF only, ≤5 MB (client)	POST `/files/resume` (progress)	invalidates profile	413 → "too large"	yes
Complete onboarding	`portal/pages/ProfileWizardPage.tsx`	—	POST `/candidates/me/complete-onboarding`	`updateUser({isOnboarded:true})`, toast, `/candidate/profile`	navigates to `/candidate/profile` anyway → guard bounces back to wizard, no message	yes (failure path loops — *runtime verification needed*)
Profile photo	`ProfileHero` → `features/candidates/components/ProfilePhotoUpload.tsx`; Account personal tab	size check	POST `/files/avatar`	invalidate, toast	toast	yes
Resume builder	`portal/pages/ResumeBuilderPage.tsx`	—	none (html2canvas + jsPDF)	downloads PDF	toast	yes (client-only, raster PDF)

Action	Component	Validation	API	Success	Failure	Wired?
Home search bar	`features/jobs/components/JobSearchBar.tsx`	—	navigates `/jobs?designation=…&location=…`	—	—	**broken**: `JobSearchPage` reads `q`, keyword dropped
Job search filters	`JobSearchPage.tsx`	—	GET `/jobs` with array params	results	`ErrorState` + retry	yes; arrays may 400 (§5, potential)
Save / unsave job	`JobSearchPage`, `JobDetailPage`, `SavedJobsPage`	—	POST/DELETE `/candidates/me/saved-jobs/:jobId`	invalidates `['candidate','saved-jobs']`, `['candidate','saved-job-ids']`	silent	yes
Apply	`features/jobs/pages/JobApplyPage.tsx`	hand-rolled required fields + email regex	POST `/jobs/:id/apply`; POST `/files/resume`	"done" stage with reference; invalidates `['candidate','applied-jobs']`, `['candidate','dashboard']`	403 → `/candidate/subscription?returnTo=…`; **any 400 → "already applied"**	yes (400 mislabelled)
Pick interview slot	`portal/components/InterviewSlotPicker.tsx`	—	POST `/recruitment/interview-rounds/:roundId/select-slot`	toast; invalidates `['recruitment','interview-rounds']`	toast	yes
Accept / decline offer	`portal/components/OfferResponseCard.tsx`	—	POST `/recruitment/offers/:id/respond {accept}`	toast; invalidates `['recruitment','offers']`	toast	yes
Upload document	`portal/pages/DocumentsPage.tsx`	`.pdf,.jpg,.png`; disabled if no `documentTypeId`	POST `/candidates/me/documents/:documentTypeId`	toast; invalidates `['candidate','documents']`	413 / generic inline	yes
Refer a friend	`portal/pages/ReferPage.tsx`	name + email required	POST `/candidates/me/referrals`	toast "Invite sent"; invalidates `['candidate','referrals']`	toast	**partly**: no email is sent; `/register?ref=` is never read
Personal info	`AccountSettingsPage.tsx` (PersonalInfoTab)	none	PUT `/candidates/me/personal`	toast	toast	yes (gender default `'M'`)
Change password	AccountSettings SecurityTab	new ≥8, confirm	POST `/candidates/me/change-password`	toast, reset	any 400 → "current password not correct"	yes
Privacy / email prefs	PrivacyTab; `EmailPreferencesPage.tsx`	—	GET/PUT `/candidates/me/privacy`, `/notification-prefs`	toast	toast	yes, but stored values are read by no API code
Data export	DangerZoneTab	—	POST `/candidates/me/export` (202)	toast "we will email you"	toast	**no-op downstream**: API only stamps a date
Delete account	DangerZoneTab + `ConfirmDialog`	confirm	POST `/candidates/me/delete-account`	toast, `logout()`, `/`	toast	yes (deactivates login)
Buy membership	`PricingPlans` → `SubscribeButton` → `RazorpayCheckout`	guest → `/login`	POST `/payments/create-order`, `/payments/verify`	invalidates `['payments','subscription']`, toast, `returnTo` or `/payment/return`	toast; cancel silent	yes
Payment return	`PaymentReturnPage.tsx`	—	GET `/payments/orders/:ref` poll; POST `/payments/verify` fallback	invalidate subscription, clear `aaj.lastOrderRef`	timed-out state	yes

Action	Component	Validation	API	Success	Failure	Wired?
Withdraw application / request reschedule	—	—	none	—	—	**Missing implementation** (no UI, no candidate API)

4.3 Employer

Action	Component	Validation	API	Success	Failure	Wired?
Post / edit job (publish or draft)	`employer/pages/ManageJobs/AddJob/AddJobPage.tsx`	hand-rolled: publish needs position, type, mode, minExp, city, education, industry, department, ≥1 skill, description; CTC and rounds ranges; scrolls to first error	POST `/clients/me/jobs[?draft=true]`, PATCH `/clients/me/jobs/:id[?draft=]`	navigate to list/drafts; invalidates `['employer','jobs']`	banner with API message	yes
Close / activate / archive / unarchive / delete job	`JobListPage.tsx` (+ `ConfirmDialog` for delete/archive)	—	POST `.../status`, `.../archive`, `.../activate`; DELETE	invalidates `['employer','jobs']`	`actionError` banner	yes
Bulk import	`BulkImportPage.tsx`	`.csv`	POST `/clients/me/jobs/bulk-upload?commit=false` then `true`	commit invalidates jobs	shows axios generic message, not API message	yes (poor error text)
Applicant decision / undo	`ApplicantListPage`, `ApplicantProfilePage`, `ComparePage` (+ `utils/decisionConfirm.ts`)	confirm	POST `.../decision {Shortlisted|Rejected}`, `.../decision/undo`	Hired		invalidates `['employer','applicants']`, `['employer','analytics']`, `['employer','billing']` (+ rounds on undo)
Schedule interview	`ScheduleInterviewForm.tsx`	HR + interviewer name/email, optional guest pair, venue for face-to-face, 3 distinct future slots	POST `.../interview-rounds`	invalidates applicants/analytics/rounds	inline	yes
Record round result	`InterviewProcessPanel.tsx`	—	POST `.../interview-rounds/:roundId/result`	invalidates pipeline + billing	inline	yes
Notes	`ApplicantProfilePage.tsx`	non-empty	PUT `.../notes {note}`	invalidates notes	banner	yes
Request / review documents	`RequestDocumentsForm.tsx`, `ApplicantDocumentsPanel.tsx`	≥1 name	POST `.../document-request`, `.../documents/review`	invalidates docs	inline	yes
Offer create / send / mark joined	`ApplicantProfilePage.tsx`	salary required	POST `/recruitment/offers`, `.../send`, `.../mark-joined`	refetch offer	banner	yes
Company profile + logo	`CompleteProfile/CompleteProfilePage.tsx`	none (`noValidate`, `*` markers only)	PATCH `/clients/me`; POST `/clients/me/logo`	inline "saved"	inline API message	yes
Analytics / billing CSV	`AnalyticsPage.tsx`, `BillingPage.tsx`	—	GET `.../analytics/export`, `.../billing/export` (blob)	download	—	yes
Team, change password, messages, notifications	`SettingsPage`, `MessagesPage`, `NotificationsPage`	—	none	—	—	**Missing implementation**

4.4 Q1 / Q2 / Q3 staff

Action	Component	Validation / confirm	API	Success	Failure	Wired?
Q1 checklist toggle / select all	`features/q1/components/ScreeningRail.tsx`	—	PATCH `/q1/candidates/:id/checklist`, `/checklist/all`	invalidates `['q1']`	toast with server message	yes
Q1 Mark verified	`ScreeningRail.tsx` (only at 100% completeness)	—	POST `.../verify`	toast + invalidate	toast	yes
Q1 Contact candidate	`ContactCandidateModal.tsx`	modal	POST `.../contact` `{channel, reason, internalNote}`	toast, then opens `UpdateStatusModal`	toast	yes (logs only; nothing is sent)
Q1 Update status	`UpdateStatusModal.tsx`	date not past, attempts 0-99, reason required	POST `.../status`	toast + invalidate	toast	yes; reachable only after a contact
Q1 Request update / request CV	`ScreeningRail.tsx`, `Q1CandidateProfilePage.tsx`	—	POST `.../request-update`, `.../request-cv`	toast	toast	yes
Q1 View / download CV (4 buttons)	`Q1CandidateProfilePage.tsx:138,144,277,284`	—	`window.open('/api/recruitment/candidates/:id/resume')`	—	—	**broken** (401, no bearer)
Q2 Toggle criterion	`Q2ApplicantDetailPage.tsx:92-97`	—	PATCH `.../criteria`	`setQueryData`	toast	yes
Q2 Forward / Send back / Reject	`Q2ApplicantDetailPage.tsx:172-198`	**no confirmation, no reason**	POST `.../forward	send-back	reject `with {}`	toast + invalidate `['q2']`
Q2 Preview resume	`Q2ApplicantDetailPage.tsx:288-296`	—	`<a href="/api/recruitment/candidates/:id/resume">`	—	—	**broken** (401)
Q3 Forward / return CVs	`Q3ReferredCvsPage.tsx`, `ReturnToQ2Modal.tsx`	return: reason required	POST `/q3/referrals/forward`, `/return`	toast; forward navigates to `/q3/company-review`	toast	yes
Q3 Round setup	`RoundSetupModal.tsx`	modal form	PUT `.../round-setup`	toast	toast	yes (no "company rejected" option)
Q3 Confirm slot & send invites	`Q3SchedulingPage.tsx:629-656`	slot required; http(s) link for virtual; ≥1 recipient	POST `/q3/rounds/:r/confirm`	toast "Invites sent"	toast	yes
Q3 Call / Email candidate	`Q3SchedulingPage.tsx:150-178`	—	Call: POST `.../contact` then `tel:`; Email: `mailto:` only	toast	—	Call yes (logged before call); Email client-only, not logged
Q3 Interview-day actions	`Q3InterviewsPage.tsx:727-758`	none (reschedule unconfirmed)	POST `/q3/rounds/:r/remind	checklist	reschedule	conducted`
Q3 Record decision	`RecordDecisionModal.tsx`	modal	POST `/q3/rounds/:r/decision`	toast; Selected → `/q3/documents/:mapId`	toast	yes
Q3 Documents verify / send back / forward / approve	`Q3DocumentsPage.tsx`, `SendBackModal`	send-back comment; approve confirm	POST `/q3/documents/:id/verify	send-back, .../documents/forward	approve`	toast
Q3 Generate offer / mark joined	`GenerateOfferModal.tsx`, `Q3DocumentsPage.tsx`	modal / confirm	POST `.../offer`, `.../joined`	toast	toast	yes (no resend/revise/withdraw)
Q3 Export report	`Q3DashboardPage.tsx:294-341`, `q3-report.ts`	—	GET `/q3/report?quarter` → jsPDF	download	toast	yes

4.5 Legacy recruitment and admin

Action	Component	Validation / confirm	API	Success	Failure	Wired?
Candidate decision	`features/recruitment/pages/CandidateDetailsPage.tsx`	reason modal (except Approve)	POST `/recruitment/candidates/:id/decision`	toast + invalidate	toast	yes
Assign job	same	modal	POST `.../assign-job`	toast	toast	yes, but **no query invalidation** (`useAssignJob`)
Refer to Q3 / score	same	—	POST `/recruitment/referrals` ; POST `/recruitment/applications/:id/score`	toast / breakdown modal	toast	yes
Schedule interview	`InterviewsPage.tsx`	RHF + zod modal	POST `/recruitment/interviews`	toast + invalidate	toast	yes
Complete / cancel interview	`InterviewsPage.tsx:58-62`	**no confirm**	POST `/recruitment/interviews/:id/status`	toast	**no onError (silent)**	yes (silent failures)
Round create / result	`InterviewRoundsPanel.tsx`	—	POST `/recruitment/interview-rounds` , `.../result`	toast	toast	yes (`next` never sent)
Approve / reject employer	`features/admin/pages/AdminEmployerApprovalsPage.tsx`	two-step modal; approval needs city	POST `/admin/employer-registrations/:id/review`	toast; invalidate `['admin']`	toast (409 on repeat)	yes
Activate / deactivate / change role / reset password	`AdminUserDetailPage.tsx`	`ConfirmDialog` ; self disabled; staff roles only	PATCH `/admin/users/:id` ; POST `.../password-reset`	toast / `SetPasswordNotice`	toast	yes
Bulk user update / create staff	`AdminUsersPage.tsx`	`ConfirmDialog` / RHF	POST `/admin/users/bulk-update` , `/admin/users`	toast / `SetPasswordNotice`	toast / inline	yes
Close / reopen job	`AdminJobsPage.tsx` , `AdminJobDetailPage.tsx`	`ConfirmDialog`	POST `/admin/jobs/:id/status` , `/bulk-moderate`	toast	toast	yes
Save settings	`AdminSettingsPage.tsx`	none (empty number → 0)	PATCH `/admin/settings` (whole form)	toast + invalidate	toast	yes, but settings are not enforced; may save defaults if GET failed (potential)
Blog create / edit / publish / delete	`BlogCmsPage.tsx`	`ConfirmDialog` on delete	POST/PATCH/DELETE `/admin/blog-posts`	toast; invalidate `['admin','blog-posts']`	toast	yes (posts never shown publicly)
CSV exports	`features/admin/components/ExportButton.tsx`	—	GET `/exports/*?format=csv&from&to` (blob)	download	toast "Export failed"	yes

---

5. Dead or broken UI

5.1 Confirmed issues

#	Issue	Evidence (file)
C1	Q1 and Q2 CV view/download open `/api/recruitment/candidates/:id/resume` by top-level navigation → 401 "Missing bearer token"	`features/q1/pages/Q1CandidateProfilePage.tsx:138,144,277,284` ; `features/q2/pages/Q2ApplicantDetailPage.tsx:288-296` ; `apps/api/src/common/guards/jwt-auth.guard.ts:34-35`

#	Issue	Evidence (file)
C2	Legacy Q3 screens 403: `/recruitment/q3/candidates`, `/interviews`, `/documents` call endpoints restricted to QC1/QC2/Admin; they are the Q3 sidebar links in `DashboardLayout`. Interviews page ignores `isError` and shows "no interviews"	`routes/router.tsx:428-430`; `components/layout/menu.ts:107-111`; `apps/api/src/modules/recruitment/recruitment.controller.ts:36`
C3	Public pages call endpoints that do not exist: `/search`, `/candidates/:id/profile` (target of every candidate's "Share Profile" in `ProfileHero.tsx`), `/clients/:id/reviews` (widget on `/companies/:id`), `/companies/:id/reviews`, `/jobs/salary-insights` (400)	§3.1; API_DOCUMENTATION.md §18
C4	Home search keyword dropped: `JobSearchBar` writes `?designation=`, `JobSearchPage` reads `q`	`features/jobs/components/JobSearchBar.tsx:23`; `features/jobs/pages/JobSearchPage.tsx:56`
C5	Contact form shows success but sends nothing (`setTimeout` 600 ms)	`features/public/pages/ContactPage.tsx:29-33`
C6	Resume "View" link is `<a href="/files/resume">` (no `/api`, no bearer) → SPA 404; neighbouring Download works	`features/candidates/portal/components/ProfileAsideCards.tsx:150-156`
C7	Employer registration fails with blank HR email (`''` vs `@IsOptional() @IsEmail()`)	`features/auth/pages/EmployerRegisterPage.tsx:35-45,121-125`; `apps/api/src/modules/auth/dto/auth.dto.ts:177-180`
C8	Forgot password has no failure feedback	`features/auth/pages/ForgotPasswordPage.tsx:21`
C9	Job apply reports every 400 as "already applied"	`features/jobs/pages/JobApplyPage.tsx:126-129`
C10	Bulk import shows axios' generic message instead of the API message	`employer/pages/ManageJobs/BulkImport/BulkImportPage.tsx:62`
C11	Dead links: CommandPalette → `/candidate/settings` (unregistered; also shows candidate/employer items to every role); PWA shortcut → `/candidate/applied-jobs` (commented out); CareerPage canonical `/careers` vs route `/career`; `MobileBottomNav` (unmounted) → `/candidate/messages`; `DashboardStats` (orphan) → `/candidate/applied-jobs`	`components/CommandPalette.tsx:52-64`; `apps/web/vite.config.ts:79`; `features/public/pages/CareerPage.tsx:22`
C12	Employer header bell always shows a red unread dot and opens a static page; sidebar badges hard-coded `0`	`employer/components/Header/EmployerHeader.tsx:96-98`; `employer/layouts/EmployerLayout.tsx:71`
C13	Portal unread badge counts only SSE events since header mount; never loads stored notifications	`hooks/useRealtimeNotifications.ts`
C14	Referral "Invite sent" toast but no email/SMS; `ref` query param never read by `RegisterPage`	`features/candidates/portal/pages/ReferPage.tsx:42,61-64`
C15	Logout leaves the React Query cache, offline queue and `aaj.lastOrderRef` intact (client-side navigation)	`features/auth/auth.store.ts:35-41`; `lib/queryClient.ts`
C16	Wizard and Account settings default blank gender to `'M'`	`portal/wizard/PersonalStep.tsx:116`; `portal/pages/AccountSettingsPage.tsx:113`
C17	Q2 Forward / Send back / Reject: final decisions with no confirmation and no reason (API accepts `note`)	`features/q2/pages/Q2ApplicantDetailPage.tsx:172-198`; `features/q2/q2.api.ts:122-144`
C18	Legacy `InterviewsPage` Complete/Cancel: no confirm, no `onError`; `QC1DashboardPage` and legacy `Q3DashboardPage` have no error branch (skeleton forever)	`features/recruitment/pages/InterviewsPage.tsx:58-62`; `QC1DashboardPage.tsx:17,38`; `recruitment/pages/Q3DashboardPage.tsx:21`
C19	`useAssignJob` has no invalidation	`features/recruitment/recruitment.api.ts:105-110`

5.2 Potential issues (needs runtime verification)

#	Issue	Evidence
P1	Job search array filters sent as `employmentTypes[]=…` (no `paramsSerializer`) may be rejected by `forbidNonWhitelisted` under Express 5's simple query parser; E2E mocks hide it	`lib/axios.ts`; `apps/api/src/main.ts:36-41`; `apps/api/src/modules/jobs/dto/jobs.dto.ts:61-78`
P2	Onboarding completion failure loops back to step 1 without a message	`features/candidates/portal/pages/ProfileWizardPage.tsx:75`
P3	Hard-coded `/api` prefixes bypass `env.apiBaseUrl` (logo `<img>`, Student book link)	`employer/pages/CompleteProfile/CompleteProfilePage.tsx` (~l.166); `features/public/pages/StudentGuidePage.tsx:13`
P4	N+1 load: `ApplicationsPage` issues 2 extra GETs per application, and polls every 60 s	`features/candidates/portal/pages/ApplicationsPage.tsx`

#	Issue	Evidence
P5	Employer filter dropdowns load only 100 jobs/applicants	`ApplicantListPage.tsx:82` ; `ComparePage.tsx:248`
P6	Admin inside `/company/*` gets 404s from `/clients/me/*`	`components/layout/menu.ts:95-105`
P7	`AdminSettingsPage` can PATCH defaults if the initial GET failed; empty numbers become 0	`features/admin/pages/AdminSettingsPage.tsx:130-155,297,323-335`
P8	`GET /q1/candidates/:id` mutates state (New → Screening) on any refetch or admin peek	`apps/api/src/modules/q1/q1.controller.ts:51-55`
P9	Legacy photo fallback `/files/no-image.png` may not exist	`features/recruitment/pages/CandidateDetailsPage.tsx:170`
P10	Q1/Q2 topbar quick search pushes one history entry per keystroke	`Q1Shell.tsx:190-196` ; `Q2Shell.tsx:261-264`
P11	Refresh token in localStorage is readable by any XSS	`lib/tokenStorage.ts:18-21`

5.3 Missing implementation

Area	What is missing	Evidence
Candidate	Withdraw application; request interview reschedule (only Q3 has an endpoint)	no UI, no candidate API
Candidate	Editors for diversity, personal details, accomplishments other than LinkedIn, GitHub link — only editor `/candidate/profile/advanced` is commented out	`routes/router.tsx:370-373`
Candidate	Data export processor, account purge job; consumers of privacy and email preferences	API_DOCUMENTATION.md §3
Employer	Messaging, notifications, settings/team, employer change password, job duplicate (`useDuplicateJob` unused), bulk applicant actions (`useBulkDecideApplicants` unused), branding	§3.5
Public	Public blog from CMS, company reviews, salary insights, global search, public candidate profile, OAuth	§3.1
Q1	Contact-log UI (`useQ1ContactLog` unused); direct "Update status" entry	`features/q1/q1.api.ts:58` ; `ContactCandidateModal.tsx:52-58`
Q3	Record a company rejection from Company Review; offer resend/revise/withdraw ("Declined"/"Expired" are disabled dead ends)	`Q3CompanyReviewPage.tsx` ; `Q3DocumentsPage.tsx:770-773`
Admin	Bulk delete UI (`POST /admin/users/bulk-delete`)	`admin.controller.ts:80`

5.4 Orphan code and mocked / hard-coded data (Improvement recommendation)

Item	Path	Note
Orphan components (imported by nothing)	`features/jobs/components/{SaveJobButton (wrong endpoint), CompareDrawer, AdvancedSearchModal, ApplicationQuestionnaire}.tsx` ; `features/candidates/components/{ResumeImport, JobRecommendations, DashboardStats, ProfileCompletionMeter, SavedSearches, DashboardWidgets}.tsx` ; `features/dashboard/DashboardHome.tsx` ; `features/clients/components/*` (AnalyticsDashboard, ApplicantNotesModal, BulkEmailUpload → non-existent `/clients/me/campaigns/bulk` , EmployerDashboardStats, JobQuestionsEditor, ScheduleInterviewModal, VideoInterviewSetup); `components/PushNotificationPrompt.tsx` (non-existent `/notifications/subscribe`); `components/layout/MobileBottomNav.tsx` ; `features/messaging/MessagingPage.tsx` (non-existent `/messaging/*`); `features/dev/ComponentShowcase.tsx` ; `features/misc/ErrorPage.tsx` ; `components/NotificationBell.tsx` (shells use `components/layout/NotificationBell`); AnimatedCounter, KeyboardShortcutsHelp, OptimizedImage, PageSkeleton, QueryErrorBoundary, WhatsAppShare *(grep-based, inferred)*	Remove or wire up
Commented-out pages still in tree	`features/candidates/pages/{CandidateProfile, CvManager, AppliedJobs, ResumePreview, JobAlerts, ChangePassword, AssessmentsList, SkillAssessment, SavedSearches, SkillGap, InterviewCalendar, ActivityTimeline, CareerPath, CompanyComparison}Page.tsx` — several call non-existent endpoints (`/candidates/me/assessments` , `/skill-gap` , `/career-path` , `/interviews/calendar` , PATCH saved-searches)	Unreachable
Deprecated re-exports	`features/clients/client.api.ts` , `client.types.ts`	—
Aspirational types	`types/api.ts` (used only by `test-utils/apiMocks.ts` ; shapes differ from the API)	—
Hard-coded data	`features/public/blogs.data.ts` , `pricing.data.ts` , `services.data.ts` , PlanComparison, testimonials, About/Help/Legal; profile "Profile Views" tile fixed "—" (`ProfileOverviewPage.tsx:269`); "Last 30 days" caption *(dashboard endpoint not date-filtered — inferred)*	`PricingPlans` itself is live
Disabled placeholders	Google/LinkedIn login; Account 2FA and Connected Accounts toggles	—
MSW fixtures	`mocks/handlers.ts` (713 lines), `mocks/data.ts` (1201 lines) — dev/CI only	Only with `VITE_USE_MOCKS=1`

6. Duplicate screens: legacy `/recruitment/*` vs `/q1` `/q2` `/q3`

Canonical staff UI: `/q1/*`, `/q2/*`, `/q3/*` and `/admin/*` (each role's `ROLE_HOME` points there). `/recruitment/*` is legacy; router comments say it is "untouched and still reachable" (`routes/router.tsx:261, 282, 303`).

Function	Canonical	Legacy duplicate	Still linked from
QC1 dashboard / queue	`/q1/dashboard` , `/q1/candidates`	`/recruitment/qc1` , `/recruitment/candidates`	QC1 legacy menu links `/q1/dashboard` , `/q1/candidates` **and** `/recruitment/documents` , `/recruitment/interviews` (`menu.ts:82–86`)
Candidate review / decision	`/q1/candidates/:id` (screening state in `tblSubscriberScreening`)	`/recruitment/candidates/:id` (registration decision)	QC2 legacy menu `/recruitment/candidates` (`menu.ts:88–91`). Two state models, not reconciled in the UI *(inferred)*
Refer to Q3	`/q2/applications/:mapId` → POST `/q2/applications/:mapId/forward`	`CandidateDetailsPage` "Refer to Q3" → POST `/recruitment/referrals`	Both reachable; same referral pipeline *(inferred)*
Q2 referral tracking	`/q2/forwarded`	`/recruitment/referrals`	QC2 legacy menu `/recruitment/referrals` ; **the QC2 legacy menu has no `/q2` link**
Q3 dashboard / forwarding	`/q3/dashboard` , `/q3/referred`	`/recruitment/q3` (forward + `InterviewRoundsPanel`)	Q3 legacy menu (`menu.ts:107–111`) links only `/recruitment/q3/*` , three of which 403 (C2); **no `/q3` link**
Interview rounds / results	`/q3/scheduling` , `/q3/interviews` , `/q3/decisions`	`InterviewRoundsPanel` and `/recruitment/interviews` (single-interview model)	QC1 legacy menu `/recruitment/interviews`
Document review	`/q3/documents` (per application)	`/recruitment/documents` , `/recruitment/q3/documents`	QC1, QC2, Q3 legacy menus
Offer / joined	`/q3/offers` (POST `/q3/.../offer` , `/joined`)	`/recruitment/offers*` (used by the employer portal, not by a legacy staff page)	employer `ApplicantProfilePage`

The legacy menu is shown only inside `DashboardLayout` ; the new shells have their own sidebars. A staff user who lands on any `/recruitment/*` page sees the legacy menu.

---

7. Mobile responsiveness (static evidence only)

> Read from Tailwind classes and handlers. **Needs runtime verification** on real devices; only the admin shell and Q2 have automated viewport checks (see TESTING_AND_ISSUES.md).

Shell / area	Navigation below breakpoint	Tables / content	Gaps seen in code
Public `Navbar` (`components/layout/Navbar.tsx:97–117`)	Below `md` , all links collapse into one "Menu" dropdown; desktop pills `hidden md:block`	—	No signed-in state in the navbar *(inferred)*
Candidate `PortalHeader`	Links `hidden lg:flex` ; hamburger `lg:hidden` toggles a disclosure list; wordmark `sr-only` below 374 px	Portal stacks sidebar over content below `lg` (`flex-col lg:flex-row` , sidebar `w-full lg:w-80`); `ModuleTiles` `grid-cols-2 sm:grid-cols-3 lg:grid-cols-2` ; wizard `StepDots` hidden below `sm`	`MobileBottomNav` exists but is not mounted
`DashboardLayout` (legacy)	Hamburger `lg:hidden` ; `Sidebar` fixed drawer `max-w-[85vw]` , overlay; closes on navigation (`DashboardLayout.tsx:67`)	Shared `components/ui/Table.tsx` `min-w-[640px]` + horizontal scroll; Membership history `overflow-x-auto`	—
`EmployerLayout`	`EmployerSidebar` fixed drawer (`-translate-x-full` / `lg:translate-x-0`), overlay, closes on route change and Escape (`EmployerLayout.tsx:24–34`); collapse persisted in `aaj.employer.sidebar.collapsed`	`employer/components/Tables/DataTable.tsx:34` `overflow-x-auto` + column-visibility menu (no card view)	Few breakpoint classes on employer pages; dense tables likely scroll sideways *(inferred)*
`Q1Shell`	Sidebar `hidden lg:block` ; hamburger drawer with backdrop, closes on link/popstate	`CandidateQueue` switches to stacked cards below `md`	Quick search hidden below `md` ; drawer lacks Escape, focus trap, scroll lock

Shell / area	Navigation below breakpoint	Tables / content	Gaps seen in code
`Q2Shell`	Same drawer; desktop collapse toggle persisted	`ApplicantsTable` / `EmployerJobsTable` `overflow-x-auto` + `min-w-[58rem]` / `[54rem]`	Quick search hidden below `md`
`Q3Shell`	Drawer closes on pathname change	Tables with `min-w` 714–1126 px inside `overflow-x-auto` (`Q3ReturnedPage.tsx:109` , `Q3ReferredCvsPage.tsx:197` , `Q3DocumentsPage.tsx:234,338` , `Q3DashboardPage.tsx:199` , `Q3InterviewsPage.tsx:380` , `Q3CompanyReviewPage.tsx:111`)	Search hidden below `md` ; no Escape on drawer; cramped on phones
`AdminShell`	`role=dialog aria-modal` drawer with Escape, body scroll lock, closes on route change; desktop collapse persisted	Admin `DataTable` (`features/admin/components/primitives.tsx:147-253`) renders a card list below `md`	Best of the shells

Testing Status & Issues

> Analysis of commit `8de48cf` (branch `main`). No production code was changed to produce this document.
>
> **Classification** used for every finding:
>
> - **Confirmed issue**: the defect follows directly from code that was read.
> - **Potential issue**: plausible from the code, but needs a runtime check.
> - **Missing implementation**: a feature the UI, schema or comments imply, but the code does not do.
> - **Improvement recommendation**: not a bug; a change that would improve the code.
>
> **Verification marks**: ✔ = re-checked line by line during the final review of this document; all other findings come from the module-by-module code trace, with the file:line evidence shown.

1. What was actually run

Check	Command	Result (2026-10-27)
Type check (web + api)	`npm run typecheck`	**Pass**: no errors
Lint (web + api, `--max-warnings 0`)	`npm run lint`	**Pass**
API unit tests	`cd apps/api && npm test` (node:test + tsx, mocked Prisma)	**189 / 189 pass**, 60 suites
Web unit tests	`cd apps/web && npx vitest run`	**539 / 539 pass**, 67 files
Playwright mock suite	`apps/web: npm run e2e` (VITE_USE_MOCKS=1)	**Not run**
Real-API end-to-end	`node tests/e2e.mjs`, `node tests/admin-e2e.mjs`	**Not run**: both seed and mutate a live database; deliberately skipped
BillDesk stub	`node tests/billdesk-stub.mjs`	**Not run** (and incompatible with the current webhook route; see P-02)

Passing unit tests do **not** demonstrate that the workflows in this document work end to end. They run against mocked Prisma and mocked HTTP. None of the workflows were exercised against a running API in this analysis.

2. Test estate

Layer	Location	Size	What it covers
API unit	`apps/api/src/**/*.test.ts`	16 files	auth (login, refresh rotation, OTP, register, reset), admin service (users, employer approval, job moderation, settings), Q1 completeness/priority, jobs search, education validation, key skills, avatar, site docs, SSE fan-out, hiring/matching helpers, PDF page count, one Q3 document-scope test, CV expiry
Web unit	`apps/web/src/**/*.test.{ts,tsx}` (Vitest + jsdom)	64–67 files	UI primitives, auth store/types, ProtectedRoute, Q1 rail/modal, Q2 detail page, formatters, hooks, lib helpers
Playwright (mocked API)	`apps/web/e2e/*.spec.ts`	43 specs, ~300 tests	Q1/Q2 workspaces, legacy recruitment screens, admin dashboard + approvals, candidate flows, employer registration/job post, public pages, a11y, PWA, responsive
Real-API crawl	`tests/e2e.mjs`	115 lines	Every public page and each of 6 roles' pages; fails on console errors, API ≥400, 404 page
Real-API admin suite	`tests/admin-e2e.mjs`	404 lines	Admin access control, user/job/employer actions against the DB, audit, responsive widths 320–1440 px
CI	`.github/workflows/ci.yml`	5 jobs	static (typecheck, lint, unit, build), e2e-mock, e2e (real Postgres + Redis), docker build, deploy (main → EC2 over SSH)

2.1 Critical paths with no automated tests

#	Path	Why it matters
T-1	Payments: Razorpay order/verify/webhook, BillDesk settle, idempotency, amount check	Money; the BillDesk settle path is already unreachable (P-02)

#	Path	Why it matters
T-2	Employer job lifecycle and applicant decisions (`employers.service.ts` , 2.7k lines, no tests)	Core employer workflow
T-3	`q2.service.ts` transitions (forward, send back, reject)	Pipeline correctness
T-4	Q3 services: round setup, confirm, decide, offer, joined, return/expire	Pipeline correctness; the two decision engines disagree (W-01)
T-5	File access control (`/files/*` , `/recruitment/*/resume` , `/recruitment/documents/:id/file`)	Personal data exposure (S-01)
T-6	Role matrix per endpoint (no test enumerates `@Roles`)	Would have caught the Q3 legacy 403s (F-02)
T-7	axios refresh / CSRF / 429 interceptors	Session handling
T-8	Email/BullMQ delivery and SSE auth	Notifications
T-9	`/q3/*` workspace and admin user/job/settings/blog/reports/audit actions in the Playwright mock suite	New screens untested in-browser except via the real-API crawl

2.2 Test-quality findings

- **Confirmed issue**: two "API contract" specs only assert their own mocks (`apps/web/e2e/offer-letter.spec.ts:18-64, 90-111` ; `video-meeting-link.spec.ts:30-51`).
- **Confirmed issue**: CI runs the web unit tests twice and lints twice, the second time with a looser threshold (`.github/workflows/ci.yml` , static job).
- **Confirmed issue**: there are duplicate unit-test files for Alert, Button, Input, Pagination, `cn` and `tokenStorage` , and two Vitest configs (`apps/web/vitest.config.ts` and a dead `test` block in `apps/web/vite.config.ts`).
- **Potential issue**: `tests/e2e.mjs` waits with `waitUntil: 'networkidle'` while the SSE stream is open. Each navigation probably runs to the timeout. Project memory also says to avoid networkidle.
- **Potential issue**: `tests/e2e.mjs` skips the `/recruitment/q3/*` pages, which is why F-02 is not caught.
- **Potential issue**: the Playwright suite mocks the API with `page.route` , so request-shape bugs such as array query params (F-04) are invisible to it.

---

3. Findings by severity

3.1 Security & personal-data exposure

ID	Class	Finding	Evidence
S-01 ✔	Confirmed issue	**Any signed-in user can download any stored file by key.** `GET /api/files/:folder/:sub/:name` has no `@Roles` and no ownership check. Keys reach employers (applicant `documentPath`), Q2 (`latestCVPath`), upload responses and audit rows.	`apps/api/src/modules/files/files.controller.ts:63-77` ; `employers.service.ts:2494` ; `q2.service.ts:851`
S-02	Confirmed issue	**Arbitrary file exposure through the public logo endpoint.** `PATCH /clients/me` accepts any string as `companyLogo` , and the registration's free-text logo is copied on approval. `GET /api/clients/:id/logo` is public and streams that storage key.	`employers/dto/employers.dto.ts:599-602` ; `employers.service.ts:409-428, 444` ; `admin.service.ts:1345`
S-03	Confirmed issue	**Employer document access is scoped to the candidate, not the application.** Company A sees, and can approve or flag, documents collected for company B's application.	`employers.service.ts:2483-2505, 2536-2540`
S-04	Confirmed issue	**Unescaped user input in HTML emails** (welcome: fullName; application-status: fullName, message; interview-scheduled: location (also used as an `href`), interviewer, notes, job title, company).	`apps/api/src/common/email/templates/welcome.template.ts:12` , `application-status.template.ts:61` , `interview-scheduled.template.ts:35-60`
S-05	Confirmed issue	**Access tokens are not revocable.** The guard trusts the token's `roleId` for its TTL (default 15 min) and ignores logout, deactivation and role changes.	`apps/api/src/common/guards/jwt-auth.guard.ts:37-52`
S-06	Confirmed issue	**The CSRF token is issued and sent, but never validated** (bearer auth makes it largely moot, so it is dead weight).	`apps/api/src/modules/auth/csrf.controller.ts` ; `apps/web/src/lib/axios.ts:38-44`
S-07	Potential issue	**Unscoped access to identity documents.** QC1/QC2 can list and download every candidate's identity documents (Aadhaar, PAN, bank) and verify or reject them. Q3's equivalent is scoped.	`recruitment.controller.ts:103-128` ; contrast `q3-documents.service.ts:78-84`
S-08	Potential issue	**Q3 can act on any application through `/recruitment/*` .** It can create rounds, results, offers and joined status on applications never referred to it (`MappingAccessService` checks Candidate and Client only).	`apps/api/src/modules/recruitment/mapping-access.service.ts:24-69`

ID	Class	Finding	Evidence
S-09	Potential issue	**Employers see and can act on applicants before the QC1→QC2→Q3 pipeline releases them.** This includes full contact details and the ability to shortlist, hire or interview; undo can also reverse a staff rejection. Needs a product decision.	`employers.service.ts:734-777, 1297-1401`
S-10	Confirmed issue	**The employer approval email contains the temporary password in plain text**, and nothing forces it to be changed (`pwdStatus` is never read; employers and staff have no change-password endpoint).	`admin.service.ts:1384-1394` ; `auth.service.ts` (no pwdStatus read)
S-11	Confirmed issue	**Logout does not clear the React Query cache, offline queue or last order ref.** A second user on the same tab can briefly see the previous user's cached data.	`apps/web/src/features/auth/auth.store.ts:35-41` ; no `queryClient.clear()` anywhere
S-12	Improvement recommendation	**Session and secret handling.** The refresh token is in localStorage. The log email driver writes OTPs and reset links to the logs. The 2Factor key goes in the URL path. Login lockout is per-IP only (5/min). Draft, Closed and Archived jobs are publicly readable by id. The public avatar route allows enumerating photos by sequential id.	`apps/web/src/lib/tokenStorage.ts:18` ; `notifications/providers/log.provider.ts:19` ; `twofactor.provider.ts:15` ; `jobs.service.ts:508-521` ; `files.controller.ts:151-170`
S-13	Confirmed issue	**A fallback SQL Server SA password is committed in source** (value intentionally not reproduced).	`apps/api/prisma/migrate-data.ts:32`

3.2 Broken or incomplete workflows

ID	Class	Finding	Evidence
W-01	Confirmed issue	**Two round-result engines disagree.** `InterviewRoundService.submitResult` (employer portal and `/recruitment`) never sets `decision` , `isFinal` , `selectedAt` or `conductedAt` , which Q3's stage logic relies on. After an employer "Hire", Q3 still shows "Decision" and accepts a contradicting second decision, and the documents/offer flow never starts. Hold → ON_HOLD (employer) vs no status change (Q3). A recruitment "final round" (roundNumber 4, isFinal=false) can never be Selected by Q3.	`recruitment/interview-round.service.ts:299-415` ; `q3/q3-interviews.service.ts:580-660` ; `shared/hiring.ts:331`
W-02	Confirmed issue	**`transitionStatus` has no from-state guard.** `PATCH /recruitment/pipeline/:id` sets any status with no log, history or audit, and QC1 can call it (no web caller). `selectSlot` , `addSlots` , `submitResult` and `sendOffer` lack status preconditions; for example, a candidate can re-select a slot on a completed round, and a resent offer reopens an accepted one.	`jobs/job-application.service.ts:167` ; `recruitment.service.ts:503` ; `interview-round.service.ts:153-229, 299` ; `offer-letter.service.ts:97`
W-03	Confirmed issue	**Q1 → Q2 hand-offs are not enforced.** Q1 "Verified" gates nothing (Q2 lists every application on active jobs), and a Q2 "send back" is a dead end: nothing returns the application to Q2.	`q2.service.ts:595-648, 975-987` ; `q1.service.ts:485`
W-04	Confirmed issue	**Company rejection marks the referral `Returned` without `returnedAt` .** The Q3 badge counts it while the Returned list omits it, and the CV can be re-referred.	`cv-referral.service.ts:441-449` ; `employers.service.ts:1359-1365` ; `q3-referrals.service.ts:392` ; `q3-read.service.ts:582-585`
W-05	Confirmed issue	**An auto-expired CV is back in Q2's queue, but its status stays 23 CV_EXPIRED** (shown as Closed by admin). A manual return moves it to 1 MAPPED.	`cv-referral.service.ts:216-300`
W-06	Confirmed issue	**The two "mark joined" paths disagree.** `/recruitment/offers/:id/mark-joined` does not set `ApplicationHiring.joinedAt` , so Q3 keeps the application in the Offer stage and the quarterly report misses it.	`offer-letter.service.ts:190` ; `q3-documents.service.ts:429`
W-07	Confirmed issue	**Shortlisting gives different statuses by path**: Q3 round-setup → 2 SHORTLISTED; employer interview request or recruitment company-review → 16 INTERVIEW_R1.	`q3-referrals.service.ts:365` ; `interview-round.service.ts`
W-08	Missing implementation	**There is no application withdrawal.** Statuses 8 RESCHEDULED, 12 DUPLICATE and 13 WITHDRAWN are never written, and the candidate has no withdraw or reschedule UI.	grep: no writer
W-09	Confirmed issue	**An offer rejection leaves the application at 20 OFFER_SENT** (there is no closed status). Offer creation checks *all* of the candidate's uploads, across applications.	`offer-letter.service.ts:38-47, 153-159`
W-10	Confirmed issue	**Document-request de-duplication is per candidate, not per application**, so company A's document approval may never complete when company B already requested the same type. Document status 3 (NeedsCorrection) is shown as "Pending".	`employers.service.ts:2663-2668, 2495, 2542`
P-01	Confirmed issue	**Razorpay checkout fails in the Docker deploy.** No compose file forwards `RAZORPAY_KEY_ID` , `RAZORPAY_KEY_SECRET` or `RAZORPAY_WEBHOOK_SECRET` , and `.env.example` omits them, so the API answers 400 "not configured".	`docker-compose.yml` , `docker-compose.prod.yml` ; `.env.example`
P-02	Confirmed issue	**BillDesk orders can never settle.** `PaymentsService.settle()` has no route, and `/api/payments/webhook` is now the Razorpay handler, so the BillDesk stub's callback gets 400. `POST /payments/orders` still creates BillDesk orders.	`payments.service.ts:400-492` ; `payments.controller.ts:62-76` ; `tests/billdesk-stub.mjs`

ID	Class	Finding	Evidence
P-03	Potential issue	**A concurrent verify and webhook can make one request 500** (unique `Subscription.orderID`). A Razorpay create-order failure returns 500 and leaves a CREATED order behind. The Razorpay HMAC comparison is not constant-time.	`payments.service.ts:128-148, 233-254`; `razorpay.service.ts:60, 82, 98`
P-04	Missing implementation	**Payment has no further effect.** It only adds a `tblSubscription` row (gating self-apply): there is no receipt email, the role 6 "Subscription" is never assigned (and has no `tblSecRoles` row), the plan tier is not tied to experience, and employers cannot pay invoices online.	`payments.service.ts`; `db/seed/tblSecRoles.psv`
F-01 ✔	Confirmed issue	**Employer registration fails when HR email is blank.** The form sends `emailHR: ''`, and `@IsOptional() @IsEmail()` rejects empty strings (400). The e2e fixture always fills it.	`apps/web/src/features/auth/pages/EmployerRegisterPage.tsx:38, 99`; `apps/api/src/modules/auth/dto/auth.dto.ts:178-180`
F-02 ✔	Confirmed issue	**The legacy Q3 screens 403.** `/recruitment/q3/candidates`, `/interviews` and `/documents` are in the Q3 sidebar, but the endpoints behind them are `@Roles(QC1, QC2, Admin)`.	`apps/web/src/components/layout/menu.ts:109-111`; `recruitment.controller.ts:36`
F-03 ✔	Confirmed issue	**Q1/Q2 "View/Download CV" fails with 401.** It opens `/api/recruitment/candidates/:id/resume` with `window.open` or `<a href>`, which sends no bearer header.	`apps/web/src/features/q1/pages/Q1CandidateProfilePage.tsx:138, 144, 277, 284`; `apps/web/src/features/q2/pages/Q2ApplicantDetailPage.tsx:288-296`
F-04	Potential issue	**Job search filters may 400 against the real API.** axios serialises arrays as `employmentTypes[]=…`, Express 5's default "simple" query parser keeps that key, and `forbidNonWhitelisted` rejects it. The mocked e2e cannot show this.	`apps/web/src/lib/axios.ts` (no paramsSerializer); `apps/api/src/main.ts:36-41`; `jobs/dto/jobs.dto.ts:61-78`
F-05	Confirmed issue	**The web app calls non-existent endpoints**: `/candidates/:id/profile` (the target of every "Share Profile"), `/companies/:id/reviews`, `/clients/:id/reviews`, `/jobs/salary-insights`, `/search`.	see API_DOCUMENTATION §18
F-06	Confirmed issue	**The home search keyword is dropped**: `JobSearchBar` sets `?designation=` while `JobSearchPage` reads `?q=`.	`features/jobs/components/JobSearchBar.tsx:23`; `JobSearchPage.tsx:56`
F-07	Confirmed issue	**Notification preferences: PUT saves only 4 of 12 fields**, so the per-topic toggles revert on reload.	`candidates.service.ts:1523-1545` ✔
F-08	Confirmed issue	**Fake or cosmetic success.** The Contact form only simulates a delay. Referral "Invite sent" sends nothing, and `?ref=` is never read. Data export promises an emailed archive that is never produced. The resume "View" link points to `/files/resume` (SPA 404).	`features/public/pages/ContactPage.tsx:29-33`; `ReferPage.tsx:42, 61-64`; `ProfileAsideCards.tsx:150-156`
F-09	Confirmed issue	**Misleading or missing error handling.** Job apply shows "already applied" for every 400. Forgot-password has no error branch. Legacy interview Complete/Cancel fail silently. The legacy QC1 dashboard shows a skeleton forever on error. The bulk import shows a generic axios message.	`JobApplyPage.tsx:126-129`; `ForgotPasswordPage.tsx:21`; `recruitment/pages/InterviewsPage.tsx:58-62`; `QC1DashboardPage.tsx:17, 38`; `BulkImportPage.tsx:62`
F-10	Confirmed issue	**Q2 Reject / Send back / Forward have no confirmation or reason capture**, and the decisions are final.	`Q2ApplicantDetailPage.tsx:172-198`
F-11	Confirmed issue	**Dead links**: CommandPalette `/candidate/settings`; PWA shortcut `/candidate/applied-jobs`; Career canonical `/careers`; the welcome email's link to `/profile`.	`components/CommandPalette.tsx:59`; `apps/web/vite.config.ts:79`; `CareerPage.tsx:22`; `auth.service.ts:498`
F-12	Confirmed issue	**Wrong-default data.** The wizard defaults gender to 'M' when left blank. The employer bell always shows an unread dot. The portal unread badge ignores stored notifications.	`PersonalStep.tsx:116`; `EmployerHeader.tsx:96-98`; `hooks/useRealtimeNotifications.ts`
A-01 ✔	Confirmed issue	**Staff cannot log in by email**, only by username. Staff logins are written with NodeType 100, which the email-login path excludes.	`admin/staff-account.ts:40, 56`; `prisma/seed.ts:269, 288`; `auth.service.ts:197`
A-02	Confirmed issue	**Migrated legacy candidates cannot use the portal.** `SubscriberID` is null, so every `/candidates/me*` call returns 404 and they are stuck in onboarding.	`candidates.service.ts:138-146`
A-03	Confirmed issue	**Admin is listed on the employer routes and the company-review route but always gets 404** (it has no linked company).	`employers.service.ts:354-367`; `recruitment.service.ts:539-552`
A-04	Confirmed issue	**Platform settings are stored but read by nothing** (maintenance mode, registration open, limits, SMTP fields). The settings page can also save defaults over stored values if loading fails.	`admin.service.ts:1097-1116`; `features/admin/pages/AdminSettingsPage.tsx:130-147`

3.3 Validation & data consistency

ID	Class	Finding	Evidence
D-01	Potential issue	**No unique (JobID, SubscriberID) constraint.** The duplicate-apply check runs outside the transaction, so a double submit can create two applications. CvReferral and InterviewRound (mapId, roundNumber) also lack uniqueness.	`job-application.service.ts:70-78` ; `schema.prisma`
D-02	Confirmed issue	**Schema ↔ migration drift.** `tblClientInvoice` has a migration FK not declared in the schema; two indexes and the Institute FK actions exist only in migrations. `prisma migrate dev` would try to drop them.	`apps/api/prisma/migrations/20261005130000_client_invoices` , `…q2_matching_workspace` ; `schema.prisma:1646-1664`
D-03	Confirmed issue	**Missing range checks.** Job CTC and experience ranges are never checked (min ≤ max), and the bulk upload bypasses the CTC validators. Company profile fields lack IsEmail and MaxLength (oversized input → 500).	`employers.service.ts:679, 1855-1857` ; `employers.dto.ts:543`
D-04	Confirmed issue	**Prisma errors are not mapped**: FK, unique and not-found violations surface as 500 rather than 400/404/409.	`common/filters/http-exception.filter.ts:68-75`
D-05	Confirmed issue	**Many multi-step transitions are not atomic.** Examples are referToQ3, forwardToCompany, companyReview, createRound, submitResult, Q3 decide, generateOffer, employer decisions and profile updates. They are race-safe where conditional claims are used, but partial failures leave inconsistent state.	see DATABASE_SCHEMA "Data lifecycle"
D-06	Confirmed issue	**Audit gaps.** `cv.*` audits use entityId 0. There is no audit on job create/update/duplicate/bulk, change-password, auto-expiry, the Q3 document actions or the recruitment document review. The admin application view reads audit only for entity `JobSubscriberMapping` , so it shows almost no pipeline audit. Q3-path transitions write no candidate history.	`cv-referral.service.ts:90-96, 136-142, 197-203` ; `admin.service.ts:918`
D-07	Confirmed issue	**Query-building collisions.** In job search, `workMode` / `workModes` and `location` / `locations` / `states` overwrite each other, and full-text filters are case-sensitive. In the Q1 queue, the New tab ignores search, priority overrides the tab, and `?cvOnly=false` is read as true. The employer applicants export has only 10 rows, and the admin jobs export ignores `status` / `company` . The job detail shows the interview process as raw JSON.	`jobs.service.ts:136-148, 281-303, 554-557` ; `q1.service.ts:245-278` ; `q1/dto/q1.dto.ts:63` ; `exports.controller.ts:213` ; `exports.service.ts:168-192`
D-08	Potential issue	**Scoring compares ids from different masters** (subFunction vs designation, degree vs educationType) and uses current CTC as "expected", and Jaccard penalises extra skills.	`recruitment/scoring.service.ts:47-48, 194-207`
D-09	Potential issue	**Time zones are mixed.** Q3 uses IST explicitly; recruitment emails, Q1 "today", Q1 analytics and admin signups use server UTC.	`recruitment.service.ts:285-286` ; `q1.service.ts:197-198`
D-10	Potential issue	**`migrate-data.ts` is destructive and its inputs are stale.** It TRUNCATEs every table (including payments and audit) without a guard, and its `db/foreign-keys.psv` still holds stale education edges that would NULL valid data.	`apps/api/prisma/migrate-data.ts:125-173` ; `db/foreign-keys.psv`

3.4 Missing implementations (feature-level)

Area	Missing
Notifications	No code creates in-app/SSE notifications; SMS is never sent; job-alert and payment-receipt emails have no caller; the in-memory store breaks the "scale api=3" claim in `deploy/DEPLOYMENT.md`
Candidate	Withdraw application, reschedule request, advanced profile editor (commented out, so completion is capped below 100%), data export / deletion processing, privacy enforcement, referral invites and crediting, CV parsing, public profile
Employer	Team / sub-users, change password, messaging, notifications, settings pages (placeholders), online invoice payment, real meeting links (placeholder URL), branding UI
Staff	Q1 contact log UI; Q3 can't record a company rejection; Q3 offers have no resend/revise/withdraw; Q1 contact actions do not actually message the candidate
Platform	Settings enforcement, public blog from the CMS (no public endpoint), company reviews, salary insights, global search, OAuth (buttons disabled), file deletion from storage, invoice creation (invoices are only read)

3.5 Duplicate / conflicting business logic

Topic	Implementations	Recommendation
Round results	`InterviewRoundService.submitResult` vs `Q3InterviewsService.decide`	One decision service that writes all fields both read models need
Interview models	legacy `tblJobInterviewStatus` vs `tblInterviewRound`	Retire the legacy path
Document review	recruitment review (1/2, no history), Q3 verify/send-back (1/2 + history), employer review (1/3 + history + hand-off)	One vocabulary, one writer
Refer to Q3	`/recruitment/referrals` vs `/q2/applications/:mapId/forward`	Route all through Q2

Topic	Implementations	Recommendation
Company review	employer decision / interview request vs `/recruitment/referrals/:id/company-review` vs Q3 round-setup	Single path
Offer & joined	`/recruitment/offers*` vs `/q3/applications/:mapId/offer` and `/joined`	Single path that sets `joinedAt`
Candidate decision	Q1 screening (`tblSubscriberScreening`) vs legacy registration decision (`flgstatus`)	Pick one
Scoring weights & completeness	`scoring.service.ts` vs `shared/matching.ts` ; three completeness loaders (Q1, Q1 analytics, Q2)	Share one implementation
Staff UI	`/q1` , `/q2` , `/q3` (canonical) vs legacy `/recruitment/*` (still linked)	Redirect or remove the legacy screens
CSV writers	exports module (with formula-injection guard) vs two employer CSVs (without)	Reuse the guarded writer

3.6 Infra & configuration

ID	Class	Finding
I-01	Confirmed issue	The base `docker-compose.yml` never forwards `EMAIL_DRIVER` / `SMS_DRIVER` , so plain `docker compose up` logs emails instead of sending them, even with SMTP set. Registration OTP therefore cannot work on that path
I-02	Potential issue	Deploys build on the EC2 host and auto-apply migrations on start with no backup step or rollback
I-03	Improvement recommendation	nginx listens on :80 only with no HSTS/CSP; no explicit no-cache for the PWA service worker; Redis has no password (internal network only)
I-04	Improvement recommendation	`apps/api/.env.example` lists stale SQL Server `DB_*` variables and omits `RAZORPAY_*` , `THROTTLE_LIMIT_PER_MIN` , `STORAGE_*`
I-05	Improvement recommendation	`/api/health` always returns HTTP 200 even when degraded, and checks disk at `/` rather than the storage root

3.7 Performance (improvement recommendations)

- Employer applicants, Q2 applicants and analytics load every row and paginate in memory.
- Q3 `loadApps` loads every referred application on each dashboard, nav-count or report call.
- Q1 analytics issues about 3,000 queries.
- Full-text search computes `to_tsvector` per row with no expression index.
- Q2 and Q1 GET endpoints perform writes (score backfill, review rows, status changes).
- Hot FK columns lack indexes (`tblSecActiveSessions.SessionID` , `tblPaymentOrder.BDOrderID` , document tables).

3.8 Mobile responsiveness (static evidence only; runtime check needed)

- **Verified at runtime by the existing suite**: `tests/admin-e2e.mjs` checks admin pages for horizontal overflow at 320–1440 px. The Q2 Playwright spec covers the mobile drawer.
- **Static evidence**:
  - Every shell (public navbar, candidate portal, DashboardLayout, employer, Q1/Q2/Q3, admin) has a hamburger or drawer below `lg` . Only AdminShell's drawer has Escape handling, focus trap and scroll lock.
  - Q1 and admin tables switch to cards on phones. Q2 and Q3 tables scroll horizontally (min-width 714–1126 px), as do employer tables.
  - The quick search in the Q1/Q2/Q3 top bars is hidden below `md` .
  - `MobileBottomNav` exists but is not mounted.

---

4. Requires runtime verification

1. Job-search filter query arrays against the real API (F-04).
2. Razorpay end-to-end with real keys, including webhook delivery through nginx and the verify/webhook race (P-01, P-03).
3. A BillDesk sandbox run, if BillDesk is kept (P-02; README already flags this).
4. Email delivery with `EMAIL_DRIVER=smtp` on each compose path (I-01).
5. SSE behaviour through nginx (60 s `proxy_read_timeout` vs 30 s heartbeat) and with more than one API replica.
6. Q1/Q2 CV open/download (F-03) and the legacy Q3 screens (F-02) in a browser.
7. Prisma-error → 500 cases (D-04) and FK failures on bad master ids.
8. Concurrent apply / Q2 forward duplicates (D-01).
9. Mobile layouts of the Q2, Q3 and employer tables on real devices.
10. A full pipeline walk-through (apply → Q2 → Q3 → company → rounds → documents → offer → joined) mixing employer and Q3 actions, to observe W-01/W-06/W-07 end to end.

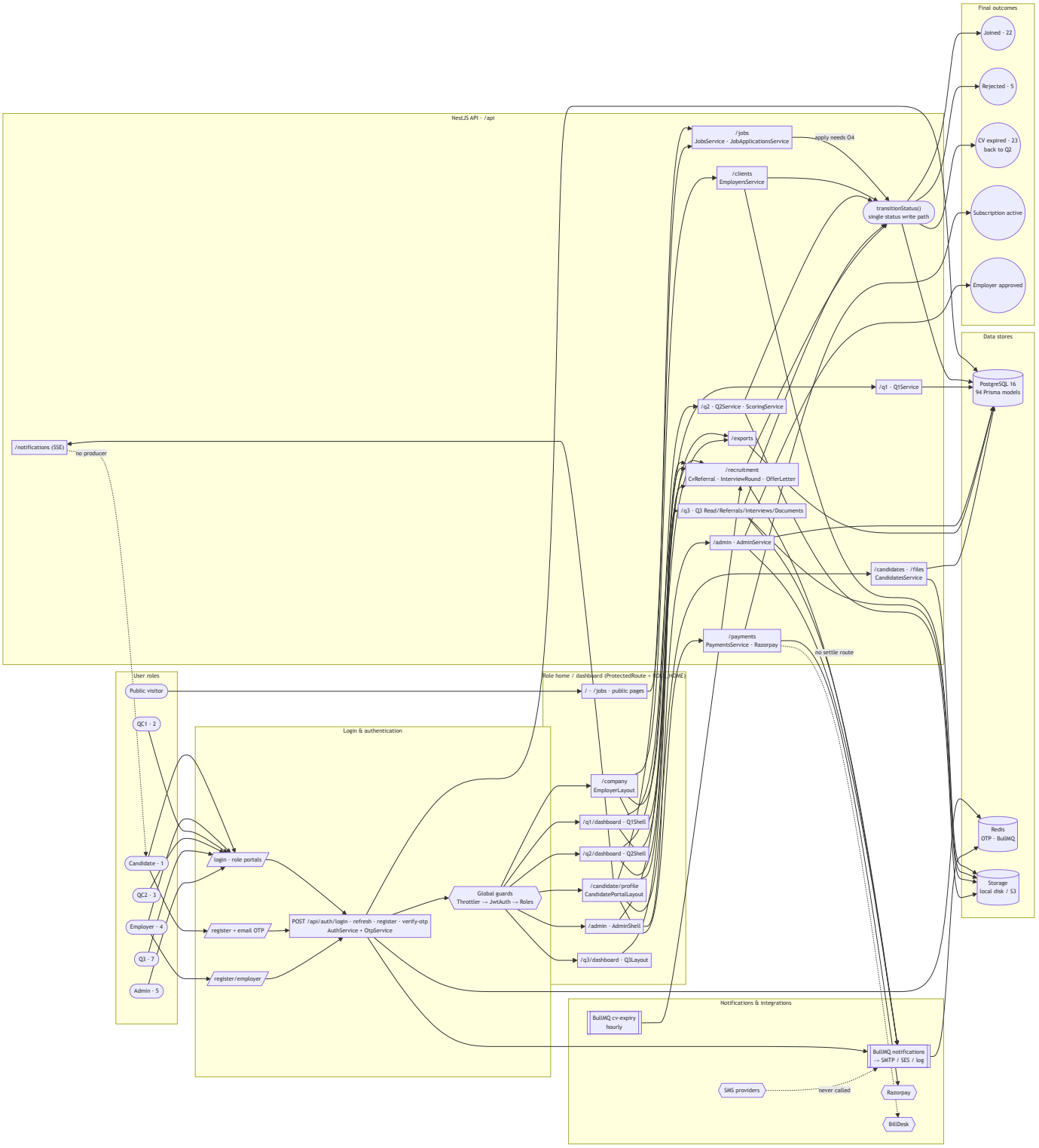
5. Suggested fix order

1. **Data exposure**: S-01, S-02, S-03, S-04 (restrict the generic file route, stop client-supplied logo keys, scope employer documents by application, escape email templates).
2. **Payments**: P-01 (forward the Razorpay env vars) and P-02 (remove or route BillDesk).
3. **Pipeline integrity**: W-01/W-06/W-07 (one decision path), W-02 (from-state guards; delete `PATCH /recruitment/pipeline/:id`), W-03 (enforce or document hand-offs).
4. **Visible breakages**: F-01, F-02, F-03, F-04, F-05, F-06, F-07, A-01.
5. **Tests**: add API tests for payments, the employer lifecycle, the Q2/Q3 services and a role-matrix test, then fix the CI duplication.

Master System Flow

> One diagram that connects roles, login, dashboards, modules, APIs, services, data stores, notifications and outcomes, followed by focused diagrams for the complex parts. Everything shown exists in code at commit `8de48cf`. Dashed lines mark paths that exist but are broken or unused (see TESTING_AND_ISSUES).

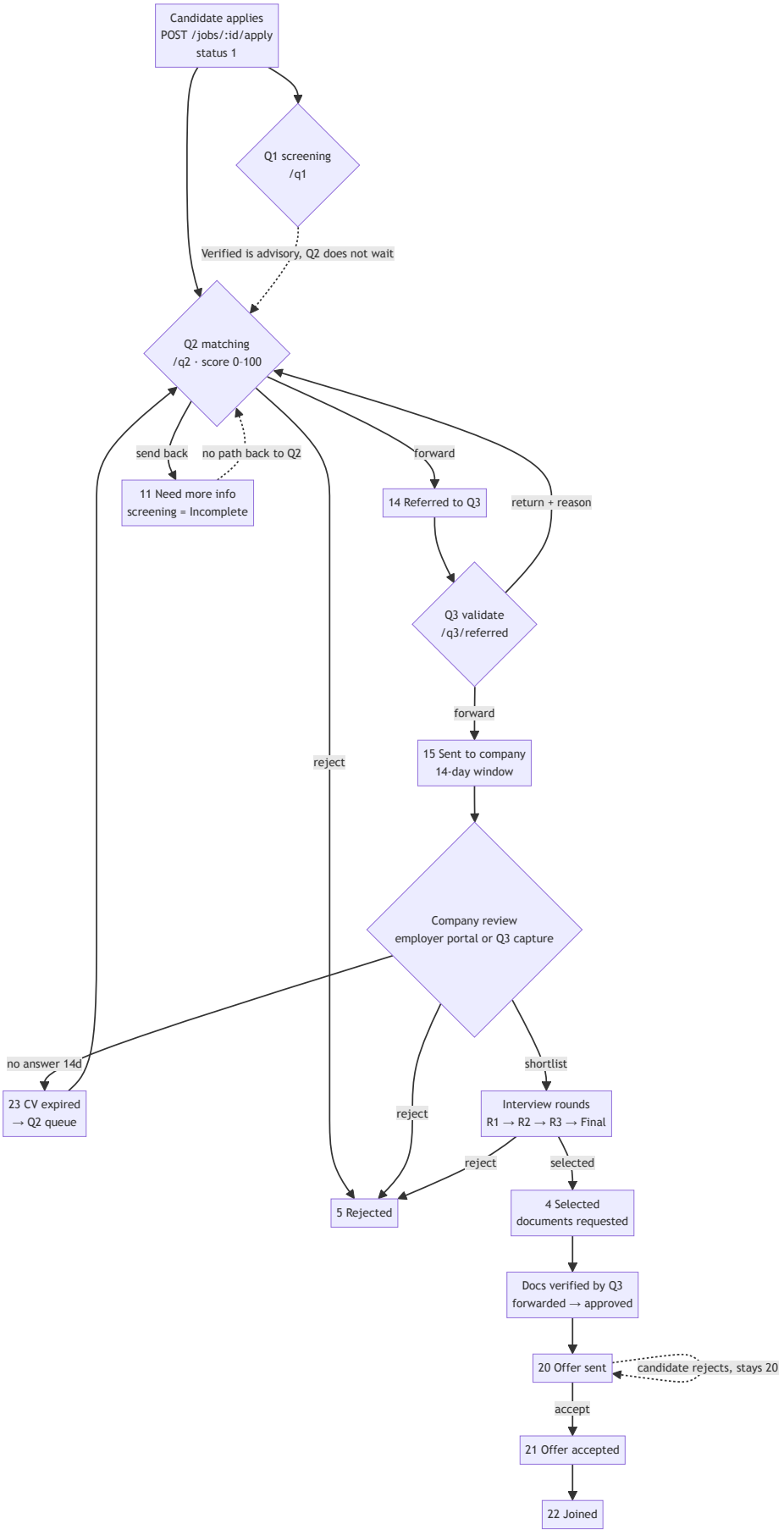
1. Master flow



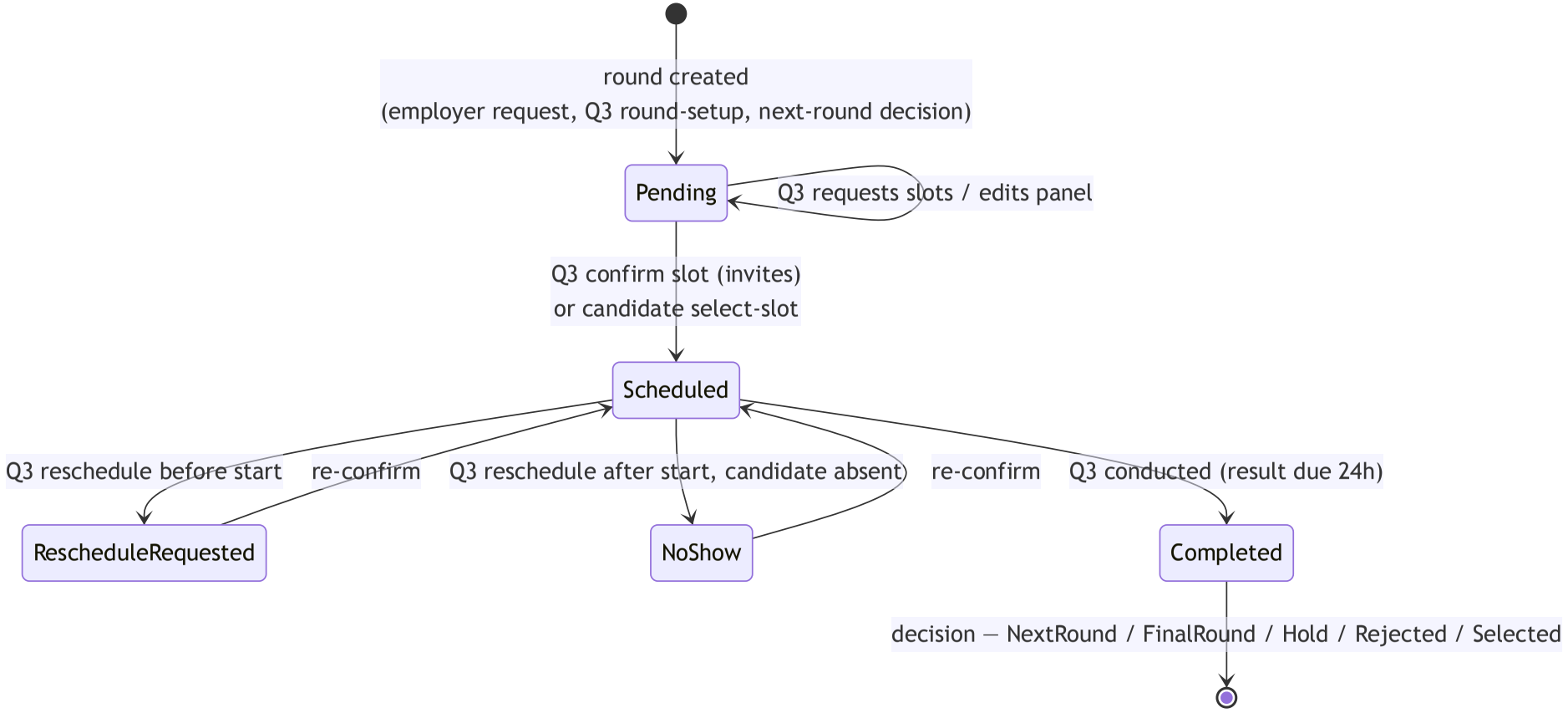
How to read it

1. **Roles → login.** Candidates self-register with email OTP. Employers submit a registration that an admin must approve. Staff accounts are created by an admin.
2. **Guards.** Every API call passes the throttler, the JWT check and the role check (`apps/api/src/app.module.ts`). On the web, `ProtectedRoute` sends each role to its home (`apps/web/src/types/roles.ts`).
3. **Dashboards → modules.** Each workspace calls its own API module. Shared hiring actions live in `/recruitment` .
4. **Status writes.** All application status changes funnel through `JobApplicationsService.transitionStatus` (`apps/api/src/modules/jobs/job-application.service.ts:167`). It writes the mapping, a status log row and the latest-status table in one transaction.
5. **Async.** Emails go through the BullMQ `notifications` queue. The hourly `cv-expiry` job returns unreviewed CVs to Q2 after 14 days.
6. **Outcomes.** These are joined, rejected, expired (re-queued), subscription active and employer approved.

2. Hiring pipeline: hand-offs between desks

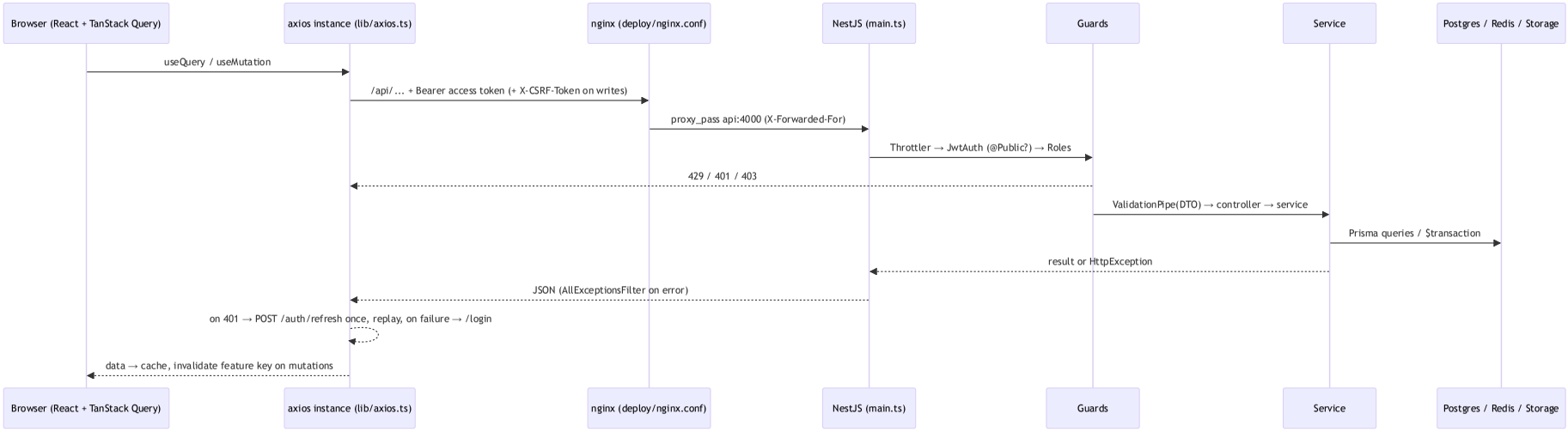


3. Interview round lifecycle (one round)

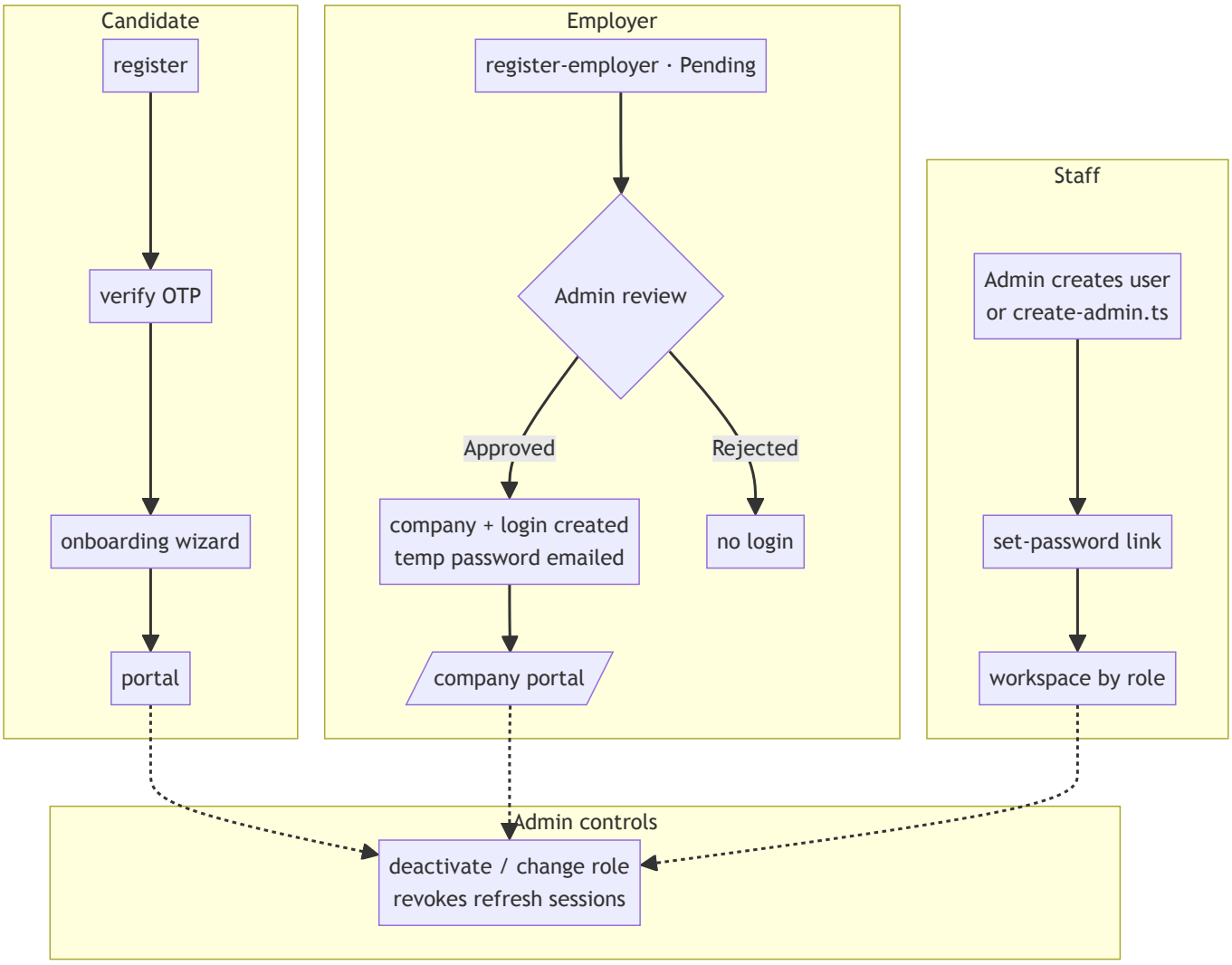

round created
(employer request, Q3 round-setup, next-round decision)
Pending
Q3 requests slots / edits panel
Q3 confirm slot (invites)
or candidate select-slot
Scheduled
Q3 reschedule before start
re-confirm
Q3 reschedule after start, candidate absent
re-confirm
Q3 conducted (result due 24h)
RescheduleRequested
NoShow
Completed
decision — NextRound / FinalRound / Hold / Rejected / Selected

Two writers record outcomes. Q3 uses `decision`, `decidedAt`, `isFinal` and `ApplicationHiring`. The employer and `/recruitment` path (`submitResult`) uses `result` only. That split is the root cause of TESTING_AND_ISSUES W-01.

4. Request lifecycle (any API call)


Browser (React + TanStack Query)
axios instance (lib/axios.ts)
nginx (deploy/nginx.conf)
NestJS (main.ts)
Guards
Service
Postgres / Redis / Storage
useQuery / useMutation
/api/... + Bearer access token (+ X-CSRF-Token on writes)
proxy_pass api:4000 (X-Forwarded-For)
Throttler → JwtAuth (@Public?) → Roles
429 / 401 / 403
ValidationPipe(DTO) → controller → service
Prisma queries / $transaction
result or HttpException
JSON (AllExceptionsFilter on error)
on 401 → POST /auth/refresh once, replay, on failure → /login
data → cache, invalidate feature key on mutations
Browser (React + TanStack Query)
axios instance (lib/axios.ts)
nginx (deploy/nginx.conf)
NestJS (main.ts)
Guards
Service
Postgres / Redis / Storage

5. Account lifecycle per role



Related documents: COMPLETE_WORKFLOW.md (step-by-step), ROLE_PERMISSION_MATRIX.md, API_DOCUMENTATION.md, DATABASE_SCHEMA.md.